

Reinforcement Learning-Based Hummingbird Agent Simulation Using Unity ML-Agents

TABLE OF CONTENTS

ABSTRACT.....	v
LIST OF TABLES	vii
LIST OF FIGURES	viii
ABBREVIATIONS	x
CHAPTER 1 INTRODUCTION	1
1.1 Background of the Study.....	1
1.2 Problem Statement	2
1.3 Objectives of the Project	4
1.4 Scope of the Project	4
1.5 Significance of the Project	5
CHAPTER 2 LITERATURE SURVEY	8
2.1 Review of Existing Systems and Research Works	8
2.2 Comparative Analysis	10
2.3 Research Gap and Limitations of Existing Systems	12
2.4 Proposed System and Proposed Approach.....	13
CHAPTER 3 SOFTWARE REQUIREMENT ANALYSIS	15
3.1 Hardware Specification.....	15
3.2 Software Specification	16
3.3 Software Tools and Their Features	18
3.3.1 Unity 6 and the Universal Render Pipeline.....	18
3.3.2 Unity ML-Agents Toolkit	18
3.3.3 Unity Inference Engine	18
3.3.4 mlagents-learn and PyTorch	18
3.3.5 TensorBoard.....	19
3.3.6 ONNX	19
3.3.7 Anaconda, Xvfb and the Jarvis Labs platform.....	19
3.3.8 IL2CPP and the Android toolchain	19
CHAPTER 4 SYSTEM ANALYSIS	21
4.1 Requirement Specification	21
4.1.1 Functional Requirements	21

4.1.2 Non-Functional Requirements	22
4.2 Characteristics of the Existing System.....	23
4.3 Proposed System	24
4.4 Feasibility Study	25
4.4.1 Technical Feasibility	25
4.4.2 Economic Feasibility.....	25
4.4.3 Operational Feasibility	25
4.4.4 Schedule Feasibility	26
4.5 Software Requirement Specification.....	26
4.5.1 Product Backlog.....	26
4.5.2 Objectives and User Stories by Sprint	26
4.5.3 User Stories for the Extended Work	28
4.5.4 Interface Requirements and Design Constraints	28
4.5.5 Requirement Traceability.....	29
CHAPTER 5 SYSTEM DESIGN	30
5.1 System Architecture and Overall Workflow	30
5.1.1 Environment and Agent Architecture	31
5.1.2 Training Architecture	32
5.1.3 Minigame Architecture	33
5.1.4 Rented-GPU Training and Deployment Architecture.....	34
5.2 Context Diagram	34
5.3 Use Case Diagram.....	35
5.4 Activity and State Diagrams	36
5.5 Data Flow, Class and Machine Learning Workflow Diagrams	37
5.5.1 Data Flow Diagrams	37
5.5.2 Class Diagram	39
5.5.3 Machine Learning Workflow	40
5.6 Dataset Design	41
5.7 User Interface Design.....	43
CHAPTER 6 SYSTEM IMPLEMENTATION	45
6.1 Modules Description	45
6.2 Dataset Description and Data Collection	46
6.3 Data Preprocessing.....	47
6.4 Feature Engineering and Feature Extraction.....	47
6.5 Model Architecture and Algorithm Design.....	48
6.6 System and Model Implementation	50

6.6.1 Agile Methodology	50
6.6.2 Project Setup	51
6.6.3 Environment Scripts.....	52
6.6.4 Agent Script	54
6.6.5 Training and Inference on the Laptop	57
6.6.6 Minigame	59
6.6.7 Retraining on Rented Cloud Hardware	61
6.6.8 Android Build and On-Device Deployment	63
6.7 Validation Checks	65
6.8 User Interface Implementation.....	66
CHAPTER 7 TESTING AND EVALUATION	68
7.1 Test Cases	68
7.2 Unit Testing.....	69
7.3 Integration Testing	70
7.4 System Testing.....	71
7.5 Model Evaluation	72
7.5.1 On-Device Baseline, HB_01	72
7.5.2 Theoretical Ceiling and Interpretation of the Reward.....	74
7.5.3 Why HB_01 Stopped at 41.29, and the HB_16 Retraining	74
7.5.4 Training Diagnostics	79
7.6 Performance Metrics	79
7.7 Comparative Analysis	81
7.7.1 On-Device versus Rented Training.....	81
7.7.2 Comparison Against a Public Reimplementation	81
7.7.3 Throughput Across Process and Arena Layouts	82
7.7.4 Headless Rendering and CPU versus GPU	84
CHAPTER 8 RESULTS AND CONCLUSION.....	85
8.1 Results	85
8.2 Discussion and Analysis	87
8.3 Limitations	89
8.4 Lessons Learnt	90
8.4.1 Sprint I, Project Setup and Foundation	90
8.4.2 Sprint II, Core Environment Scripting.....	90
8.4.3 Sprint III, Agent Development and Scene Wiring.....	91
8.4.4 Sprint IV, Training and Inference	91
8.4.5 Sprint V, Minigame and Course Wrap-up	92

8.4.6 Development Environment	93
8.4.7 Extended Work on Rented Hardware and Android	94
8.4.8 Personal Learning Outcomes	95
8.5 Conclusion	96
8.6 Future Enhancements	98
CHAPTER 9 APPENDICES	100
Appendix 1 Plagiarism Certificate	100
Appendix 2 Screenshots	100
Appendix 3 Sample Coding	102
A3.1 HummingbirdAgent.cs	102
A3.2 Flower.cs	112
A3.3 FlowerArea.cs	115
A3.4 IslandSpawner.cs	118
A3.5 DemoController.cs	121
A3.6 trainer_config_16.yaml (settings section)	123
Appendix 4 User Documentation	125
i) Installation Instructions	125
ii) README and Instructions for Using the System	126
Appendix 5 Glossary	128
CHAPTER 10 BIBLIOGRAPHY / REFERENCES	131

ABSTRACT

Designing believable, adaptive non-player character (NPC) behaviour is a long-standing challenge in game artificial intelligence. Traditional approaches such as finite state machines and behaviour trees require every possible response to be explicitly hand-authored, and cannot improve beyond what was originally programmed. This project explores an alternative, learning-based approach by training an autonomous hummingbird agent inside the Unity game engine using the Unity ML-Agents Toolkit, following Unity Learn's official "ML-Agents: Hummingbirds" course as a structured base curriculum, executed and adapted end-to-end for Unity 6 (6000.4.7f1) and Unity ML-Agents 4.1.0.

The agent (a simulated hummingbird placed on a floating island scattered with nectar-bearing flowers) is trained using Proximal Policy Optimization (PPO), a state-of-the-art on-policy deep reinforcement learning algorithm, to autonomously locate and feed from flowers while avoiding the island's boundary. The agent perceives its environment through ten vector observations (local rotation, direction to nearest flower, alignment and distance cues) supplemented by ray-perception sensors, and acts through a five-dimensional continuous action space governing 3D movement force, pitch, and yaw. All C# scripts (Flower.cs, FlowerArea.cs, HummingbirdAgent.cs) were written and adapted from the 2019.3-era course transcript to the current ML-Agents 4.x ActionBuffers API, alongside several genuine Unity 6-specific and hardware-specific engineering obstacles that were diagnosed and resolved during development.

Training was carried out entirely on integrated (CPU-only) graphics due to a disabled discrete GPU, using a dedicated Python 3.10.12 environment and PyTorch (CPU build). After scaling the environment down to a single floating island to fit within an 8GB RAM budget, the agent was trained for 5,000,024 steps, reaching a final Mean Reward of 41.29, substantially exceeding a comparable public reimplementation of the same tutorial benchmarked at this project's own request. The trained policy was exported to ONNX format and verified to run correctly under Unity's Inference Engine with no live Python connection, and was further deployed into a second, competitive two-player minigame scene where the trained agent competes against a human, keyboard-controlled player for nectar collection under a shared time limit. A subsequent training run extended this result using rented cloud GPU infrastructure, reaching a final Mean Reward of 96.58 over 50,000,000 steps; this extended run and its comparison against the on device baseline are presented in Chapter 7.

LIST OF TABLES

Table 2.1 Comparison of traditional game-AI techniques vs. reinforcement learning ..	11
Table 2.2 Comparison of reinforcement learning algorithms relevant to this project ...	11
Table 3.1 Hardware used in the project	15
Table 3.2 Pinned server software versions (setup_server.sh) and the failure each prevents	17
Table 3.3 Summary of tools, versions and their use in the project	20
Table 4.1 Product backlog, key user stories with desired outcomes	26
Table 4.2 Additional user stories added after the base course	28
Table 4.3 Requirement traceability from requirement to design and test	29
Table 5.1 Observation design (HB_16, 66 values per decision)	42
Table 5.2 Action design (5 continuous values in the range -1 to +1).....	42
Table 5.3 Reward design (unchanged between HB_01 and HB_16).....	42
Table 6.1 Modules of the Unity project	45
Table 6.2 PPO trainer configuration of HB_01 (trainer_config.yaml) and HB_16 (trainer_config_16.yaml)	50
Table 6.3 Sprint-wise summary of objectives and outcomes	50
Table 7.1 Test cases and results	68
Table 7.2 Model integrity tests (TC08 to TC10).....	70
Table 7.3 Checkpoint rewards recorded during the HB_01 training run	72
Table 7.4 Key values recorded during the HB_16 training run	77
Table 7.5 Performance metrics of the system	79
Table 7.6 Time and throughput comparison between on device training (HB_01) and rented cloud GPU training (HB_16)	81
Table 7.7 Final result comparison against a public tutorial reimplementation	81
Table 7.8 Measured throughput of process and island layouts on the L4 instance	82
Table 9.1 Controls in the Unity Editor	127
Table 9.2 Buttons in the Android application	127
Table 9.3 Glossary of terms used in the report.....	128

LIST OF FIGURES

Fig. 1.1 Traditional rule-based NPC AI vs. reinforcement-learning-based agent	3
Fig. 2.1 Agent, environment and policy interaction loop in reinforcement learning.....	9
Fig. 5.1 Overall system architecture of the Hummingbird ML-Agents system	31
Fig. 5.2 Component diagram of the Unity Editor and Python trainer communication architecture	33
Fig. 5.3 Context diagram of the Hummingbird ML-Agents system.....	34
Fig. 5.4 Use case diagram of the Hummingbird ML-Agents system actors and interactions	35
Fig. 5.5 Activity diagram of one training episode in HummingbirdAgent.cs	36
Fig. 5.6 State diagram of the minigame GameManager	37
Fig. 5.7 Level 0 data flow diagram	38
Fig. 5.8 Level 1 data flow diagram of one training step	39
Fig. 5.9 Class diagram of the Unity project	40
Fig. 5.10 Machine learning workflow followed in the project	41
Fig. 5.11 User interface layouts: (a) minigame HUD, (b) Android demo	44
Fig. 6.1 Policy network architecture of the HB_16 model	49
Fig. 6.2 The Training scene: one floating island with 120 flower plants and 360 flowers	52
Fig. 6.3 The hummingbird agent beside a flower plant in the Training scene	55
Fig. 6.4 Hummingbird agent colliders, agent parameters and Behavior Parameters in the Inspector.....	56
Fig. 6.5 Ray Perception Sensor 3D gizmos visible in the Scene view	56
Fig. 6.6 The nine ray sensors rendered from their own ray geometry	57
Fig. 6.7 mlagents-learn running in a terminal, waiting for the Unity Editor	58
Fig. 6.8 Trained agent performing inference in the Editor with no Python trainer connected.....	59
Fig. 6.9 Flower Island minigame, player vs. trained opponent, with the live UI	60
Fig. 6.10 The Flower Island minigame scene rendered from outside the island	60
Fig. 6.11 The Flower Island scene in the Unity Editor.....	61
Fig. 6.12 Rented-GPU training pipeline from laptop build to downloaded model.....	61
Fig. 6.13 The 28-island grid built by IslandSpawner for one training process.....	62
Fig. 7.1 HB_01 mean cumulative reward with saved checkpoints marked (0 to 5,000,024 steps)	73
Fig. 7.2 TensorBoard view of HB_01's Environment/Cumulative Reward	73

Fig. 7.3 (a) Learning-rate schedule and (b) policy entropy of both runs against progress through max_steps	75
Fig. 7.4 HB_16 mean cumulative reward (0 to 50,000,000 steps)	77
Fig. 7.5 Mean cumulative reward of HB_01 and HB_16 over (a) the full budgets and (b) the first 5M steps.....	78
Fig. 7.6 (a) Value estimate and (b) policy loss of both runs.....	79
Fig. 7.7 Wall-clock breakdown of HB_01's final training session	80
Fig. 7.8 Throughput of six ways of arranging the same 28 arenas	83
Fig. 7.9 Throughput against total number of parallel arenas	83
Fig. 7.10 (a) Average throughput of both runs, (b) effect of -batchmode, (c) CPU vs CUDA training.....	84
Fig. 9.1 Android demo on the Galaxy S24 Ultra, start screen	100
Fig. 9.2 Android demo with the debug view switched on (rays and the ten observations)	101
Fig. 9.3 Android demo split-screen race between the HB_16 and HB_01 models with trails.....	101
Fig. 9.4 Top view of the floating island, showing the flower plants spread across it...	102
Fig. 9.5 Close view of a flower plant; each bloom carries a nectar trigger collider	102

ABBREVIATIONS

Abbreviation	Expansion
ADB	Android Debug Bridge
AI	Artificial Intelligence
API	Application Programming Interface
APK	Android Package
ARM	Advanced RISC Machines (processor architecture)
CPU	Central Processing Unit
CUDA	Compute Unified Device Architecture
DFD	Data Flow Diagram
DQN	Deep Q-Network
DRL	Deep Reinforcement Learning
FSM	Finite State Machine
GAE	Generalized Advantage Estimation
GLX	OpenGL Extension to the X Window System
GPU	Graphics Processing Unit
gRPC	Google Remote Procedure Call
HUD	Heads-Up Display
IL2CPP	Intermediate Language to C++
LR	Learning Rate
MCA	Master of Computer Applications
MD5	Message Digest 5 (checksum)
ML	Machine Learning
MLP	Multilayer Perceptron
NPC	Non-Player Character
ONNX	Open Neural Network Exchange
PPO	Proximal Policy Optimization
RAM	Random Access Memory
RL	Reinforcement Learning
SDG	Sustainable Development Goal
SDK	Software Development Kit
SRM	SRM Institute of Science and Technology
SRS	Software Requirement Specification
SSH	Secure Shell
TRPO	Trust Region Policy Optimization
UI	User Interface
UML	Unified Modeling Language
URP	Universal Render Pipeline
vCPU	Virtual Central Processing Unit
VRAM	Video Random Access Memory
Xvfb	X Virtual Framebuffer
YAML	YAML Ain't Markup Language

CHAPTER 1

INTRODUCTION

1.1 Background of the Study

This project presents the design, development, and training of an autonomous reinforcement learning (RL) agent within the Unity game engine, using the Unity ML-Agents Toolkit. The project implements a simulated hummingbird agent that learns, through trial-and-error interaction with its environment, to locate and feed from nectar-bearing flowers scattered across a floating island environment, while avoiding boundary collisions.

The agent is trained using Proximal Policy Optimization (PPO), a policy-gradient reinforcement learning algorithm, through Unity's ML-Agents Python trainer (`mlagents-learn`), which communicates with the Unity Editor via a gRPC-based communicator. The agent perceives its environment through a combination of nine ray-perception raycasts (an undifferentiated proximity sense of nearby geometry and of the island boundary) and ten vector observations (orientation, direction, alignment and distance to the nearest flower), and acts through a continuous, five-dimensional action space governing its movement force, pitch, and yaw.

The project was carried out by following Unity Learn's official "ML-Agents: Hummingbirds" course (originally authored by Adam Kelly / Immersive Limit on Unity 2019.3) unit by unit as a structured base curriculum, while independently re-deriving every piece of code and configuration for the current environment: Unity 6.4 (6000.4.7f1) and Unity ML-Agents Toolkit 4.1.0. This required a substantial and genuine migration effort, since the underlying C# scripting API, Python package ecosystem, and trained-model file format have all changed materially since the course was originally recorded.

Beyond the base tutorial scope, the trained model was also deployed in a second, competitive minigame scene, where the trained agent (running in pure inference mode, without any live Python connection) competes against a human, keyboard-controlled player for nectar collection within a fixed time limit, demonstrating that the trained policy generalizes from its training conditions to a distinct, gameplay-oriented deployment context.

The project did not stop at the base curriculum. Once the first training run (identified as HB_01) had finished on the project laptop, its logs were analysed to find out why it had stopped at a mean reward of 41.29. Two concrete causes were identified: the run had simply exhausted its step budget while still improving, and the agent had no way to observe its own velocity. A second run (HB_16) was then designed to address both causes without touching the reward function, and was trained for 50,000,000 steps on a rented cloud machine with 112 parallel training arenas. It reached a final mean reward of 96.58, with a best recorded value of 97.728, which is 2.35 times the on-device baseline.

Finally, the trained models were packaged into an Android application and tested on a Samsung Galaxy S24 Ultra. The application lets a viewer switch between the two trained models while the bird is flying, race them against each other in split screen, and switch on a debug view that draws the agent's ray sensors and prints its ten vector observations live. This turns the difference between the two training runs from a pair of numbers into behaviour that can be watched and questioned.

1.2 Problem Statement

Traditional game AI relies on hand-authored, rule-based behaviour trees or finite state machines, which require explicit programming of every possible agent response and do not adapt or improve with experience. Designing believable, adaptive non-player character (NPC) behaviour (such as a creature that must learn to efficiently locate scarce, moving resources in a continuous 3D environment) is difficult to hand-code convincingly, and such systems cannot improve past what was explicitly designed into them.

Fig. 1.1 contrasts the two approaches: a rule-based character follows a fixed decision structure written in advance, while a reinforcement learning agent improves its behaviour from the reward it receives.

Traditional Rule-Based NPC AI vs. Reinforcement-Learning-Based Agent

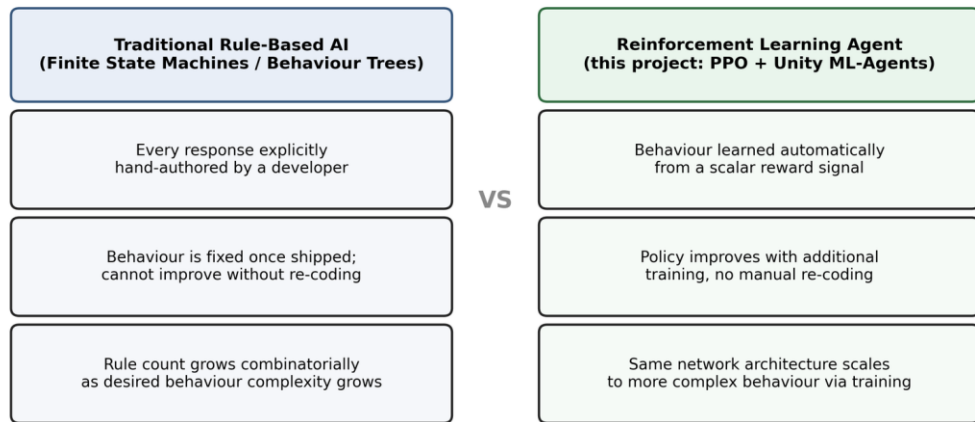


Fig. 1.1 Traditional rule-based NPC AI vs. reinforcement-learning-based agent

This project addresses the problem of training an agent to learn efficient foraging behaviour (nectar-seeking flight) purely from a numeric reward signal, without any hand-coded pathfinding or decision logic, within an environment characterised by sparse and non-uniformly distributed resources (flowers that deplete and must be re-located), continuous physics-based 3D movement (rigidbody-driven flight rather than grid-based movement), and hard environmental constraints (an island boundary that must be avoided). A secondary, practically significant problem addressed by this project is that of toolkit and API migration: adapting a multi-year-old, version-specific tutorial codebase to a current game engine release and a current version of its associated machine learning toolkit, which is a realistic and common challenge in applied software and ML engineering.

A third problem emerged during the work itself and is treated as part of the problem statement rather than as an afterthought: how to scale reinforcement learning training beyond a consumer laptop in a controlled and affordable way. The laptop reached roughly 44 training steps per second. Answering the question of how much better the same agent could become therefore required moving training to rented hardware, and doing so exposed a chain of practical problems (headless rendering, pinned Python dependencies, throughput tuning and checkpoint recovery) that each had to be diagnosed and solved.

1.3 Objectives of the Project

The objectives below were fixed at the start of the project, with the last two added once the base course was complete and the first training run had been analysed.

- To implement a complete Unity ML-Agents pipeline (environment, agent script, reward function, and training configuration) for a continuous-control foraging task, adapted correctly from a legacy tutorial to a current engine and toolkit version.
- To train a PPO policy to convergence under real, consumer-hardware constraints (CPU-only training, limited RAM) and to document the practical adaptations required to do so.
- To verify that the trained policy exports correctly and performs correctly under pure inference (no live training connection), including in a second, independently-built deployment context (the competitive minigame).
- To identify, diagnose, and resolve genuine software-engineering obstacles encountered along the way (deprecated APIs, dependency conflicts, Unity-version-specific bugs), and to document root causes and fixes rather than only workarounds, as a record of the underlying engineering process.
- To identify, from the first run's own logs, why its reward stopped at 41.29, and to retrain the agent on rented cloud hardware in a way that improves the policy without changing the reward function, the episode length or the bird's physics, so that the new score remains directly comparable with the old one.
- To package both trained models into an Android application that runs entirely on the phone, supports switching between the models at runtime, and exposes the agent's perception through an on-screen debug view.

1.4 Scope of the Project

The scope of this project is a single-agent, continuous-control foraging task implemented in the Unity 6 engine and trained with the Proximal Policy Optimization algorithm from the Unity ML-Agents Toolkit. Within this scope the project covers the complete lifecycle of a reinforcement learning system: environment and agent scripting, observation, action and reward design, training configuration, training on two very

different hardware platforms, evaluation from the trainer's own logs, export of the trained policy to the ONNX format, and deployment of that policy in three separate contexts (the Unity Editor, a two-player minigame, and an Android application).

The following items are explicitly inside the scope of the work reported here:

- the three environment scripts (Flower.cs, FlowerArea.cs, HummingbirdAgent.cs) migrated to Unity 6 and ML-Agents 4.1.0;
- two complete PPO training runs (HB_01 on the laptop CPU and HB_16 on a rented NVIDIA L4 instance), with a short random-policy baseline run for reference;
- the tooling written to make the rented run possible: a runtime island spawner, command-line Linux and Android builds, a server setup script and a training script with smoke, sweep, run and status modes;
- the Flower Island minigame, the Android demo application and the presentation toolkit (debug view, flight trails, split-screen race and touch camera);
- measured throughput benchmarks across process and arena layouts, and comparisons against a public reimplementation.

The following items are outside the scope of this report and are recorded as future work in Chapter 8: multi-agent or self-play training, curriculum learning, procedurally generated or AI-generated garden layouts, a formal generalisation study on unseen layouts, and any change to the reward function or episode length. The last of these is excluded deliberately, because changing how reward is earned would make every new score incomparable with the HB_01 baseline.

1.5 Significance of the Project

The motivation for this project is threefold. First, reinforcement learning is a core pillar of modern applied AI, and the MCA Generative AI specialization benefits from hands-on exposure to the full RL pipeline (environment design, reward shaping, policy training, and deployment) rather than RL studied only as theory. Second, Unity ML-Agents was chosen specifically because it uses a widely-adopted, industry-relevant game engine as the simulation substrate, offering direct, practical experience with the same class of tools used in applied robotics simulation, game AI research, and simulation-to-

real transfer work. Third, this project was deliberately undertaken as a from-scratch, hands-on build, writing and understanding every script rather than importing a pre-trained solution, specifically to build genuine, defensible technical depth ahead of AI/ML-focused campus placements, where the ability to explain a working system end-to-end (not just describe it) is a meaningful differentiator.

This project's primary contribution is pedagogical and skill-building in nature, aligning most directly with SDG 4, Quality Education. The project constitutes an applied, hands-on exploration of reinforcement learning concepts (reward shaping, policy optimization, observation and action space design) that are directly relevant to a Generative AI specialization, executed using a self-contained, well-documented open-source toolkit. The substantial debugging and root-cause analysis work performed over the course of the project, resolving version-compatibility issues, migrating a deprecated scripting API, and working within genuine hardware constraints, reflects a real, transferable skills-development outcome consistent with SDG 4's emphasis on building technical and vocational skills relevant to employment.

Secondarily, the project touches SDG 9 (Industry, Innovation and Infrastructure) by working directly with an applied AI/ML toolchain spanning simulation, training, and inference deployment, reflecting the kind of applied innovation infrastructure skillset the SDG targets at a project scale.

More specifically, this project maps onto SDG Target 4.4, which calls for substantially increasing the number of people with relevant technical and vocational skills for employment; the reinforcement learning pipeline built here (spanning environment design, agent scripting, training, and inference deployment) constitutes exactly this kind of applied, employment-relevant technical skill, developed through direct practice rather than passive study. It is also consistent with Target 4.7's emphasis on education that equips learners with the knowledge and skills needed to engage with emerging technology responsibly, since a meaningful portion of the project's effort went into understanding and correctly resolving the practical, real-world engineering issues that arise when adopting a fast-moving AI toolkit rather than treating it as a black box.

From an SDG 9 perspective, the project's engineering record (documented in Chapters 6 and 8) is itself a small-scale illustration of the kind of applied innovation

infrastructure work the goal targets: adapting existing tools to new technical environments, diagnosing dependency and compatibility issues, and building a working, deployable AI pipeline on constrained consumer hardware rather than specialised infrastructure. This is directly relevant to the broader industry context the MCA Generative AI specialization prepares students for, where the ability to build and maintain applied AI systems under real-world constraints is as valuable as theoretical understanding of the underlying algorithms. The significance of the extended work is similar in kind: the rented-GPU run and the Android application show the same pipeline scaling up to a server with 112 parallel arenas and down to a phone with no network connection, which is the range over which applied reinforcement learning systems are actually deployed.

The rest of this report follows the structure prescribed by the department. Chapter 2 surveys the literature and positions the proposed approach. Chapter 3 lists the hardware and software used and the features of each tool. Chapter 4 analyses the requirements, the existing approaches and the feasibility of the project, and Chapter 5 presents the system design through architecture, context, use case, activity, state, data flow and class diagrams. Chapter 6 describes the implementation, including the model architecture and the rented-GPU and Android pipelines, and Chapter 7 reports the testing and the full evaluation of both training runs. Chapter 8 discusses the results, limitations and lessons learnt and concludes the report, Chapter 9 holds the appendices and Chapter 10 lists the references.

CHAPTER 2

LITERATURE SURVEY

2.1 Review of Existing Systems and Research Works

Artificial intelligence for non-player characters (NPCs) in games has historically been dominated by hand-authored techniques: finite state machines (FSMs), behaviour trees, and scripted decision rules. Yannakakis and Togelius [19] provide a comprehensive survey of these classical approaches alongside the more recent shift toward learning-based game AI, noting that hand-authored systems scale poorly with behavioural complexity and cannot improve beyond their designed rule set. Colledanchise and Ögren [4] similarly document behaviour trees as the dominant modular alternative to FSMs in both robotics and games, but note the same fundamental ceiling: behaviour, however modular, remains explicitly authored rather than learned.

Deep reinforcement learning (DRL) offers a fundamentally different paradigm, in which an agent learns a policy (a mapping from observations to actions) purely by interacting with an environment and receiving scalar reward signals, without any hand-coded decision logic. The modern DRL era is widely dated to Mnih et al.'s Deep Q-Network (DQN) [9], which demonstrated human-level performance on Atari games using only raw pixel input and a scalar reward, and was followed by landmark results such as AlphaGo [15] in the game of Go and, more recently, large-scale multi-agent systems such as OpenAI Five for Dota 2 [2] and AlphaStar for StarCraft II [18]. Both of these, notably, are built on general-purpose game engines and simulation environments rather than bespoke physical hardware, reinforcing the suitability of game engines as RL research and application platforms.

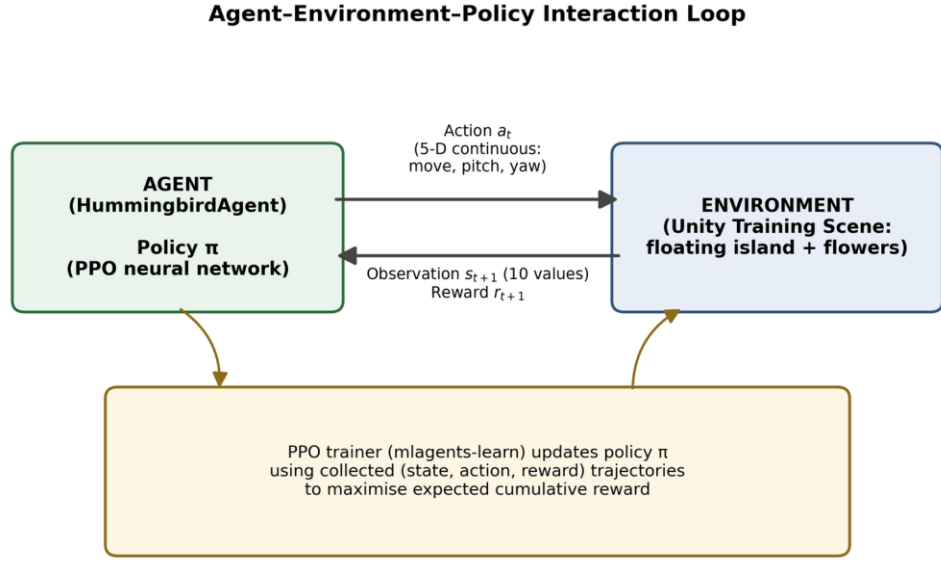


Fig. 2.1 Agent, environment and policy interaction loop in reinforcement learning

The formal foundation for all of these systems is the Markov decision process described by Sutton and Barto [16]. At each time step the agent observes a state, chooses an action according to its policy, receives a scalar reward and moves to a new state, as shown in Fig. 2.1. The agent's goal is to maximise the expected discounted sum of future rewards, and the discount factor (gamma, 0.99 in this project) controls how far ahead it effectively plans. Two families of methods follow from this definition. Value-based methods such as DQN [9] learn how good each action is and act greedily, which works well for small discrete action sets. Policy-gradient methods learn the policy directly and extend naturally to continuous actions such as the force and rotation outputs of a flying bird.

Standard benchmark suites made these algorithms comparable. OpenAI Gym [3] defined a common interface between environments and learning code, and asynchronous actor-critic methods such as A3C [10] showed that running many copies of an environment in parallel both speeds up training and decorrelates the collected experience. That second idea is used directly in this project: the rented-GPU run trains one policy from 112 arenas at once.

Unity Technologies formalised this suitability with the release of the Unity ML-Agents Toolkit, described by Juliani et al. [6] as a general platform enabling any Unity-

authored simulation to serve as a training environment for intelligent agents via a Python-based training API. This is the toolkit used throughout the present project.

The toolkit's own documentation [17] describes the parts used here: the Agent base class with its Initialize, OnEpisodeBegin, CollectObservations, OnActionReceived and Heuristic callbacks, the Behavior Parameters component that fixes the observation and action shapes, the Decision Requester that sets how often a decision is made, ray perception sensors, and the mlagents-learn trainer that runs outside Unity and exports ONNX models. The project itself follows the Unity Learn course by Kelly [7], which provided the environment assets and the original 2019-era scripts that had to be migrated.

2.2 Comparative Analysis

The reinforcement learning algorithm underlying this project is Proximal Policy Optimization (PPO), introduced by Schulman et al. [14]. PPO is an on-policy, actor-critic policy-gradient method that constrains each policy update to remain close to the previous policy using a clipped surrogate objective, avoiding the destructively large updates of earlier policy-gradient methods while remaining simpler to implement than its predecessor, Trust Region Policy Optimization (TRPO) [12]. PPO's advantage estimates are commonly computed using Generalized Advantage Estimation (GAE) [13], which trades off bias and variance in the estimated advantage via an exponentially-weighted combination of n-step returns; both are used, by default, inside Unity ML-Agents' PPO trainer, and both were used, without modification, in this project's training runs.

Reward design follows the classical reward-shaping framework of Ng, Harada, and Russell [11], who formally established the conditions under which a shaping reward can be added to a sparse base reward without changing the optimal policy. The HummingbirdAgent's reward function reflects this directly: a small positive reward per tick of nectar contact (with an alignment bonus for approaching the flower head-on), and a negative reward for boundary collisions, both designed to guide the policy without altering its underlying objective.

Curriculum learning, as formalised by Bengio et al. [1], proposes training on progressively harder examples rather than the full task distribution from the outset. This

project follows the base course's flat training schedule, but the concept is directly relevant to a parked future-work item in Chapter 8.

Multi-agent extensions of these algorithms (such as MADDPG [8]) become relevant in the Minigame unit (Chapter 6), where the trained single-agent policy is deployed in a two-agent competitive setting, although no multi-agent training algorithm was itself used; the opponent is simply the same policy running in pure inference mode against a human player.

Table 2.1 summarises the qualitative differences between the traditional, hand-authored game-AI techniques discussed above and the reinforcement-learning-based approach used in this project.

Table 2.1 Comparison of traditional game-AI techniques vs. reinforcement learning

Aspect	Finite State Machines / Behaviour Trees	Reinforcement Learning (this project)
Behaviour source	Explicitly hand-authored rules	Learned from reward signal
Adaptability	Fixed; cannot improve without re-coding	Improves with additional training
Scaling with complexity	Rule count grows combinatorially	Same network architecture, more training
Design effort	Manual authoring of every case	Reward function + observation/action design
Predictability	Fully deterministic, easy to debug	Emergent; requires empirical evaluation

Among reinforcement learning algorithms themselves, the choice of PPO is explained by the comparison in Table 2.2. PPO is not the most sample-efficient method available, but it handles continuous actions, is stable with default hyperparameters and is the default trainer in ML-Agents. That makes it the right choice for a project whose aim is a working, well-understood pipeline rather than a new algorithm.

Table 2.2 Comparison of reinforcement learning algorithms relevant to this project

Algorithm	Type	Action space	Main strength	Main weakness
DQN [9]	Value-based, off-policy	Discrete only	Sample reuse via replay buffer	Cannot output continuous force
A3C [10]	Actor-critic, on-policy	Discrete or continuous	Parallel workers, fast wall clock	Sensitive to step size

Algorithm	Type	Action space	Main strength	Main weakness
TRPO [12]	Policy gradient, on-policy	Continuous	Guaranteed monotonic improvement	Second-order, complex to implement
PPO [14]	Policy gradient, on-policy	Continuous	Clipped objective, stable, simple	Discards data after each update
MADDPG [8]	Actor-critic, multi-agent	Continuous	Handles competing agents	Needs centralised critic, heavier

2.3 Research Gap and Limitations of Existing Systems

Three practically-relevant gaps were identified while surveying the above literature and cross-referencing it against the base course material, and each directly shaped decisions made in this project (documented in Chapters 6 to 8):

- Version and API drift: academic papers and toolkit documentation describe algorithms in their general, version-independent form, but say little about the practical reality that a specific toolkit release (e.g. ML-Agents 4.x) may have materially different scripting APIs, model export formats, and Python package requirements than the tutorial or paper being followed. This gap had to be closed through direct migration work (Chapter 6).
- Hardware-constrained training in the literature: most published DRL results (e.g. [9], [15], [2], [18]) assume substantial GPU or distributed compute resources. Guidance for training small vector/ray-observation-based policies entirely on CPU-only, memory-constrained consumer hardware (as required by this project's disabled discrete GPU and 8GB RAM budget) is comparatively sparse in the literature, and had to be established empirically (Chapters 6 and 7).
- Benchmarking against comparable small-scale reimplementations: formal literature on PPO and Unity ML-Agents does not provide task-specific benchmark numbers for the Hummingbirds environment; the only available comparison point was a community GitHub reimplementation of the same tutorial [5], used in Chapter 7 for an approximate outcome comparison.

A fourth gap became visible only after the first training run. Tutorials and papers report final rewards, but rarely show how to read a trainer's own logs to decide whether a run has converged or has merely run out of budget. With a linear learning-rate schedule

the learning rate reaches zero exactly at the step budget, so any run that ends on budget shows a flat tail that looks like convergence. Distinguishing the two, and checking whether the observation space is sufficient for the physics being controlled, is a practical skill the literature leaves to the practitioner. Chapter 7 applies it to HB_01.

2.4 Proposed System and Proposed Approach

The proposed approach is a learned policy in place of hand-authored behaviour: the hummingbird is given only a numeric description of its surroundings and a reward signal, and PPO is used to learn a mapping from that description to continuous flight controls. The approach has four stages. First, the base course environment is rebuilt and migrated to current engine and toolkit versions. Second, a baseline policy is trained on the available laptop hardware. Third, the baseline's logs are analysed and a second policy is trained on rented hardware with an improved observation space and a larger step budget, under the constraint that reward, episode length and physics stay identical. Fourth, both policies are exported and deployed where they can be compared side by side.

The project followed the ten-unit structure of the base Unity Learn course, mapped onto five development sprints described in Section 6.6.1: Sprint I (Project Setup & Foundation), Sprint II (Core Environment Scripting), Sprint III (Agent Development & Scene Wiring), Sprint IV (Training & Inference), and Sprint V (Minigame & Course Wrap-up), summarised in Table 6.3.

This chapter surveyed the shift from hand-authored, rule-based game AI toward learning-based agents, situating this project's use of Unity ML-Agents and PPO within the broader reinforcement learning literature. Three practically-relevant gaps were identified (version and API drift between tutorials and current toolkits, sparse guidance on CPU-only training under real hardware constraints, and the absence of published benchmark figures for the Hummingbirds environment specifically), each of which directly motivated a concrete decision documented in Chapters 6 to 8. The product backlog (Table 4.1) and the five-sprint road map derived from it form the execution plan carried out in Chapter 6, where each user story listed here is traced through to a verified, working outcome.

The literature surveyed here also directly grounds four candidate directions considered for extending this project beyond the base tutorial, revisited as future work in Chapter 8: applying Bengio et al.'s curriculum learning framework [1] to a staged easy-to-hard flower layout, using generative AI to procedurally author novel garden layouts and testing the trained policy's adaptation to them; a multi-agent competitive extension building on the MADDPG line of work [8]; and a formal generalization study measuring performance drop on unseen layouts. Each was deliberately parked until the base tutorial was complete, consistent with the project's own decision to follow the base course exactly before adding original contributions.

Taken together, the proposed approach answers each of the gaps identified in Section 2.3 with a concrete practice rather than an assumption. Version and API drift is handled by migrating every script against the current toolkit's own source and documentation, and by pinning every Python dependency to a version that is known to work. The hardware gap is handled first by shrinking the environment to what the laptop can run, and then by renting only the resource the workload actually needs, measured from the trainer's own timing data. The benchmark gap is handled by comparing against the one public reimplementation while stating clearly how far that comparison can be trusted. The convergence gap is handled by reading the checkpoint history and the learning-rate schedule before drawing any conclusion from a final number. The rest of the report documents each of these practices in turn.

CHAPTER 3

SOFTWARE REQUIREMENT ANALYSIS

This chapter lists the hardware and software on which the project was developed, trained and deployed, and describes the features of each tool that the project actually relies on. Three machines are involved: the development laptop, which ran the Unity Editor and the first training run; a rented cloud server, which ran the second training run; and an Android phone, which runs the final demo application.

3.1 Hardware Specification

Table 3.1 summarises the three machines and the role each played.

Table 3.1 Hardware used in the project

Item	Development laptop	Rented server (HB_16)	Demo phone
Processor	Intel Core i5, 10th generation	28 vCPU	Snapdragon 8 Gen 3 (8 cores, ARMv9, 64-bit only)
Memory	8 GB (about 7.8 GB usable)	124 GB	about 11 GB reported by Unity
Graphics	Integrated only; NVIDIA MX130 disabled	NVIDIA L4 GPU	Adreno (on-chip)
Storage	Local SSD, 10 GB or more free	100 GB, set explicitly at rental	Internal flash
Operating system	Windows 11	Ubuntu 22.04 (Jarvis Labs PyTorch image)	Android 16
Role	Editor, scripting, HB_01 training, builds	HB_16 training, 112 arenas	Runs the 54.4 MB demo APK

The development laptop is the constraint that shaped most early decisions. Its details are listed below, exactly as they applied throughout development:

- Processor: Intel Core i5, 10th Generation.
- RAM: 8 GB total system memory.
- Graphics: Integrated graphics only. The machine's discrete GPU (NVIDIA GeForce MX130, 2GB VRAM) is permanently disabled at the Device

Manager/BIOS level due to hardware instability that previously caused system crashes, and is not re-enabled for this project under any circumstances.

- Storage: sufficient free disk space for the Unity project, ML-Agents training checkpoints, and a dedicated Python environment (10GB or more recommended).
- Display: a standard laptop display capable of running the Unity Editor.

The rented server was a Jarvis Labs instance in the IN2 region. The L4 tier was chosen over the A30 after checking every tier's live specification in the provider's picker, because the L4 bundles 28 vCPU and 124 GB of RAM for Rs 41.31 per hour while the A30 bundles only 16 vCPU for Rs 38.88 per hour. As Chapter 7 shows, training speed on this task is limited by the number of CPU cores running Unity physics, not by the GPU, so 75 percent more vCPU for about 6 percent more money was the decisive factor. Storage was set to 100 GB explicitly, because the picker's default of 20 GB could not hold the CUDA build of PyTorch, the 154.7 MB Unity build and forty kept checkpoints.

The phone is a Samsung Galaxy S24 Ultra (model SM-S928B). Its Snapdragon 8 Gen 3 is an ARMv9 processor with no 32-bit support at all, which fixed two build settings described in Chapter 6: the scripting backend must be IL2CPP and the target architecture must be ARM64 only.

3.2 Software Specification

The software used on the development laptop, which is also the environment needed to open and run the Unity project, is listed below:

- Unity Editor 6000.4.7f1 (Unity 6.4), Universal Render Pipeline (URP) template, as the game engine and simulation environment.
- `com.unity.ml-agents` version 4.1.0, the Unity-side reinforcement learning agent framework (installed explicitly by technical name to avoid the deprecated 2.0.1 default and the broken 3.0.0 release).
- `com.unity.ai.inference` (Inference Engine, shown as Sentis 2.6 in the Editor), providing on-device neural network inference; installed automatically as a dependency of ML-Agents.

- Python 3.10.12, run inside a dedicated Anaconda environment isolated from two other, unrelated Python installations (3.12.8 and 3.14.5) already present on the machine.
- mlagents (PyPI package) version 1.1.0, the Python-side PPO trainer that communicates with the Unity Editor over a local gRPC connection.
- PyTorch, CPU-only build (installed via plain pip3 install torch, with no CUDA index URL), the neural network training backend.
- Anaconda3 (2025.12-2), used for Python environment management.
- Windows 11 as the host operating system, with Microsoft Visual Studio Code used as the C# editor.
- Unity Android Build Support and Linux Build Support (Mono) modules for Unity 6000.4.7f1, used to produce the phone APK and the headless training build respectively. The Android module bundles its own copy of the Android Debug Bridge (adb), which was used to install and launch the APK.

The rented server needed its own Python environment, rebuilt from scratch by a setup script on every new instance because the provider wipes the /root directory whenever an instance is paused or destroyed. Every version in that script is pinned, and each pin exists because a newer version caused a real, diagnosed failure. Table 3.2 lists them together with the failure each one prevents.

Table 3.2 Pinned server software versions (setup_server.sh) and the failure each prevents

Component	Pinned version	Failure without the pin
Python	3.10.12	mlagents 1.1.0 accepts only 3.10.1 to 3.10.12; the image shipped 3.10.20, which pip refused
setuptools	below 81	version 81 removed pkg_resources, which mlagents still imports
PyTorch	2.2.2	the 2.13 ONNX exporter needs onnxscript, which needs numpy 1.25 or newer, which mlagents forbids
numpy	1.23.5	mlagents requires below 1.24 because it still calls the removed np.float
protobuf	3.20.3	mlagents' generated protocol files fail on 3.21 and newer
onnx	1.15.0	the exact version mlagents 1.1.0 pins
Xvfb	system package	the Unity build segfaults without a display, so a virtual 24-bit display is required

3.3 Software Tools and Their Features

This section describes each tool in terms of the specific features the project uses, rather than as a general product description.

3.3.1 Unity 6 and the Universal Render Pipeline

Unity is a real-time 3D engine with a built-in physics system (PhysX), an editor for building scenes from reusable prefabs, and a C# scripting layer. The project uses Unity 6000.4.7f1 with the Universal Render Pipeline (URP). The features that matter here are the Rigidbody physics that moves the bird, trigger and solid colliders that detect nectar contact and boundary hits, the fixed physics timestep of 0.02 seconds that sets the rhythm of the whole simulation, prefabs that let one floating island be cloned many times, and batch mode, which lets the editor be driven from the command line to produce builds without opening its window.

3.3.2 Unity ML-Agents Toolkit

ML-Agents connects Unity scenes to reinforcement learning. The Unity package (com.unity.ml-agents 4.1.0) supplies the Agent base class, the Behavior Parameters component that declares observation and action sizes, the Decision Requester that decides how often the agent acts, Ray Perception Sensor components, observation stacking, and a communicator that talks to the Python trainer over gRPC on port 5004. In a built player the communicator does not open any socket at all, which is why the phone application needs no network permission.

3.3.3 Unity Inference Engine

The Inference Engine (shown as Sentis in the Package Manager) runs ONNX neural networks inside Unity. It is installed automatically as a dependency of ML-Agents. The project runs it with the Burst CPU backend on both the laptop and the phone, so the compiled GPU compute kernels it ships with are never used.

3.3.4 mlagents-learn and PyTorch

mlagents-learn is the Python trainer. It reads a YAML configuration, launches or waits for Unity environments, collects experience, runs PPO updates in PyTorch, writes TensorBoard summaries and training_status.json, and exports ONNX checkpoints. It

supports resuming a stopped run from its last checkpoint with the `--resume` flag, a feature the HB_16 run depended on, and passing arguments through to the Unity player with `--env-args`, which is how the island count reaches the runtime spawner.

3.3.5 TensorBoard

TensorBoard displays the scalar summaries written during training: cumulative reward, episode length, policy entropy, learning rate, value estimate and losses. The same event files were read programmatically for this report, so every training curve in Chapter 7 is drawn from the raw logged values rather than copied from a screenshot.

3.3.6 ONNX

The Open Neural Network Exchange format stores a trained network as a portable graph. Its input names and shapes act as a contract between trainer and runtime: the HB_01 models expect a 10-value vector observation and the HB_16 models a 30-value one, a difference that is visible in the file itself and that Chapter 6 shows must be matched by the Behavior Parameters stacking setting.

3.3.7 Anaconda, Xvfb and the Jarvis Labs platform

Anaconda isolates the pinned training environment (ml-agents-1.0 on the laptop, a conda environment named mla on the server) from other Python installations. Xvfb is an X11 display server that renders into memory; it lets the Unity Linux player start on a machine with no monitor. Jarvis Labs rents GPU instances by the hour with SSH access and persistent `/home` storage, which is where the training folder and its results were kept.

3.3.8 IL2CPP and the Android toolchain

IL2CPP converts the project's compiled C# into C++ and compiles it with the Android NDK's clang for ARM64. It is the only route Unity provides to a 64-bit ARM build. The Android Debug Bridge then installs the APK over USB and can launch it and capture the phone screen from the command line.

Table 3.3 summarises the tools with the exact versions used.

Table 3.3 Summary of tools, versions and their use in the project

Tool	Version	Used for
Unity Editor (URP)	6000.4.7f1	Scenes, physics, scripting, builds
com.unity.ml-agents	4.1.0	Agent, sensors, communicator
Inference Engine (Sentis)	2.6	ONNX inference, Burst CPU backend
mlagents (Python)	1.1.0	PPO trainer, ONNX export
PyTorch	2.13.0 CPU (laptop), 2.2.2 CUDA (server)	Neural network training
Python	3.10.12	Trainer runtime
TensorBoard	2.20.0	Training curves
Xvfb	Ubuntu package	Virtual display for the headless build
Android Build Support, IL2CPP	Unity 6000.4.7f1 module	ARM64 APK

Two points about this toolchain matter more than any single version number. First, the tools form pinned pairs that must be kept together: the Unity package `com.unity.ml-agents` 4.1.0 speaks communicator API 1.5.0, which the Python package `mlagents` 1.1.0 expects, and `mlagents` 1.1.0 in turn fixes the acceptable Python, numpy, protobuf and onnx versions. Upgrading one side on its own is the most common way for an ML-Agents project to stop working, and the version pins in this chapter are what kept both machines compatible.

Second, the training environment is kept separate from everything else on each machine. On the laptop the `ml-agents-1.0` conda environment is used only for training, alongside other Python installations used for unrelated coursework, so a package installed for one purpose cannot silently break the other. On the server the whole environment is disposable and is rebuilt from the setup script on every fresh instance. This separation is the reason every training run in this report can be repeated from the files in the project.

CHAPTER 4

SYSTEM ANALYSIS

This chapter analyses what the system has to do and whether it could be built with the resources available. It states the functional and non-functional requirements, characterises the existing approaches the project replaces, describes the proposed system, assesses its feasibility, and closes with the software requirement specification, including the user stories that drove each development sprint.

4.1 Requirement Specification

This section states the hardware and software requirements under which the project was developed and trained, and the functional and non-functional requirements the implemented system satisfies, addressing the analysis-and-requirements component of the project review process. Hardware and software requirements were covered in Chapter 3.

4.1.1 Functional Requirements

- The system shall allow the agent to observe its environment through ten vector observations per decision step, re-evaluated every five physics steps (Decision Requester period of 5).
- The system shall allow the agent to act through a five-dimensional continuous action space governing three-axis movement force, pitch, and yaw.
- The system shall train the agent's policy using Proximal Policy Optimization (PPO), with all trainer hyperparameters configurable through a YAML trainer configuration file.
- The system shall export the trained policy to ONNX format at the end of training and at periodic checkpoints during training.
- The system shall support running the trained model in pure in-Editor inference mode, with no live Python trainer connection required.
- The system shall provide a manual (Heuristic) control mode allowing a human to control the same agent through keyboard input, for testing and for the competitive minigame.

- The system shall support a second deployment scene in which a trained agent competes against a human, keyboard-controlled player for nectar collection under a shared timer and win condition.
- The system shall build its training arena grid at runtime from a `--islands N` launch argument, so that the number of parallel arenas can be changed without rebuilding the Unity player.
- The system shall resume an interrupted training run from its last saved checkpoint without changing the step budget or the learning-rate schedule.
- The Android application shall let the viewer switch between the HB_01 and HB_16 models at runtime, always changing the model and its observation stacking together.
- The Android application shall offer a debug view of the agent's ray sensors and vector observations, flight trails, a split-screen race between the two models and a reset control.

4.1.2 Non-Functional Requirements

- The system shall be trainable to convergence entirely on CPU-only hardware, with no dependency on a discrete GPU, given the target machine's permanently disabled GPU.
- The system shall operate within an 8GB RAM budget during both Unity Editor use and ML-Agents training, achieved in practice by scaling the training environment down to a single floating island after an eight-island configuration caused a full system freeze.
- Trained inference shall run in real time, with no perceptible lag, when deployed with no live Python connection.
- The training environment and Python trainer shall remain version-pinned and mutually compatible (Unity 6000.4.7f1 with ML-Agents 4.1.0, Python 3.10.12, and mlagents 1.1.0), to avoid the class of dependency-conflict issues documented in Chapter 8.
- The rented-server environment shall be reproducible from a single setup command, with every dependency version pinned.

- Training on rented hardware shall checkpoint often enough that an interruption loses only minutes of work; a checkpoint and ONNX export every 250,000 steps was used.
- The Android build shall run fully offline, with no network connection or trainer process, and remain responsive on the target phone.
- Any new result shall remain directly comparable with the HB_01 baseline: the reward function, the 5,000-step episode length and the bird's physics shall not be changed.

4.2 Characteristics of the Existing System

Two different things play the role of the existing system in this project. The first is the conventional way of authoring non-player character behaviour in games, which the project's learned agent is meant to replace. Its characteristics, drawn from the literature in Chapter 2, are:

- behaviour is written as explicit rules, finite state machines or behaviour trees, so every situation must be anticipated by the designer;
- the rule count grows quickly as the environment becomes continuous and three-dimensional, and small changes to the environment require re-authoring;
- behaviour is fully predictable and easy to debug, but it cannot improve with experience;
- believable skilled movement, such as a bird hovering precisely in front of a moving flower, is difficult to hand tune in a physics-driven world.

The second existing system is the original course codebase that the project started from. It was authored for Unity 2019.3 and an early ML-Agents release, and its characteristics made it unusable as-is on current software:

- agent callbacks used raw float arrays (`OnActionReceived(float[] vectorAction)`, `Heuristic(float[] actionsOut)`) instead of the current `ActionBuffers` API;
- physics code used `Rigidbody.velocity`, which Unity 6 replaced with `linearVelocity`;
- trained models were exported in the old `.nn` Barracuda format rather than ONNX;

- the trainer configuration used a legacy single-file YAML layout with a default block, and a scientific-notation step budget that current PyYAML reads as a string;
- the default training scene assumed eight parallel islands in one Editor, which froze an 8 GB laptop completely.

4.3 Proposed System

The proposed system replaces hand-written behaviour with a policy network trained by PPO, and replaces the legacy codebase with a migrated one that runs on Unity 6 and ML-Agents 4.1.0. The agent receives nine raycasts and ten vector observations, outputs five continuous actions, and is rewarded only for drinking nectar with good beak alignment and penalised for hitting the island boundary. The same agent script serves three purposes without modification: training, human play through the heuristic, and inference from an exported model.

Around this core the proposed system adds three capabilities that the existing course material does not have. A runtime island spawner and a pair of shell scripts allow training on a rented Linux server with many parallel arenas. A command-line Android build packages both trained models into a phone application with live model switching. A presentation toolkit makes the agent's perception and the difference between the two models visible on screen. The advantages of the proposed system are:

- behaviour is learned from a reward signal, so it adapts to flower positions it was never explicitly told about;
- training scales from one arena on a laptop to 112 arenas on a server with no change to the agent code;
- a trained model runs on a phone with no network and no Python process;
- every claim about the model can be demonstrated live, from its raw observations to its score against the other model.

4.4 Feasibility Study

4.4.1 Technical Feasibility

The technical risk lay in three places: whether ML-Agents 4.1.0 would work with Unity 6 at all, whether an 8 GB laptop with no usable GPU could train the agent, and whether the Unity player could run headless on a Linux server. The first was confirmed early by avoiding the broken 3.0.0 package. The second was confirmed by reducing the environment to a single island, after which HB_01 trained to completion. The third needed a virtual display and was confirmed on a one-day validation rental on 31 August 2026, before any long paid run was started. All three risks were therefore retired by direct test, not assumed away.

4.4.2 Economic Feasibility

All software used is free for this purpose: Unity Personal, the open-source ML-Agents Toolkit, PyTorch, Anaconda and TensorBoard. The laptop and phone already existed. The only direct cost was the rented server, billed at Rs 41.31 per hour for compute plus Rs 1.30 per hour for 100 GB of storage, Rs 42.61 in total. HB_16's two sessions used 5.11 hours of active training, which is roughly Rs 218 at that rate; the training script's own estimate for the second session alone was Rs 113. Deleting the instance between sessions meant the 7.5 hour gap cost nothing. The same 50 million steps at the laptop's measured 44 steps per second would have taken about 13 days of continuous running, so renting was also the cheaper option in time.

4.4.3 Operational Feasibility

Operationally the system has to be usable by three kinds of user: the developer who trains it, a player who competes against it, and a viewer who watches the demo. Training is operated through four documented commands (smoke, sweep, run and status). The minigame needs only a keyboard and a single Start button. The phone demo needs only taps, with every control labelled on screen and a Reset button that returns the whole demo to its opening state. No step requires specialist knowledge beyond what is documented in the user guide in Appendix 4.

4.4.4 Schedule Feasibility

HB_01 needed 31.44 hours of active computation spread across 38.18 calendar hours and six stop-and-resume sessions, because training had to fit around the laptop's other uses. HB_16 needed only 5.11 hours of active computation and finished within a single day. Both fitted comfortably within the project timeline.

4.5 Software Requirement Specification

The software requirement specification below records the requirements as user stories, the form in which they were planned and tracked during development. Table 4.1 is the product backlog; the subsections that follow list the objectives of each sprint and the additional stories added for the extended work.

4.5.1 Product Backlog

Table 4.1 Product backlog, key user stories with desired outcomes

#	User Story	Desired Outcome
1	As the developer, I want a scripted flower/nectar system, so that the agent has a depletable resource to forage for.	Flower.cs correctly tracks, feeds, and resets nectar per flower.
2	As the developer, I want a flower-area manager, so that the agent can query all flowers in the scene efficiently.	FlowerArea.cs indexes all flowers via a collider-to-flower dictionary.
3	As the developer, I want a fully-scripted RL agent, so that it can observe, act, and be rewarded correctly each step.	HummingbirdAgent.cs implements Initialize, OnEpisodeBegin, CollectObservations, OnActionReceived, Heuristic.
4	As the developer, I want to train the agent under CPU-only constraints, so that training is feasible on the available hardware.	A completed 5,000,024-step training run with a positive, stable final reward.
5	As the developer, I want the trained model deployable without Python, so that the agent works in a shipped build.	ONNX export verified working under Unity's Inference Engine in Play mode.
6	As the developer, I want a second, competitive deployment of the trained agent, so that its generalization can be demonstrated.	A working Player-vs-trained-Opponent minigame scene with a functioning UI and win condition.

4.5.2 Objectives and User Stories by Sprint

Sprint I, Project Setup and Foundation.

- Set up a new Unity 6 (URP) project and import the base course assets (hummingbird, flowers, floating island, materials).
- Fix scene lighting and understand the imported asset hierarchy.
- Install the Unity ML-Agents package at a version compatible with Unity 6000.x, and create the base (empty) agent scripts.

Sprint II, Core Environment Scripting.

- Implement Flower.cs so that individual flowers can be fed from, deplete, and reset.
- Implement FlowerArea.cs so that all flowers in the scene can be found, indexed, and reset as a group.

Sprint III, Agent Development and Scene Wiring.

- Write the complete HummingbirdAgent.cs across all twelve conceptual parts of the base course (Explanation through Update/FixedUpdate).
- Wire the agent's components (Behavior Parameters, Decision Requester, Ray Perception Sensors) onto the Hummingbird prefab.
- Apply the flower_plant, nectar, and boundary tags to their target GameObjects, and verify manual (Heuristic) control works correctly.

Sprint IV, Training and Inference.

- Set up a dedicated Python 3.10.x environment satisfying ML-Agents' Python version constraint, isolated from two existing, unrelated Python installations.
- Configure PPO hyperparameters and successfully run a full training session to convergence under CPU-only, memory-constrained hardware.
- Export and verify the trained model under Unity's Inference Engine, with no live Python connection.

Sprint V, Minigame and Course Wrap-up.

- Build a second scene in which the trained agent (in pure inference mode) competes against a human, keyboard-controlled player for nectar under a shared timer.
- Wire a GameManager and UI Controller to drive game state, progress bars, and a win/lose banner.

4.5.3 User Stories for the Extended Work

The extended work added the user stories in Table 4.2, numbered on from the original backlog.

Table 4.2 Additional user stories added after the base course

#	User Story	Desired Outcome
7	As the developer, I want to know why HB_01 stopped at 41.29, so that a retrain changes the right thing.	Cause identified from the run's own logs: budget exhausted while climbing, and no velocity observation.
8	As the developer, I want to train on many arenas in parallel on rented hardware, so that a much larger step budget is affordable.	A 50,000,000-step run on 112 arenas, resumable after interruption.
9	As the developer, I want to know the fastest process and arena layout before paying for a long run.	A measured sweep of six splits and three arena counts; 28 islands x 4 processes chosen.
10	As a viewer, I want to run both models on a phone and switch between them live.	A 54.4 MB ARM64 APK with a runtime model switcher, verified on the S24 Ultra.
11	As a viewer, I want to see what the agent perceives and how the two models differ.	Debug view, flight trails and a split-screen race with a scoreboard.

4.5.4 Interface Requirements and Design Constraints

- Trainer interface: the Unity environment and mlagents-learn communicate over gRPC on local port 5004; on the server each of the four Unity processes uses its own port.
- Model interface: an HB_16 model must be paired with Stacked Vectors 3 (input width 30) and an HB_01 model with Stacked Vectors 1 (input width 10); the ray sensors' detectable-tag list must not change, because it sets the width of every ray's observation.
- User interfaces: keyboard (W, A, S, D, E, C and arrow keys) for heuristic flight; mouse or touch buttons for the minigame and phone demo; R, T and Y keys for the toolkit in the Editor.
- Build constraints: IL2CPP and ARM64 for Android; the Unity Editor must be closed during command-line builds, because it locks the project.
- Measurement constraint: no change to the reward function, the 5,000-step episode or the bird's physics.

4.5.5 Requirement Traceability

Table 4.3 traces the main requirements to the part of the design that satisfies each one and to the test case in Chapter 7 that verifies it. Every requirement has at least one verifying test, and no test exists without a requirement behind it.

Table 4.3 Requirement traceability from requirement to design and test

Requirement	Satisfied by	Verified by
10 vector observations, 5 continuous actions	HummingbirdAgent.cs, Behavior Parameters	TC01, TC08, TC11
PPO training configured by YAML	trainer_config.yaml, trainer_config_16.yaml	TC05, TC14
ONNX export at checkpoints and at the end	mlagents-learn, checkpoint_interval 250,000	TC09, TC10
Inference with no Python connection	Inference Engine, Behavior Type Inference Only	TC06
Heuristic keyboard control	HummingbirdAgent.Heuristic	TC01
Player versus trained agent minigame	GameManager.cs, UIController.cs	TC07
Runtime arena count from --islands	IslandSpawner.cs	TC12
Resume after interruption	--resume, results/HB_16 re-upload	TC14
Model and stacking always changed together	DemoController.SelectModel, RaceController	TC15, TC18
Offline Android application, ARM64	BuildAndroid.cs, Demo.unity	TC16, TC17
Training without a display on the server	Xvfb 24-bit display, -batchmode	TC13
Comparable results (no reward or physics change)	Unchanged reward code and MaxStep 5000	TC11 and Section 7.5

CHAPTER 5

SYSTEM DESIGN

This chapter presents the design of the system before its implementation is described. It starts with the overall architecture and workflow, then gives the context, use case, activity, state, data flow and class diagrams, the design of the experience data the agent learns from, and the design of the two user interfaces.

5.1 System Architecture and Overall Workflow

Fig. 5.1 shows the architecture as a whole. The system has two halves that run as separate processes during training. The Unity environment, written in C#, owns the world: the floating island with its 360 flowers, the physics that moves the bird, the sensors that describe the world to the agent, and the rules that turn contact with nectar and the boundary into rewards. The Python trainer owns learning: it receives observations and rewards, runs the policy network to produce actions, stores experience, performs PPO updates and exports the network. After training the Python half is no longer needed, because the exported ONNX file is loaded directly by Unity's Inference Engine in any of the three deployment scenes.

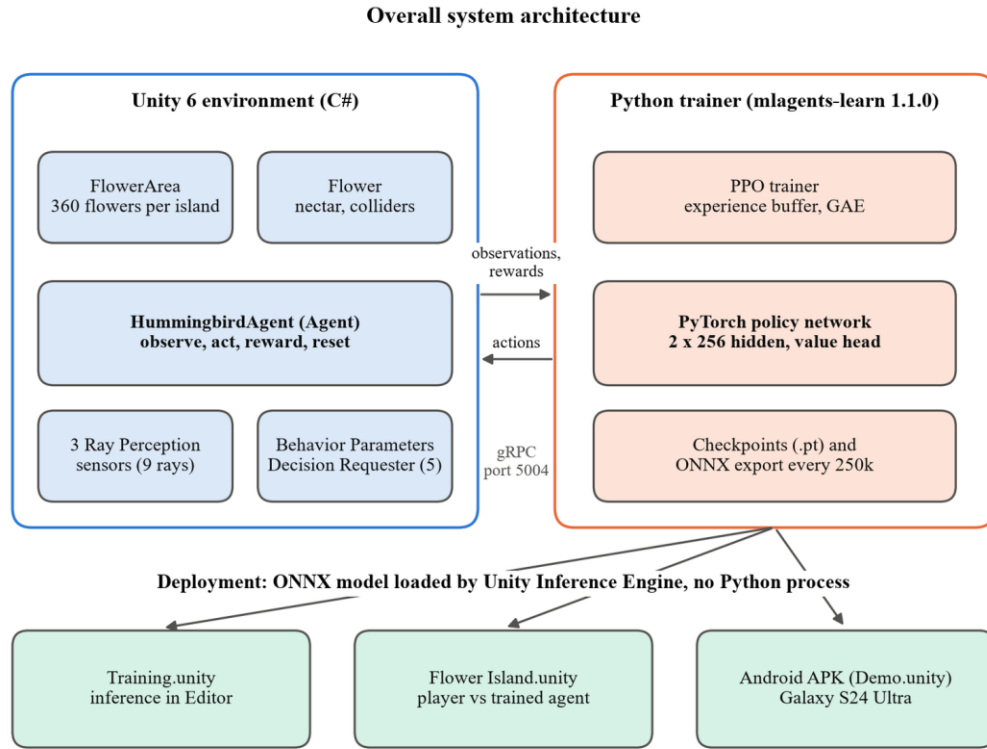


Fig. 5.1 Overall system architecture of the Hummingbird ML-Agents system

5.1.1 Environment and Agent Architecture

At the end of Sprint I, the project's dependency structure was: Unity 6000.4.7f1 (URP) as the runtime engine, com.unity.ml-agents 4.1.0 providing the Agent base class and training communicator, and com.unity.ai.inference (Inference Engine, "Sentis 2.6") providing on-device neural network inference, installed automatically as a transitive dependency of ML-Agents rather than requiring a separate manual install.

FlowerArea.cs is attached to the root of FloatingIsland.prefab and recursively walks its own children, using two tag checks (flower_plant to recurse into a flower plant, and a Flower component check on any other child) to build the flowerPlants list, the flowers list, and the nectarFlowerDictionary in a single pass. This recursive design allows the flower hierarchy to be restructured within a plant without requiring any change to FlowerArea.cs itself.

The Hummingbird prefab, nested inside FloatingIsland.prefab, carries four key components once Sprint III is complete: HummingbirdAgent (the learning logic), Behavior Parameters (Behavior Name "Hummingbird", Vector Observation Space Size

10, Continuous Actions 5, Team ID 0, Use Child Sensors enabled), Decision Requester (with a Decision Period of 5, meaning the agent re-decides every 5 physics steps, i.e. every 0.1 real seconds), and three child Ray Perception Sensor 3D components (RayForward, whose 3 rays per direction give 7 rays fanned across plus or minus 60 degrees, and RayUp and RayDown with a single ray each, 9 rays in all), all with a Sphere Cast Radius of 0 (pure raycasts) and a 20-unit ray length, detecting Untagged and boundary-tagged colliders.

One property of this sensor design is easy to miss and matters for how the agent works. The sensors' detectable tag list is empty-string and boundary, and exactly one object on the island carries the boundary tag. A ray that strikes a flower, a rock or the ground therefore reports only its distance, with both tag channels at zero. The rays are an undifferentiated proximity sense that keeps the bird away from solid geometry and the island edge; all of the flower-seeking behaviour comes from the ten vector observations. The two parts of the agent's perception are genuinely separate systems, a point demonstrated on screen by the debug view described in Chapter 6.

5.1.2 Training Architecture

The training architecture follows Unity ML-Agents' standard external-trainer pattern: the Unity Editor (or a headless build) runs the environment and the agent's C# logic, and communicates over a local gRPC connection with the separate Python process running `mlagents-learn`, which owns the PPO trainer, the neural network (in PyTorch), and periodic ONNX checkpoint export. During training, RAM constraints (8GB total) made an 8-island parallel training configuration entirely infeasible (it caused a full system freeze) so the environment was scaled down first to 4 islands, and finally to a single floating island, which became the stable, confirmed-working configuration for the full run.

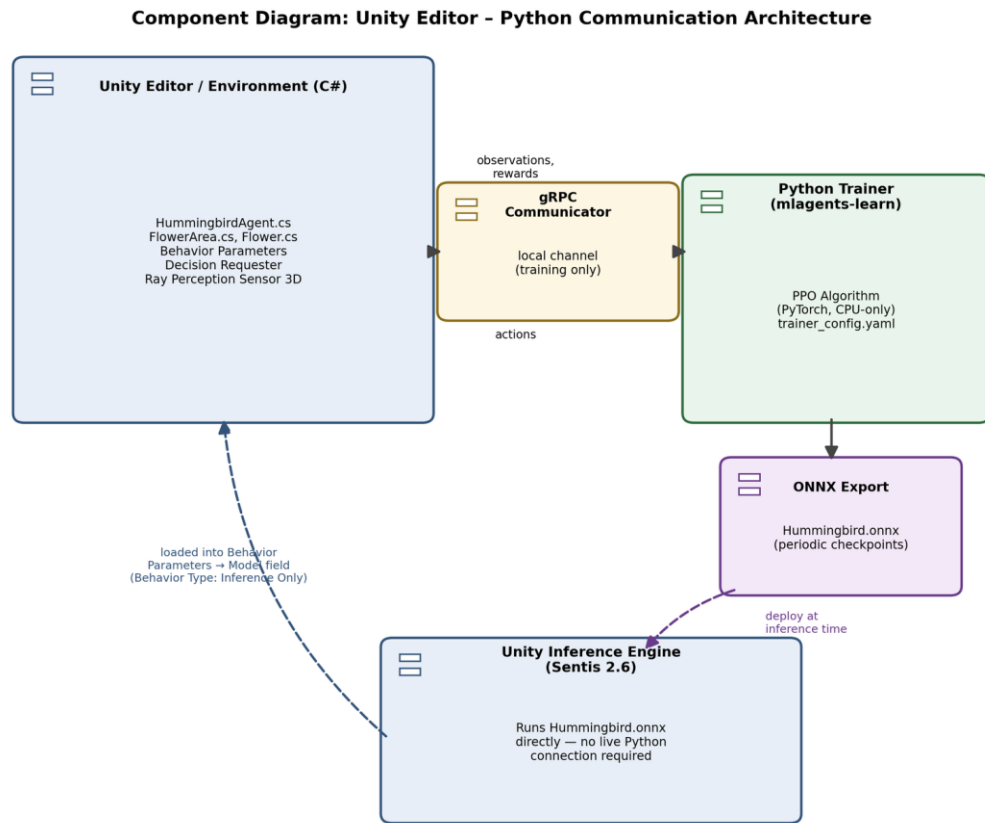


Fig. 5.2 Component diagram of the Unity Editor and Python trainer communication architecture

Fig. 5.2 shows the system's runtime architecture across its two deployment modes. During training, the Unity Editor and the Python trainer (mlagents-learn) communicate over a local gRPC channel, with the trainer's PPO algorithm periodically exporting ONNX checkpoints. During inference (both in the original training scene and in the competitive minigame), Unity's own Inference Engine loads the exported ONNX model directly into the Behavior Parameters component, with no live Python process required.

5.1.3 Minigame Architecture

UI.prefab (course-provided) ships with its own nested EventSystem, and its UIController.cs exposes an `OnButtonClicked` event that GameManager subscribes to in `Start()`. Both `HummingbirdAgent` instances additionally use the project's own custom `CameraFollowBird.cs` (a lightweight third-person chase camera, not part of the base course) attached to each agent's own dedicated camera.

5.1.4 Rented-GPU Training and Deployment Architecture

For the HB_16 run the architecture was stretched across two machines. The Unity project is built on the laptop into a headless Linux player, uploaded with the training folder to the server, and launched four times by the training script, each process building a grid of 28 islands at startup. The four processes connect to one mlagents-learn trainer, which sees 112 agents feeding a single policy. A virtual X display stands in for a monitor. Results are downloaded, verified by checksum and copied into the Unity project as a new model file, where they are used by the Editor scenes and packaged into the Android build. The details of each stage are given in Section 6.6.

5.2 Context Diagram

The context diagram in Fig. 5.3 treats the whole system as a single process and shows everything that exchanges data with it. The developer supplies scripts and configuration and receives reward curves and trained models. The Unity Editor provides scenes and prefabs. The Unity Editor provides scenes and prefabs. The rented server receives a Linux build and a configuration and returns checkpoints and event logs. The human player sends keyboard input and receives the minigame's timer, progress bars and winner banner. The viewer of the phone demo sends taps and gestures and sees the bird's flight and the live scores.

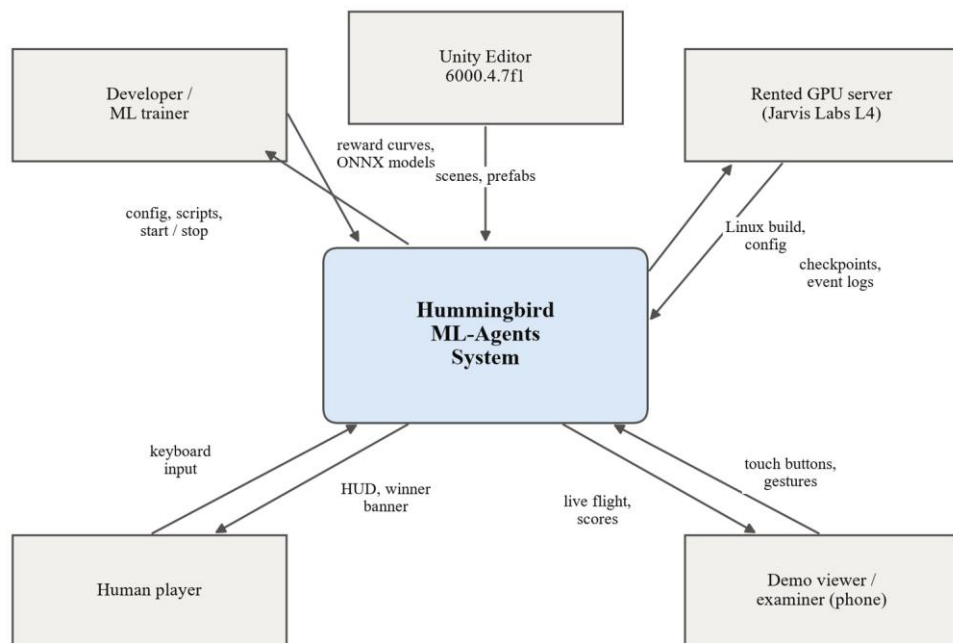


Fig. 5.3 Context diagram of the Hummingbird ML-Agents system

5.3 Use Case Diagram

This section presents a structured analysis of the system using standard UML diagrams, addressing the problem-analysis component of the project review process. Each diagram is derived directly from the actual implementation (HummingbirdAgent.cs and the project's training architecture) rather than from a generic template.

Fig. 5.4 identifies the system's principal actors and their interactions. The ML Researcher/Trainer configures and runs PPO training and exports the resulting policy; the Human Player controls the agent manually via the Heuristic control path or competes against the trained agent in the minigame; and the Trained Agent, running under pure inference, interacts with both the training environment and the minigame independently of any live Python connection.

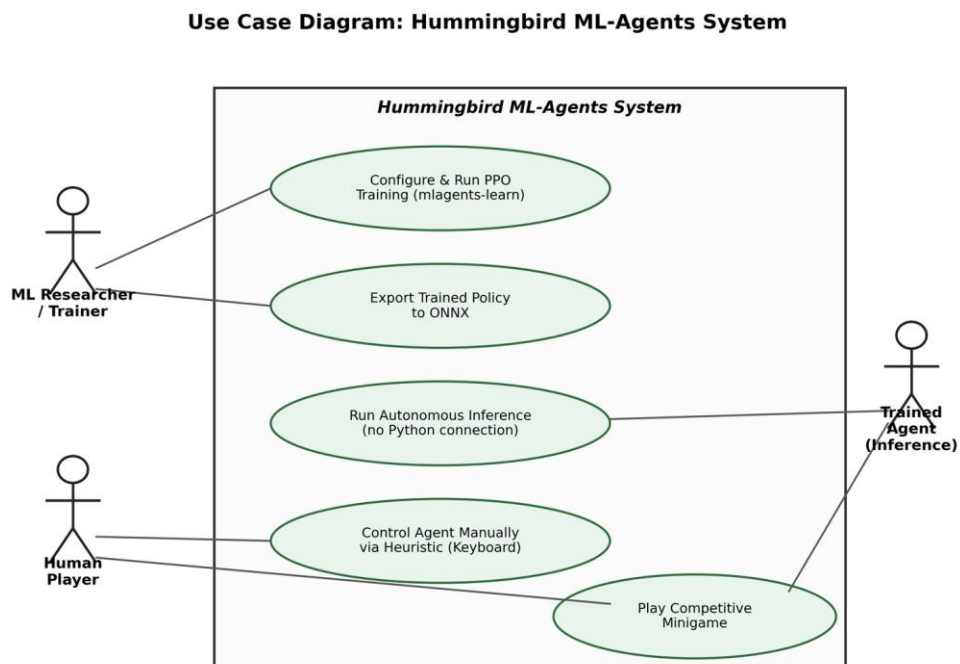


Fig. 5.4 Use case diagram of the Hummingbird ML-Agents system actors and interactions

Two further use cases were added for the extended work and belong to the same actors. The ML researcher can run a throughput sweep and a resumable long training run on a rented server. The viewer, a new actor who is neither trainer nor player, can start the

phone demo, switch models, toggle the debug view and trails, start a split-screen race and reset the demo.

5.4 Activity and State Diagrams

Fig. 5.5 traces the control flow of a single training episode exactly as implemented in `HummingbirdAgent.cs`: episode reset, observation collection, the branch between Heuristic (keyboard) and neural network control, action execution, the nectar-contact and boundary-collision reward checks, and the loop back to observation collection until the 5,000-step episode cap is reached.

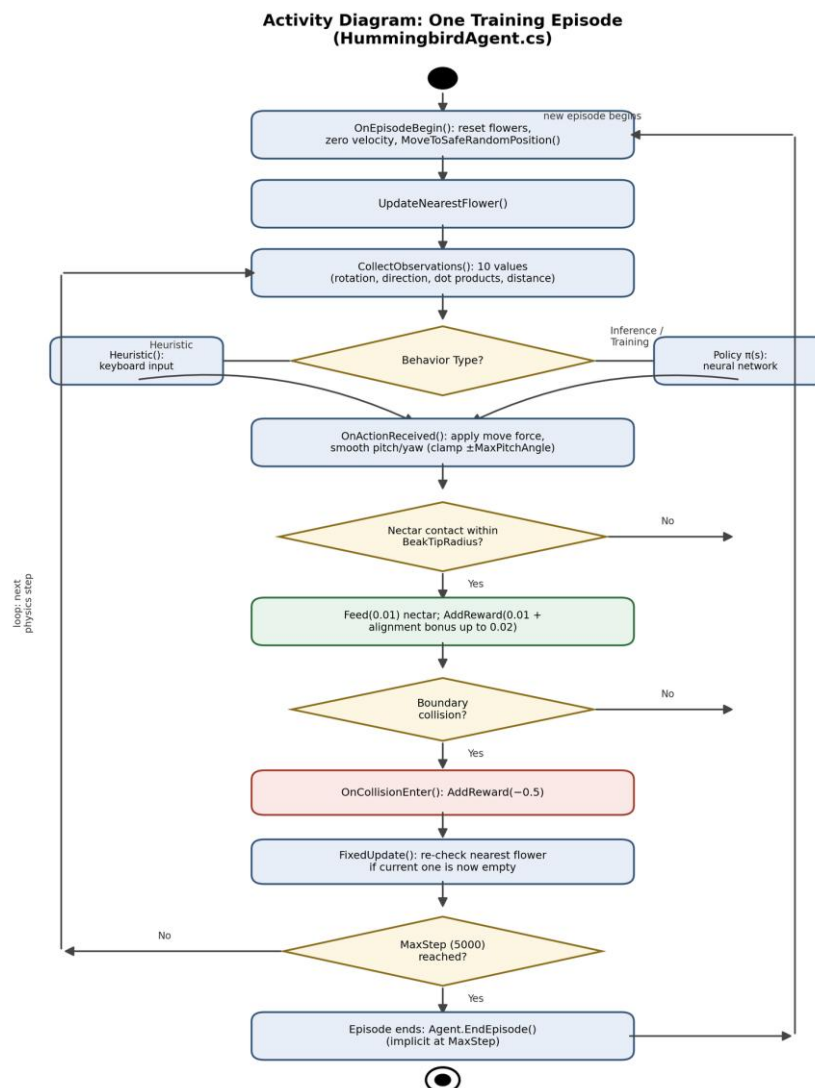


Fig. 5.5 Activity diagram of one training episode in `HummingbirdAgent.cs`

The minigame is controlled by a simple state machine in GameManager.cs, shown in Fig. 5.6. The game starts in the MainMenu state with both birds frozen and the flowers reset. Pressing Start moves it to Preparing, where a four-second countdown runs; the birds are then unfrozen and the game enters Playing. Every frame in Playing checks whether the 60-second timer has expired or either bird has collected 8 units of nectar, and moves to Gameover when either happens. The Main Menu button returns the game to MainMenu.

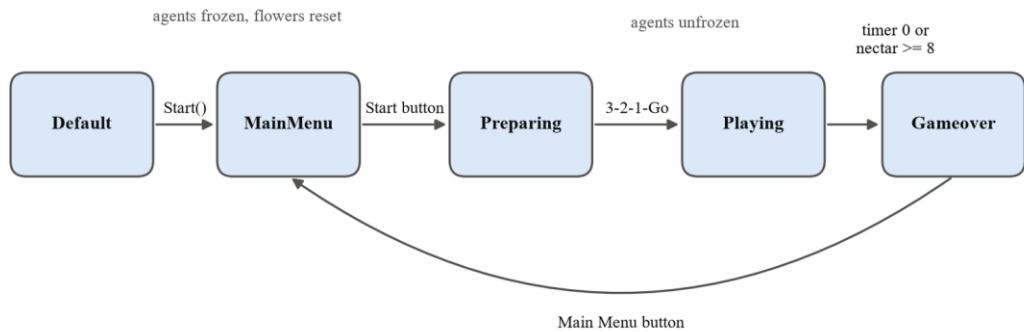


Fig. 5.6 State diagram of the minigame GameManager

5.5 Data Flow, Class and Machine Learning Workflow Diagrams

5.5.1 Data Flow Diagrams

At level 0 (Fig. 5.7) the system is a single process that turns configuration and scenes from the developer into curves and trained models, and turns inputs from players and viewers into flight and scores.



Fig. 5.7 Level 0 data flow diagram

Level 1 (Fig. 5.8) opens up one training step. Process 1.0 collects the 66 observation values, reading flower positions from the nectar state store through FlowerArea. Process 2.0 passes them through the policy network to choose five actions. Process 3.0 applies the force and rotation in the physics engine, and process 4.0 detects nectar and boundary contact, updates the flower state and computes the reward. The tuple of observation, action and reward is written to the experience buffer, and every 40,960 steps process 5.0 performs a PPO update that changes the policy used by process 2.0. Process 6.0 writes a checkpoint and ONNX file every 250,000 steps.

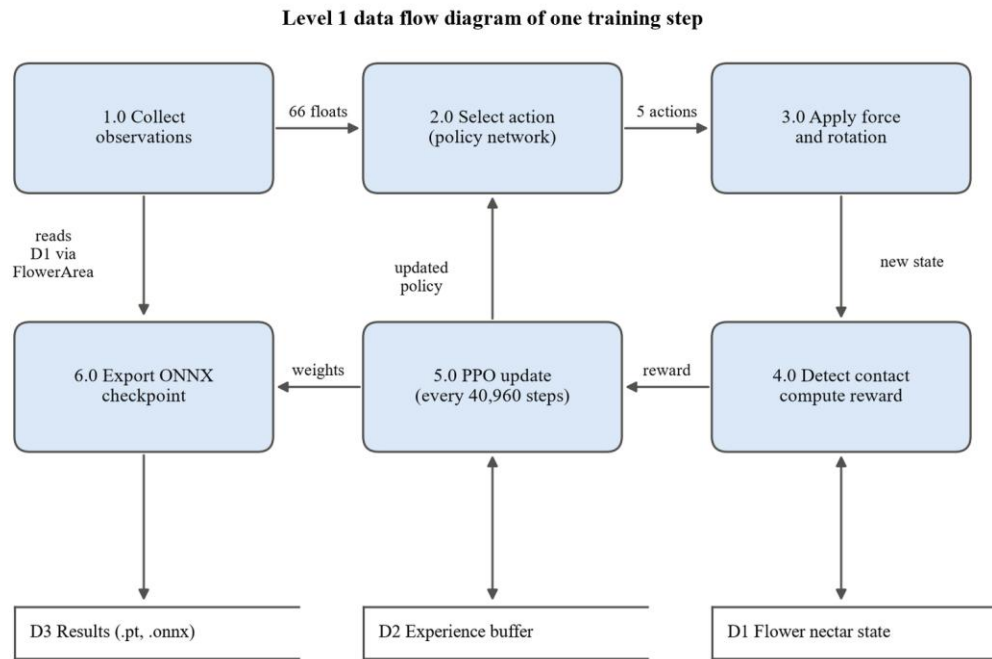


Fig. 5.8 Level 1 data flow diagram of one training step

5.5.2 Class Diagram

Fig. 5.9 shows the classes of the Unity project and their relationships. HummingbirdAgent inherits from ML-Agents' Agent class and is associated with exactly one FlowerArea, which owns up to 360 Flower objects. GameManager coordinates two agents and drives UIController. The classes shown in green were added by this project beyond the base course: the runtime IslandSpawner used on the server, the DemoController and DemoToggles that run the phone demo, RaceController for the split-screen race, the visualisation components, the touch camera input and the shared UiScaling helper.

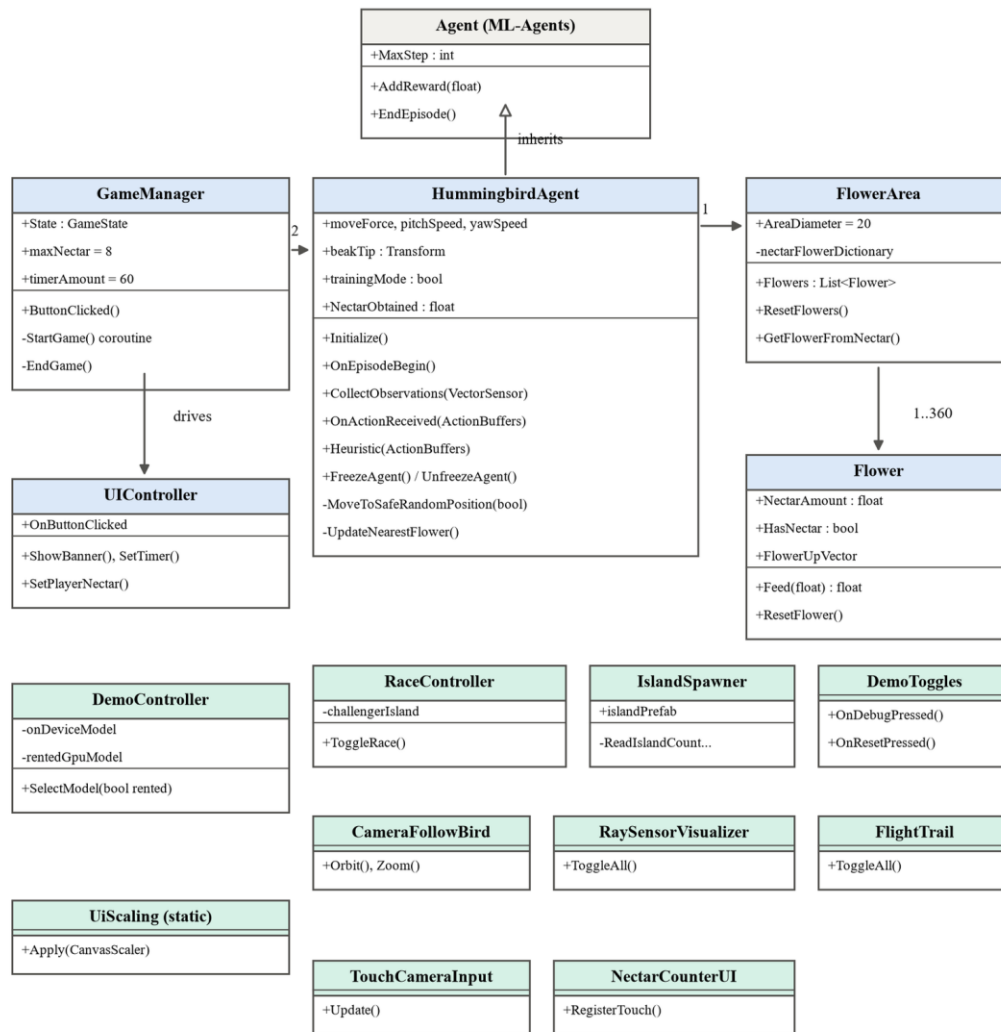
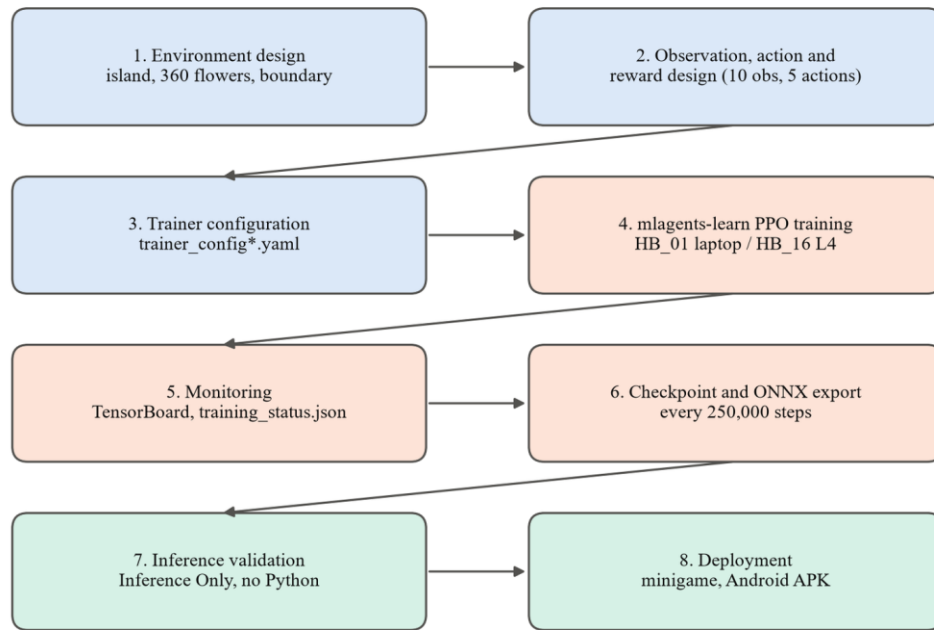


Fig. 5.9 Class diagram of the Unity project

5.5.3 Machine Learning Workflow

Fig. 5.10 summarises the machine learning workflow followed in the project. The environment and the observation, action and reward design come first, because they fix what the policy can learn. The trainer configuration sets how it learns. Training is monitored through TensorBoard and the checkpoint history, and every checkpoint is exported to ONNX. Models are validated in pure inference before being deployed. The loop back from monitoring to design is what produced HB_16: analysis of HB_01's logs led to a change in the observation design (stacked vectors) and in the trainer configuration (step budget, batch size and normalisation).



Steps 2 to 6 repeat whenever the design changes (for example stacked vectors 1 to 3 between HB_01 and HB_16)

Fig. 5.10 Machine learning workflow followed in the project

Together, the use case, activity, and component diagrams in this chapter give a complete structural and behavioural picture of the system before any implementation detail is discussed: who interacts with the system and how (Fig. 5.4), what happens inside a single training episode at the code level (Fig. 5.5), and how the Unity and Python halves of the system communicate across both training and deployment (Fig. 5.2). The dataset and interface design in the remaining sections complete the design, and Chapter 6 then turns to how the system was actually built.

5.6 Dataset Design

A reinforcement learning project has no fixed dataset. The data is generated by the agent itself as it acts, and the design decision is what that data contains. Each experience record holds the observation, the action taken, the reward received, whether the episode ended, and the trainer's value estimate and action log-probability. The observation is the most important part of that design and is specified in Table 5.1.

Table 5.1 Observation design (HB_16, 66 values per decision)

Input	Source	Values	Meaning
obs_0	RayDown sensor, 1 ray	4	tag-0 hit, boundary hit, no hit, normalised distance
obs_1	RayForward sensor, 7 rays	28	4 values per ray, rays fanned across plus or minus 60 degrees
obs_2	RayUp sensor, 1 ray	4	4 values, straight up
obs_3 (1-4)	transform.localRotation	4	bird orientation as a normalised quaternion
obs_3 (5-7)	beak tip to nearest flower	3	unit direction to the flower
obs_3 (8)	dot(direction, flower axis)	1	+1 when the beak is on the flower's open side
obs_3 (9)	dot(beak forward, flower axis)	1	+1 when the beak points into the flower
obs_3 (10)	distance / 20	1	distance to the flower scaled by the area diameter
stacking	last 3 decisions of obs_3	$3 \times 3 = 30$	lets the network infer velocity (HB_16 only)

With stacking set to 1, as in HB_01, the vector part is 10 values and the whole input is 46 values; with stacking 3, as in HB_16, it is 30 values and the input is 66. The action and reward designs are given in Table 5.2 and Table 5.3.

Table 5.2 Action design (5 continuous values in the range -1 to +1)

Index	Meaning	Applied as
0, 1, 2	Movement along x, y and z	Rigidbody.AddForce(move x moveForce 2)
3	Pitch change	smoothed with MoveTowards, x pitchSpeed 100, clamped to plus or minus 80 degrees
4	Yaw change	smoothed with MoveTowards, x yawSpeed 100

Table 5.3 Reward design (unchanged between HB_01 and HB_16)

Event	Reward	Frequency
Beak tip inside a nectar trigger	+0.01	every 0.02 s physics step while drinking
Beak aligned with the flower	up to +0.02 extra	same step, scaled by alignment
Collision with the boundary	-0.5	once per collision
Episode length	5,000 physics steps	= 1,000 decisions = 100 simulated seconds

Experience is grouped into trajectories of 128 decisions (the time horizon) and collected into a buffer before each update. HB_01 used a buffer of 20,480 decisions and mini-batches of 2,048; HB_16 doubled both to 40,960 and 4,096, because 112 agents fill a buffer 112 times as fast as one. Each buffer is used for three epochs of updates and then discarded, as PPO requires.

5.7 User Interface Design

The project has two user interfaces, laid out in Fig. 5.11. The minigame interface (part a) is the course-provided UI prefab: a nectar progress bar for each bird at the top corners, a countdown timer in the centre, a large banner for the countdown and the result, and one button whose label changes between Start and Main Menu. It is driven entirely by GameManager through UIController, which has no logic of its own.

The phone interface (part b) was designed for a landscape screen held at arm's length. A single row of six large buttons (On-Device, Rented GPU, Debug, Trails, Split and Reset) is anchored to the bottom edge of the screen, so it stays in place on any aspect ratio. The score label at the top names the model currently loaded together with its trained reward, and during a race it splits into two labels, one over each half of the screen, in each bird's trail colour. The debug readout sits at the top left, below the score, and the rest of the screen is left to the chase camera, which the viewer can orbit and zoom by touch.

(a) Flower Island minigame HUD (UI.prefab, driven by GameManager)



(b) Android demo layout, landscape (Demo.unity + DemoToggles + RaceController)

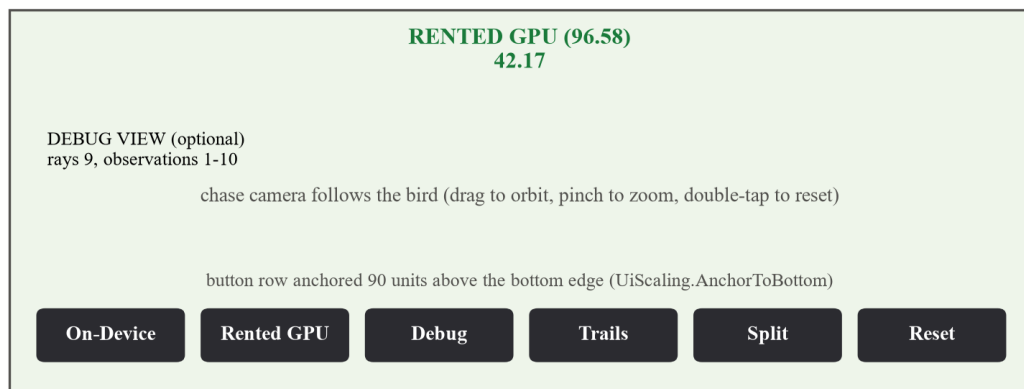


Fig. 5.11 User interface layouts: (a) minigame HUD, (b) Android demo

CHAPTER 6

SYSTEM IMPLEMENTATION

This chapter describes how the design of Chapter 5 was implemented. It lists the modules, explains how the training data is generated and prepared, describes the observation features and the network, and then walks through the implementation in the order in which it was built, from project setup to the Android application. It closes with the validation checks built into the code and the implementation of the user interfaces.

6.1 Modules Description

The Unity project contains seventeen C# scripts. Five come from the base course and were migrated; the other twelve were written for this project. Table 6.1 lists them with their size and purpose.

Table 6.1 Modules of the Unity project

Script	Lines	Origin	Purpose
HummingbirdAgent.cs	435	Course, migrated	The learning agent: observations, actions, reward, reset, heuristic
FlowerArea.cs	154	Course, migrated	Finds and indexes one island's flowers; resets them
Flower.cs	114	Course, migrated	One flower's nectar, colliders and colour
GameManager.cs	257	Course, migrated	Minigame state machine, timer, winner
UIController.cs	132	Course, migrated	Minigame UI bindings, no logic
CameraFollowBird.cs	127	Project	Chase camera with orbit and zoom offsets
NectarCounterUI.cs	70	Project	Runtime-built on-screen nectar touch counter
IslandSpawner.cs	103	Project	Builds the training grid from --islands N
DemoController.cs	114	Project	Phone demo Start button and live model switcher
DemoToggles.cs	199	Project	Phone button bar: Debug, Trails, Split, Reset
RaceController.cs	447	Project	Split-screen race between the two models
RaySensorVisualizer.cs	374	Project	Draws the 9 rays and prints the 10 observations
FlightTrail.cs	148	Project	Fading ribbon behind each bird

Script	Lines	Origin	Purpose
TouchCameraInput.cs	210	Project	Drag to orbit, pinch to zoom, double-tap to reset
UiScaling.cs	55	Project	The single CanvasScaler configuration for every canvas
BuildLinuxHeadless.cs	110	Project (Editor)	Command-line headless Linux training build
BuildAndroid.cs	130	Project (Editor)	Command-line ARM64 Android APK build

The modules fall into four groups. The environment group (Flower, FlowerArea, HummingbirdAgent) is everything the trainer interacts with. The minigame group (GameManager, UIController, CameraFollowBird, NectarCounterUI) turns the environment into a game. The training-infrastructure group (IslandSpawner and the two build scripts) exists to move training off the laptop. The demo group (DemoController, DemoToggles, RaceController, RaySensorVisualizer, FlightTrail, TouchCameraInput, UiScaling) exists to make the trained models demonstrable on a phone. The training side has three further artefacts outside Unity: the trainer configurations `trainer_config.yaml` and `trainer_config_16.yaml`, the server setup script `setup_server.sh` and the training script `train_16.sh`.

6.2 Dataset Description and Data Collection

As described in Section 5.6, the data is collected by the agent while it trains. In HB_01 a single agent on a single island in the Unity Editor produced all 5,000,024 decision steps. In HB_16, 112 agents on 112 islands, spread over four Unity processes of 28 islands each, produced 50,002,848 decision steps, so each agent contributed about 446,000. One decision step corresponds to five physics steps, because the Decision Requester period is 5; an episode of 5,000 physics steps is therefore 1,000 decisions, and the trainer's own Episode Length statistic reads 999 throughout both runs.

Collection produced four kinds of record per run, all of which are used in Chapter 7. TensorBoard event files hold the logged summaries: every 10,000 steps for HB_01 (500 points) and every 50,000 for HB_16 (1,000 points). The `training_status.json` file lists each saved checkpoint with its step, mean reward and time. The `timers.json` file breaks the last session's wall-clock time down by activity. Finally, the checkpoint folder holds a

PyTorch .pt file and an ONNX file for each saved step: HB_01 kept twelve, and HB_16 kept the last forty at 250,000-step intervals.

A short third run, HB_baseline_random, was also recorded. It stopped at 13,896 steps with a mean reward of 0, and its exported model is kept in the project as Hummingbird_Random_Baseline.onnx to show what an untrained policy does. Its checksum matches the run's exported file exactly, as shown in Chapter 7.

6.3 Data Preprocessing

Preprocessing is done in two places. The first is the agent's own CollectObservations method, which puts every vector observation on a comparable scale before it leaves Unity: the rotation is a normalised quaternion, the direction to the flower is a unit vector, the two alignment values are dot products of unit vectors and so lie between -1 and +1, and the distance is divided by the island's diameter of 20 units. The ray sensors produce values that are already 0 or 1 for the three flag channels and between 0 and 1 for distance.

The second place is the trainer. HB_01 ran with input normalisation switched off. For HB_16 it was switched on (normalize: true), which makes the trainer keep a running mean and variance of every input and feed the network standardised values. The reason is visible in the observation design: the distance observation, divided by 20, sits around 0.01 to 0.1 during close flight, about fifty times smaller than the other inputs, so its gradient signal was correspondingly weak. The normalisation is baked into the exported model; the HB_16 ONNX graph contains the subtract, divide and clip operations of the normaliser, while the HB_01 graph does not.

Rewards are not rescaled. Advantages are computed by the trainer with Generalized Advantage Estimation [13] using a discount factor of 0.99 and a lambda of 0.95, which is a standard form of reward preprocessing inside PPO.

6.4 Feature Engineering and Feature Extraction

The ten vector observations are engineered features rather than raw state. Instead of giving the network absolute positions, which would differ on every island, they describe the relationship between the beak and the nearest flower in the bird's own terms: which way to go, how far, whether the bird is on the flower's open side, and whether the

beak is pointing in. These are exactly the quantities a hand-written controller would need, so the network does not have to discover them.

The most important feature-engineering decision in the project was made after HB_01. The bird is moved by `Rigidbody.AddForce` on a body of mass 1 with linear damping 2, which gives it real momentum with a time constant of about half a second, and it has to hold its beak within 0.008 units of a nectar trigger for 100 consecutive physics steps to drain a flower. None of the ten observations contains velocity. A policy that cannot see its own velocity can only apply proportional control to a second-order system, and such a controller overshoots and oscillates around its target, which is exactly the behaviour visible in HB_01's flight.

The fix was a single Inspector field: Behavior Parameters, Stacked Vectors, changed from 1 to 3. The trainer then receives the last three decisions' vector observations, 0.1 seconds apart, and the network can difference them into an estimate of velocity. This widened the network input from 10 to 30 vector values, which is why HB_16 had to be trained from scratch rather than continued from HB_01's checkpoint, and why the two models are not interchangeable.

The ray features were left exactly as they were. Each ray produces four numbers: a hit flag for the empty tag, a hit flag for the boundary tag, a no-hit flag and the normalised hit distance. Because only one object carries the boundary tag, the rays extract a single kind of information, distance to solid geometry, and the boundary flag marks the one surface the reward function penalises. Adding flower tags to the sensors would change the number of values per ray and silently break both trained models, so the list was deliberately left unchanged.

6.5 Model Architecture and Algorithm Design

The learning algorithm is Proximal Policy Optimization [14]. PPO alternates between collecting a buffer of experience with the current policy and improving the policy on that buffer. Its key idea is the clipped surrogate objective: the ratio between the new and old probability of each action is clipped to the range $1 - \epsilon$ to $1 + \epsilon$, with ϵ 0.2 at the start of training, so a single update cannot move the policy far from the one that collected the data. An entropy bonus (beta 0.005 at the start) rewards the

policy for keeping some randomness while it is still exploring. In both runs the learning rate, beta and epsilon all decay linearly to near zero at the step budget, which is why the choice of max_steps is fixed before a run starts.

The network, shown in Fig. 6.1, is a multilayer perceptron with two hidden layers of 256 units and the Swish activation (x times sigmoid of x), confirmed by the Sigmoid and Mul operations in the exported graph. For HB_16 its 66 inputs pass through the running normaliser first. The policy head outputs the mean of a Gaussian distribution for each of the five actions, with a learned log standard deviation, and actions are clipped to the range -1 to +1. A value head of the same body estimates expected return for PPO's advantage calculation during training; it is not included in the exported ONNX file, which contains only what inference needs. The HB_16 policy has 84,234 trainable parameters and the HB_01 policy 79,114, the difference being the wider first layer.

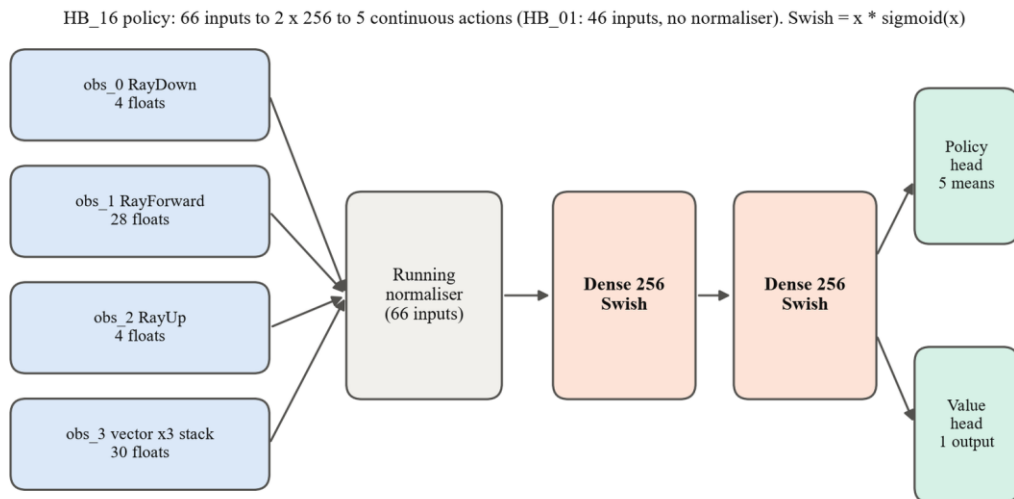


Fig. 6.1 Policy network architecture of the HB_16 model

Table 6.2 lists the trainer configuration of both runs side by side. Every difference was chosen for a stated reason: the step budget because HB_01 ran out of it, the larger batch and buffer because 112 agents fill a buffer far faster, normalisation because of the badly scaled distance input, and the summary and checkpoint settings because of the higher step rate on the server.

Table 6.2 PPO trainer configuration of HB_01 (trainer_config.yaml) and HB_16 (trainer_config_16.yaml)

Parameter	HB_01	HB_16
trainer_type	ppo	ppo
batch_size	2,048	4,096
buffer_size	20,480	40,960
learning_rate	3.0e-4, linear schedule	3.0e-4, linear schedule
beta (entropy)	5.0e-3	5.0e-3, linear schedule
epsilon (clip)	0.2	0.2, linear schedule
lambda (GAE lambda)	0.95	0.95
num_epoch	3	3
normalize	false	true
hidden_units x num_layers	256 x 2	256 x 2
reward signal	extrinsic, strength 1.0, gamma 0.99	extrinsic, strength 1.0, gamma 0.99
max_steps	5,000,000	50,000,000
time_horizon	128	128
summary_freq	10,000	50,000
checkpoint_interval / keep	default (500,000)	250,000 / 40
Stacked Vectors (Unity)	1	3
Parallel arenas	1	112 (28 islands x 4 processes)

6.6 System and Model Implementation

6.6.1 Agile Methodology

Development was organised into five sprints, each corresponding to a coherent group of units from the base Unity Learn course, followed by the extended work on rented hardware and the Android application. Each sprint had its own objectives (listed as user stories in Section 4.5), a functional and an architectural document, an outcome review and a retrospective. In this report those parts have been placed in the chapters where the handout expects them: objectives in Chapter 4, architecture in Chapter 5, the functional work in the subsections that follow, outcomes in Chapters 7 and 8, and retrospectives in Section 8.4, Lessons Learnt.

Table 6.3 Sprint-wise summary of objectives and outcomes

Sprint	Focus	Key Outcome
I	Project setup & foundation	Unity 6 project ready; ML-Agents 4.1.0 installed correctly

Sprint	Focus	Key Outcome
II	Core environment scripting	Flower.cs and FlowerArea.cs implemented and verified
III	Agent development & scene wiring	HummingbirdAgent.cs complete; manual flight verified
IV	Training & inference	5,000,024-step PPO run; Mean Reward 41.29; ONNX inference verified
V	Minigame & wrap-up	Player-vs-trained-Opponent minigame fully playable end-to-end

6.6.2 Project Setup

A new Unity project ('Machine Learning', URP template, Unity 6000.4.7f1) was created at C:\Users\rohit\Machine Learning. The course's HummingbirdScene_1.0.unpackage was imported, providing the Training scene, hummingbird and flower meshes/materials, and the floating island prefab. The Default Contact Offset physics setting (relocated in Unity 6 from the Shared tab to the GameObject tab under Physics Settings, verified directly against Unity's official 6000.1 documentation rather than assumed from the (Unity 2019.3-era) tutorial) was set to 0.001 as instructed. Scene lighting, initially very dim, was corrected via Window → Rendering → Lighting → Environment tab, by assigning a Skybox Material and regenerating lighting.

The ML-Agents package was then installed via Package Manager's "Install package by technical name" dialog. This step carries a documented version trap: Unity 6's Package Manager offers ML-Agents 2.0.1 by default, which is deprecated, and version 3.0.0 is known to be broken on Unity 6000.x (confirmed "Sentis does not exist in namespace" errors and Package Manager metadata rejection). The package was instead installed explicitly by technical name (com.unity.ml-agents), which resolved to version 4.1.0, signed by Unity Technologies, sourced from the Unity Registry, with a Minimum Editor Version of 6000.0, compatible with the project's 6000.4.7f1. Unity's Inference Engine (formerly "Sentis", itself the successor to the older Barracuda engine) was confirmed to install automatically as a dependency, requiring no separate step.

Fig. 6.2 shows the resulting Training scene, rendered for this report from the project in batch mode: a single floating island with its tree, rocks, bushes, path and flower plants.



Fig. 6.2 The Training scene: one floating island with 120 flower plants and 360 flowers

Finally, an empty Scripts folder was created at Assets/Hummingbird/Scripts, containing three empty MonoBehaviour templates (Flower.cs, FlowerArea.cs, and HummingbirdAgent.cs) verified to compile cleanly (0 errors, 0 warnings) before any real logic was added.

6.6.3 Environment Scripts

Flower.cs exposes a full and an empty colour, a nectar collider, read-only FlowerUpVector and FlowerCenterPosition properties, and an auto-implemented NectarAmount property. Its core method, Feed(amount), clamps the amount actually taken to what remains, disables both colliders once nectar reaches zero, and swaps the flower's material colour to indicate an empty flower; ResetFlower() reverses this. The relevant excerpt is shown below.

```
public float Feed(float amount)
{
    // Track how much nectar was successfully taken (cannot take more than is
    // available)
    float nectarTaken = Mathf.Clamp(amount, 0f, NectarAmount);
    NectarAmount -= amount;
    if (NectarAmount <= 0)
    {
        NectarAmount = 0;
        flowerCollider.gameObject.SetActive(false);
        nectarCollider.gameObject.SetActive(false);
        flowerMaterial.SetColor("_BaseColor", emptyFlowerColor);
    }
}
```

```

    }
    return nectarTaken;
}

```

A version-relevant fact confirmed directly from the imported material asset: the URP shader colour property is `_BaseColor`, not the legacy Built-in Render Pipeline property `_Color`, using the wrong property name would have silently failed to recolour the flower.

`FlowerArea.cs` maintains a list of flower plants, a dictionary mapping each nectar collider to its owning `Flower` (for $O(1)$ lookup from a physics collision), and a lazily-populated `Flowers` list. The list is deliberately populated on first access rather than unconditionally in `Awake()` or `Start()`, to avoid a race condition where an external caller (for example the agent, or later the `GameManager`) might request the flower list before Unity has finished calling every object's own `Awake()`.

```

private List<Flower> flowers = new List<Flower>();
private bool flowersFound = false;

private void EnsureFlowersFound()
{
    if (flowersFound) return;
    flowersFound = true;
    FindChildFlowers(transform);
}

```

The collections in this excerpt are created as field initialisers, and `FlowerArea` has no `Awake` method at all. That is the result of a bug found much later, when islands started being created at runtime by the spawner and the race. Unity does not run every `Awake` before any `OnEnable` when it activates a newly created hierarchy; it interleaves them object by object. The agent's `OnEnable` could therefore start an episode, which resets the flowers, before `FlowerArea`'s own `Awake` had created its lists, and the first island built at runtime crashed. Field initialisers run when the component is constructed, ahead of both `Awake` and `OnEnable` on every path, so the race cannot occur. `Awake` was deleted rather than left empty, because re-creating the collections in `Awake` after an early scan had already filled them would silently empty the arena with no error at all.

6.6.4 Agent Script

HummingbirdAgent.cs is the core learning agent, extending Unity ML-Agents' Agent base class. Its full implementation (435 lines) is reproduced in Appendix 3; its key methods are summarised here.

Initialize() acquires the Rigidbody and the parent FlowerArea, and sets MaxStep to 0 (unlimited) outside of training mode. OnEpisodeBegin() resets the flowers (training only), zeroes the agent's linear and angular velocity, and moves the agent to a safe random position; 50% of the time directly in front of a flower during training, to accelerate early learning, and always in front of a flower during gameplay.

CollectObservations(VectorSensor sensor) supplies exactly ten observations: the agent's local rotation as a normalized quaternion (4 values), a normalized vector from the beak tip to the nearest flower (3 values), a dot product indicating whether the beak is positioned in front of the flower (1 value), a dot product indicating whether the beak is oriented toward the flower (1 value), and the relative distance to the flower normalized by the flower area's diameter (1 value).

```
sensor.AddObservation(transform.localRotation.normalized); // 4
Vector3 toFlower = nearestFlower.FlowerCenterPosition - beakTip.position;
sensor.AddObservation(toFlower.normalized); // 3
sensor.AddObservation(Vector3.Dot(toFlower.normalized,
    -nearestFlower.FlowerUpVector.normalized)); // 1
sensor.AddObservation(Vector3.Dot(beakTip.forward.normalized,
    -nearestFlower.FlowerUpVector.normalized)); // 1
sensor.AddObservation(toFlower.magnitude / FlowerArea.AreaDiameter); // 1
```

OnActionReceived(ActionBuffers actionBuffers) reads five continuous action values; a 3D movement vector applied as a physics force, and pitch/yaw deltas smoothed via Mathf.MoveTowards and applied to the agent's rotation, with the pitch clamped to a maximum of 80° to prevent the bird from flipping upside down. TriggerEnterOrStay(Collider); invoked from both OnTriggerEnter and OnTriggerStay; checks whether the collision is with a nectar-tagged collider within a small radius of the beak tip specifically (not any part of the bird's body), feeds 0.01 nectar per physics tick from the corresponding flower, and, in training mode, adds a reward of 0.01 plus an alignment bonus of up to 0.02 for approaching the flower directly head-on.

OnCollisionEnter(Collision) applies a -0.5 reward, once per collision, for contact with a boundary-tagged collider.

The agent's own manual control path, Heuristic(in ActionBuffers actionsOut), is only invoked when the agent's Behavior Type is set to Heuristic Only, and maps keyboard input (W/A/S/D relative to the bird's own facing direction, E/C for vertical movement, and the arrow keys for pitch/yaw) into the same five-value continuous action array used by the trained policy, meaning the exact same OnActionReceived() code path is exercised whether the actions originate from a human or from the neural network.

Fig. 6.3 shows the agent in the Training scene beside one of the flower plants it feeds from.



Fig. 6.3 The hummingbird agent beside a flower plant in the Training scene

Fig. 6.4 shows the agent's components in the Inspector: the sphere and capsule colliders that give the bird its physical body, the HummingbirdAgent script with its Max Step of 5000, move force of 2 and pitch and yaw speeds of 100, and the Behavior Parameters component with a vector observation size of 10, 5 continuous actions, the model slot, the Burst inference device and the behaviour type.

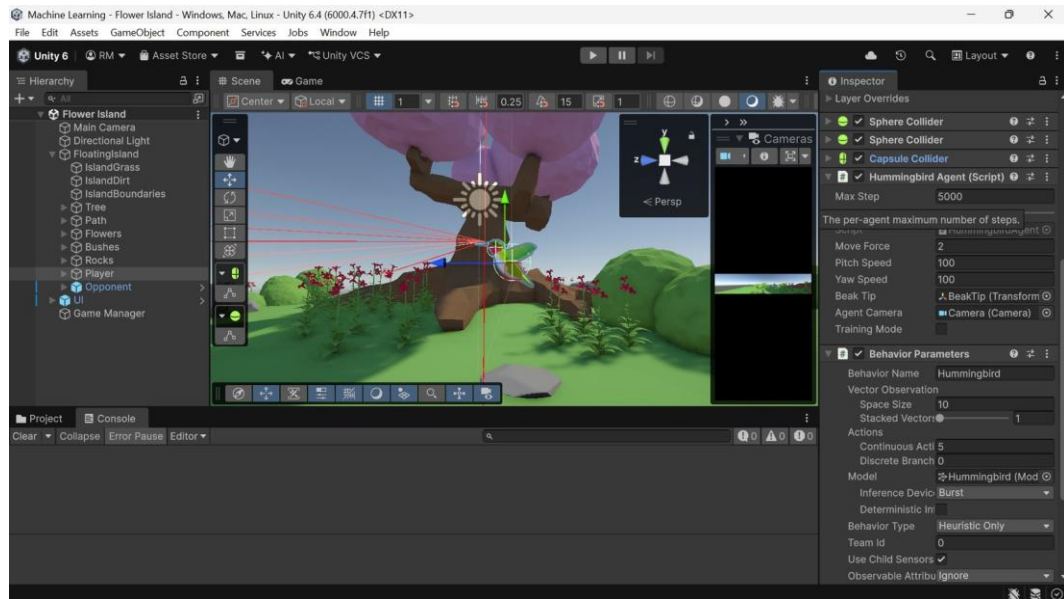


Fig. 6.4 Hummingbird agent colliders, agent parameters and Behavior Parameters in the Inspector

The ray sensors are shown twice. Fig. 6.5 is the Scene view with the sensors' own debug gizmos. Fig. 6.6 was rendered for this report directly from the sensors' ray geometry, using the same raycast calls the sensors use: red rays end on the boundary and yellow rays on untagged geometry, and a ray that reached its full 20-unit length would be drawn in blue. The measured result for this pose was nine rays, all of them hitting something, five on the boundary and four on untagged geometry, at distances from 0.65 to 11.02 units.

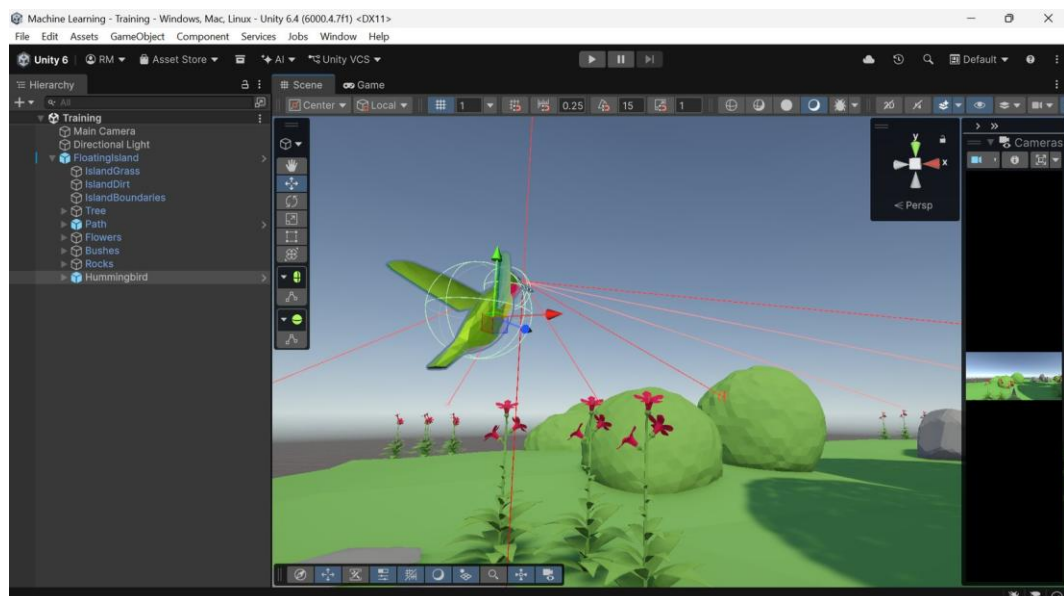


Fig. 6.5 Ray Perception Sensor 3D gizmos visible in the Scene view

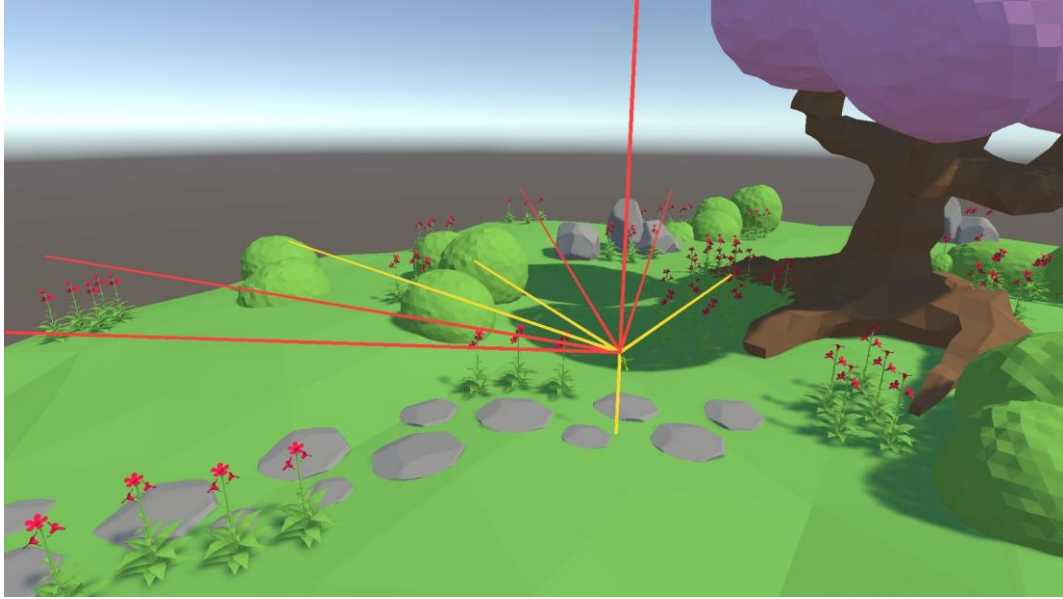


Fig. 6.6 The nine ray sensors rendered from their own ray geometry

Manual (Heuristic) flight testing confirmed all controls responsive, with ray gizmos correctly turning red on contact and the Console remaining clean (0 errors, 0 warnings) throughout. Two genuine API migrations were required and confirmed via source-level verification rather than assumption.

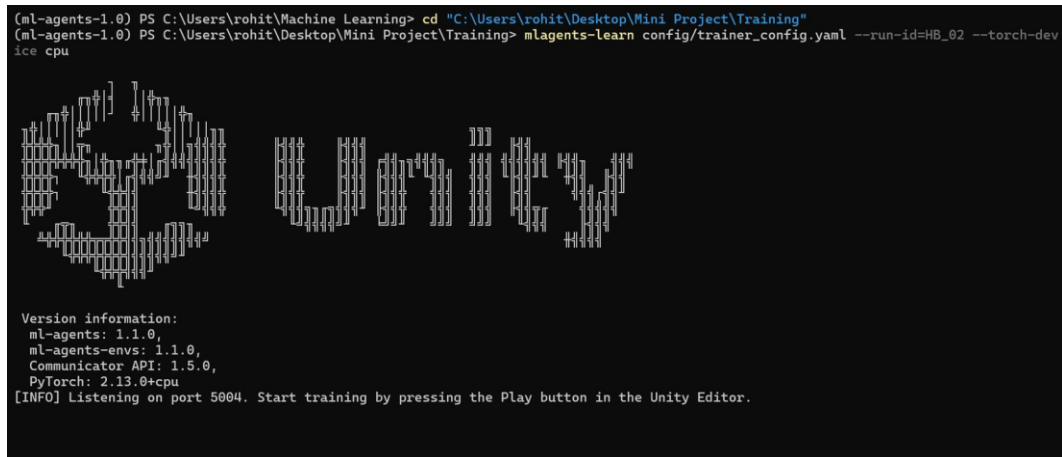
- `float[]`-based `Heuristic(float[] actionsOut)` and `OnActionReceived(float[] vectorAction)` from the 2019.3-era tutorial were migrated to the current `ActionBuffers`-based signatures, `Heuristic(in ActionBuffers actionsOut)` and `OnActionReceived(ActionBuffers actionBuffers)`, reading and writing through `ActionSegment<float>` indexers instead of raw arrays.
- `rigidbody.velocity`, used throughout the tutorial transcript, was migrated to `rigidbody.linearVelocity`, since Unity 6 deprecated the former in favour of the latter (`angularVelocity` is unaffected).

6.6.5 Training and Inference on the Laptop

A dedicated Anaconda environment running Python 3.10.12 was created, satisfying the PyPI `mlagents` package's version constraint (3.10.1–3.10.12), and kept entirely separate from two other, unrelated Python installations already present on the machine (Python 3.12.8 and 3.14.5, used for other coursework). PyTorch was installed via the plain, CPU-only `pip3 install torch` (no CUDA index URL), since the machine's discrete GPU (an NVIDIA MX130) is permanently disabled at the Device Manager/BIOS

level after previously causing system crashes, meaning both the Unity Editor and ML-Agents training run entirely on integrated graphics and CPU.

The PPO trainer configuration used for this run is the HB_01 column of Table 6.2. Fig. 6.7 shows `mlagents-learn` started from the `ml-agents-1.0` environment and waiting for the Editor's Play button.

A terminal window with a black background. At the top, there are two lines of PowerShell commands and their outputs. Below the commands is a large, stylized white logo of the word "Unity". Underneath the logo, there is a section titled "Version information:" followed by several lines of text listing software versions. At the bottom, there is a line of text in brackets indicating it is listening on port 5004.

```
(ml-agents-1.0) PS C:\Users\rohit\Machine Learning> cd "C:\Users\rohit\Desktop\Mini Project\Training"
(ml-agents-1.0) PS C:\Users\rohit\Desktop\Mini Project\Training> mlagents-learn config/trainer_config.yaml --run-id=HB_02 --torch-device cpu

Version information:
ml-agents: 1.1.0,
ml-agents-envs: 1.1.0,
Communicator API: 1.5.0,
PyTorch: 2.13.0+cpu
[INFO] Listening on port 5004. Start training by pressing the Play button in the Unity Editor.
```

Fig. 6.7 `mlagents-learn` running in a terminal, waiting for the Unity Editor

On this laptop training could not run as the course intended. The course's default of eight islands froze the machine, four islands were still too tight, and the run was completed with a single island. It was also stopped and resumed six times to fit around the laptop's other uses. Stopping Play in the Editor does not stop the trainer cleanly, so a PowerShell script was written that attaches to the trainer's console and sends it a genuine Ctrl+C through the Win32 `GenerateConsoleCtrlEvent` call, which makes `mlagents-learn` save a checkpoint at the exact current step before exiting.

The trained model was exported as an `.onnx` file (the current model format used by Unity's Inference Engine, replacing the older `.nn` / Barracuda format referenced in the original tutorial) copied into `Assets/Hummingbird/NN Models/` and wired into the Hummingbird prefab's Behavior Parameters Model field. Pressing Play with no Python trainer running confirmed the bird flying and feeding entirely under inference, using the exported neural network alone.

Fig. 6.8 shows this check: the Console reports that the trainer could not be reached on port 5004 and that the agent will perform inference instead, while the bird feeds in the Game view.

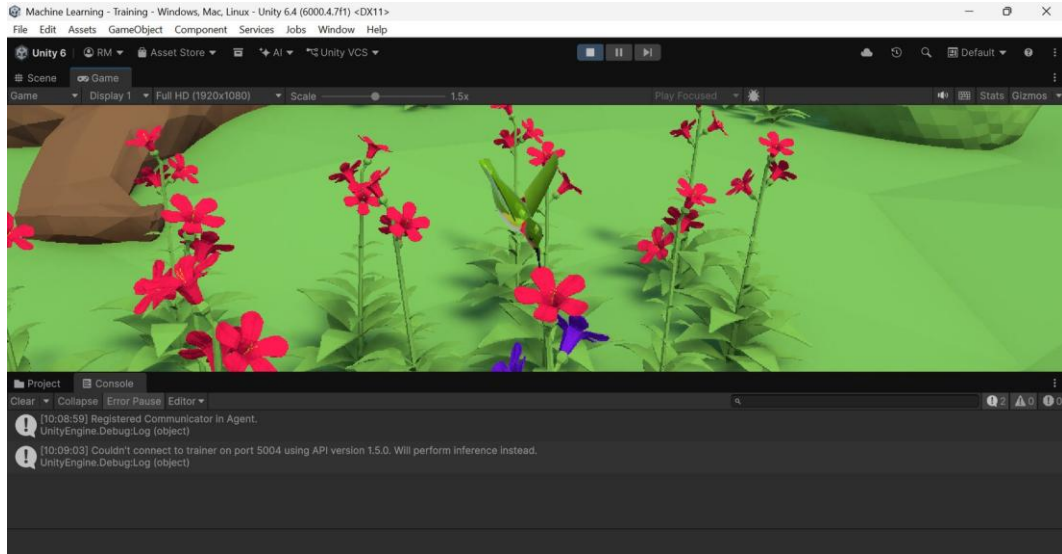


Fig. 6.8 Trained agent performing inference in the Editor with no Python trainer connected

6.6.6 Minigame

GameManager.cs drives a simple state machine (MainMenu → Preparing → Playing → Gameover) via a public enum GameState. StartGame() runs a four-step countdown coroutine ("3", "2", "1", "Go!") before unfreezing both agents; Update() checks, every frame while Playing, whether the timer has elapsed or either agent has reached the configured maxNectar threshold, ending the game and declaring a winner based on a simple comparison of NectarObtained between the two agents (a tie is awarded to the human player).

```
if (player.NectarObtained >= opponent.NectarObtained)
    uiController.ShowBanner("You win!");
else
    uiController.ShowBanner("ML-Agent wins!");
```

The scene contains two nested instances of the Hummingbird prefab (one configured as Player (Behavior Type: Heuristic Only, keyboard-controlled) and one as Opponent (Behavior Type: Inference Only, running the trained Hummingbird.onnx, Training Mode off)), both driven by the same underlying HummingbirdAgent.cs script used throughout the project, demonstrating that a single agent implementation correctly supports human control, training, and trained inference without modification.

Fig. 6.9 shows a match in progress from the player bird's camera, with the timer and both nectar bars live.

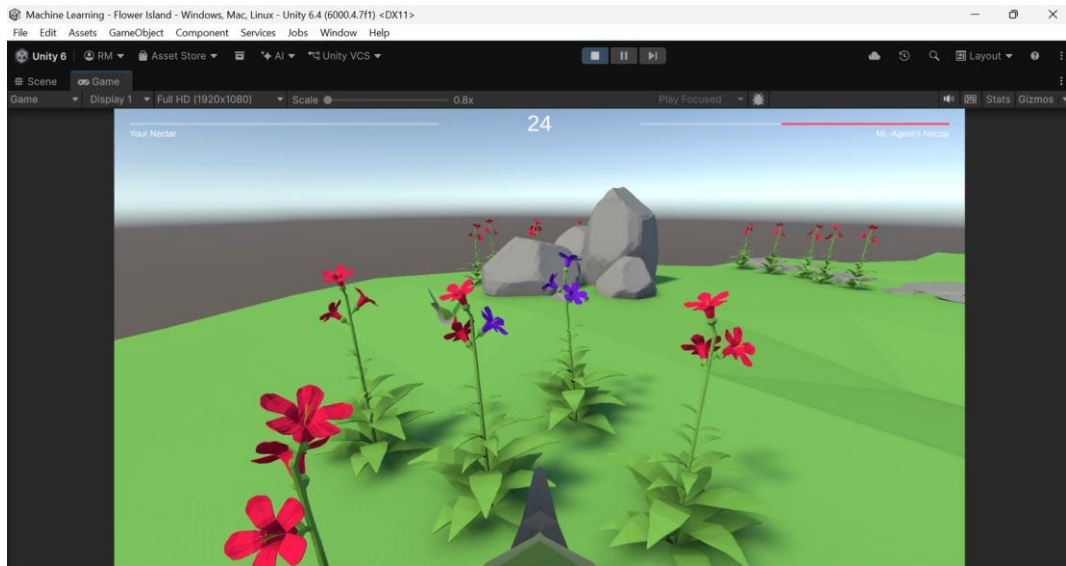


Fig. 6.9 Flower Island minigame, player vs. trained opponent, with the live UI

Fig. 6.10 shows the Flower Island scene from outside, and Fig. 6.11 the same scene in the Editor with its hierarchy of Player, Opponent, UI and Game Manager objects. Both birds in this scene run with training mode off, which means their episodes never end and the flowers are never refilled; the scene is built for play, and all measurement in this report is done in the Training scene instead.



Fig. 6.10 The Flower Island minigame scene rendered from outside the island

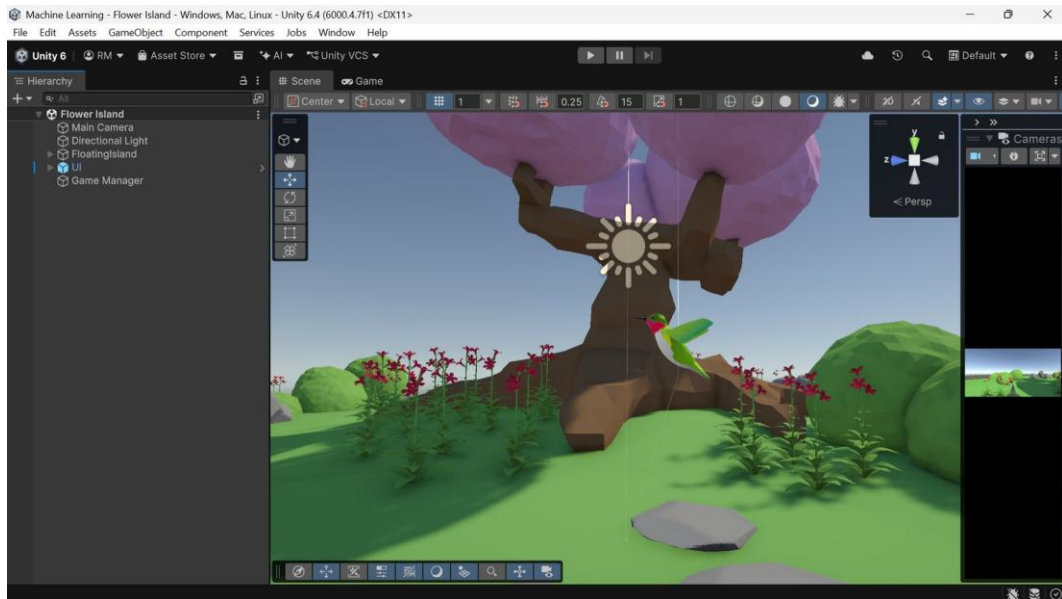
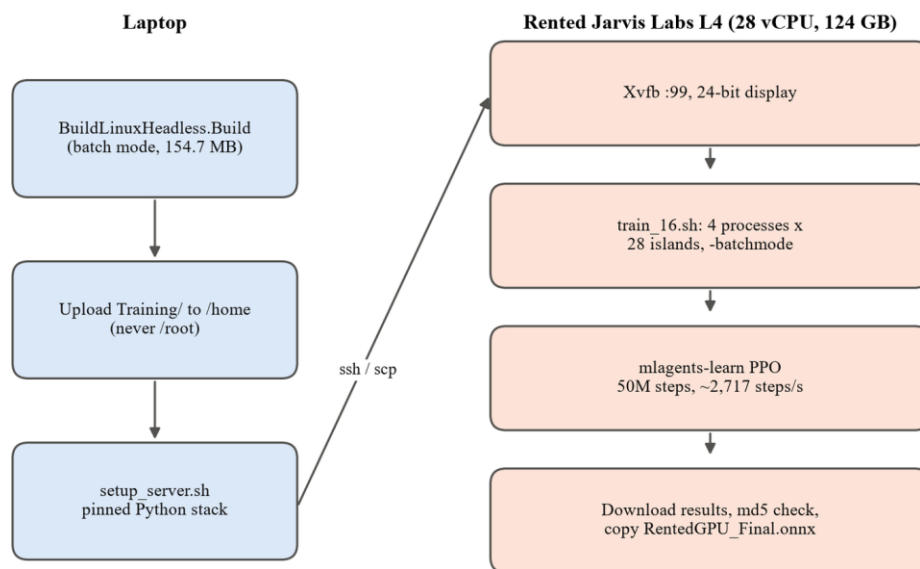


Fig. 6.11 The Flower Island scene in the Unity Editor

6.6.7 Retraining on Rented Cloud Hardware

The second training run needed a way to run many arenas in parallel on a Linux server. Fig. 6.12 shows the pipeline that was built for it, which consists of five pieces.



Resume: re-uploading results/HB_16 lets --resume continue from step 25,442,592

Fig. 6.12 Rented-GPU training pipeline from laptop build to downloaded model

The first piece is IslandSpawner.cs. One island is 3,837 GameObjects, so placing 28 of them in the scene by hand would create a scene of over 107,000 objects, too heavy to edit on the laptop and fixed at one count. The spawner instead reads a --islands N

argument when the player starts and clones the island prefab into a square grid 25 units apart, reusing the scene's own island as the first slot. It logs the count it actually built, so the server's log proves the argument took effect. Fig. 6.13 is a render of the 28-island grid it produces; measured in the Editor, that grid contains 107,438 GameObjects and 10,080 flowers.

```
int count = Mathf.Max(1, ReadIslandCountFromCommandLine(defaultIslandCount));
int columns = Mathf.CeilToInt(Mathf.Sqrt(count));
int rows = Mathf.CeilToInt(count / (float)columns);
for (int i = 0; i < count; i++)
{
    Vector3 position = new Vector3((i % columns) * spacing - xOffset, 0f,
                                   (i / columns) * spacing - zOffset);
    if (i == 0 && existingIsland != null) { existingIsland.transform.position = position;
    continue; }
    GameObject clone = Instantiate(islandPrefab, position, Quaternion.identity);
    clone.name = "FloatingIsland_" + i;
}
Debug.Log("IslandSpawner: spawned " + count + " islands in a " + columns + "x" +
rows + " grid");
```

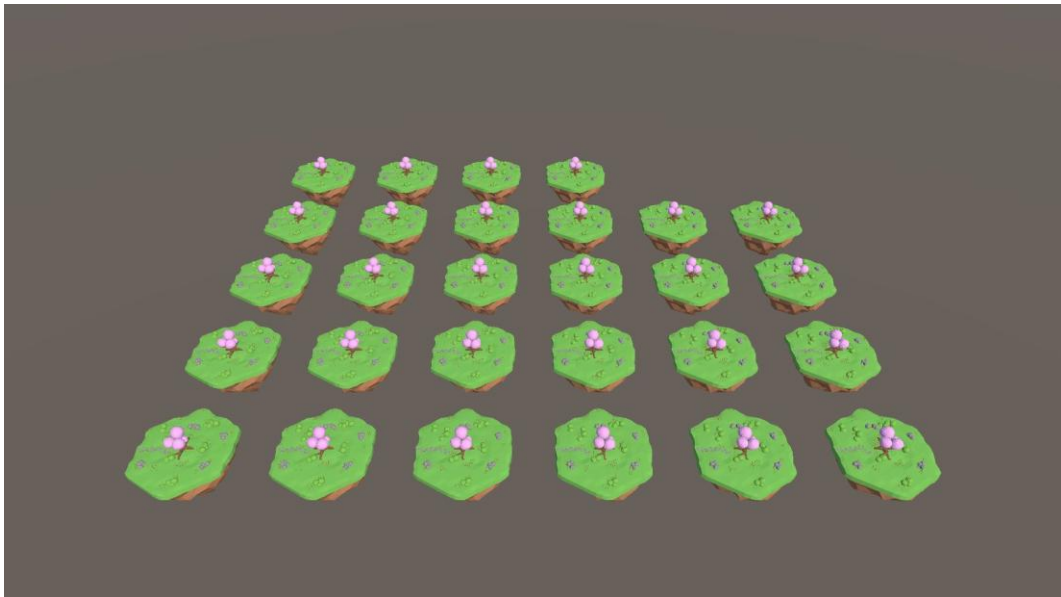


Fig. 6.13 The 28-island grid built by IslandSpawner for one training process

The second piece is BuildLinuxHeadless.cs, an Editor script run in batch mode that builds the Training scene into a 154.7 MB 64-bit Linux player. It names the scene explicitly instead of reading the Build Settings list, which only contained Unity's empty sample scene, and it logs an explicit SUCCESS line because a batch-mode Editor does not reliably report failure through its exit code. The third piece is setup_server.sh, which

rebuilds the entire pinned Python environment of Table 3.2 on a fresh instance in five to ten minutes.

The fourth piece is `train_16.sh`, which has four modes. Smoke runs a short test to prove the setup works. Sweep measures the throughput of several process and island layouts and ranks them. The default mode runs the real training and resumes automatically if results already exist. Status reads the console log from a second terminal and reports steps per second, time remaining, rupee cost and whether the 41.29 baseline has been beaten.

Getting the Unity player to run on the server produced the project's most useful infrastructure finding. The obvious approach, running the player with `--no-graphics`, crashed with a segmentation fault in `PlayerMain` the moment the scene loaded, because Unity 6's null graphics device cannot serve a URP project. Running it under a virtual X display fixed the crash only when the display was 24-bit; at 8-bit there is no GLX and Unity exits with a different error. Finally, with a working display, the player rendered every frame in software through the `llvmpipe` driver, and throughput was only 64 steps per second. Passing `-batchmode` to the player through `--env-args` kept the display but stopped the rendering, and throughput rose to 832 steps per second on the same settings, a factor of 13. The working recipe is therefore a 24-bit `Xvfb` display plus `-batchmode`, and never `--no-graphics`.

The fifth piece is the procedure for interruption. The first session was deliberately stopped at step 25,442,592. The instance was deleted, and later the same day the training folder, including the downloaded `results/HB_16` folder with its `checkpoint.pt` and `training_status.json`, was uploaded to a fresh instance. Running the training script again printed `Resuming training from step 25442592`, and the remaining 24.6 million steps completed in 2 hours 38 minutes at about 2,591 steps per second. Because the learning-rate schedule is tied to `max_steps`, and `max_steps` was never changed, the resumed run is mathematically the same run. The results were downloaded, verified by MD5 checksum against the server and copied into the Unity project as `RentedGPU_Final.onnx`.

6.6.8 Android Build and On-Device Deployment

The Android application uses a separate scene, `Demo.unity`, which is a copy of `Training.unity` with a canvas, the Start button, the two model buttons, a model label and

the presentation toolkit added. Training.unity was left untouched so it can still be shown as the plain training setup, and only Demo.unity is included in the APK. In the demo scene the bird starts disabled, so no sensors exist until Start is pressed and the first model is chosen.

The APK is produced by BuildAndroid.cs, run in batch mode with the Editor closed. The target phone's processor is 64-bit only and Unity reaches 64-bit ARM only through IL2CPP, so the script asserts both settings rather than setting them, and stops with an error if either has drifted. A build that silently used Mono or 32-bit ARM would install and then crash on the phone with nothing useful in the log.

```
var backend = PlayerSettings.GetScriptingBackend(NamedBuildTarget.Android);
if (backend != ScriptingImplementation.IL2CPP)
{
    Fail("scripting backend is " + backend + ", expected IL2CPP (ARM64 requires it).");
    return;
}
if ((PlayerSettings.Android.targetArchitectures & AndroidArchitecture.ARM64) == 0)
{
    Fail("ARM64 is not in target architectures.");
    return;
}
```

The first build took 49 minutes and the second 37.5 minutes. The first paid two one-off costs, re-importing every texture for the Android target and compiling the Inference Engine's GPU compute shaders, which the application never uses because it runs inference on the Burst CPU backend. The final build took 1,130 seconds (19 minutes) and produced an APK of 57,039,972 bytes (54.4 MB) containing only the arm64-v8a library. Unity's build summary reported the output size as 1,212.1 MB, which counts intermediate files; the file on disk is the true figure. During the build, Unity.exe itself used about 70 MB and almost no CPU, because the work was done by child processes: the Bee build backend and about ten parallel clang compilers translating the IL2CPP output, which lowered free memory on the laptop from 2.30 GB to 1.03 GB.

The APK was installed on the Galaxy S24 Ultra with the adb copy bundled in Unity's Android module, which took about 12 seconds, and launched through its launcher intent. Unity 6 uses the GameActivity entry point, so the older UnityPlayerActivity name

does not start it. The application needs no internet permission: in a player build ML-Agents does not attempt a trainer connection at all. On the phone Unity reported 8 cores and about 11 GB of memory, the 360 flower colliders that cost about 2.5 ms per step on the laptop ran smoothly, and every button was confirmed to respond to touch.

6.7 Validation Checks

The code contains a number of explicit checks that protect the correctness of training and deployment. The most important are listed here.

Nectar feeding clamps the amount taken to what is left (the Feed excerpt in Section 6.6.3), so a flower can never go negative or pay out more reward than it holds. When a flower is empty its colliders are disabled, so the agent cannot keep drinking from it.

Every episode starts by moving the agent to a random position that is checked to be clear of colliders, retrying up to 100 times and asserting that a safe position was found:

```
while (!safePositionFound && attemptsRemaining > 0)
{
    attemptsRemaining--;
    // ... choose potentialPosition and potentialRotation ...
    Collider[] colliders = Physics.OverlapSphere(potentialPosition, 0.05f);
    safePositionFound = colliders.Length == 0;
}
Debug.Assert(safePositionFound, "Could not find a safe position to spawn");
```

- CollectObservations writes ten zeros when there is no flower with nectar left, so the observation width never changes.
- OnEpisodeBegin, FreezeAgent and UIController.Awake return early for objects in a scene with an empty name, a signature of the invisible preview copies Unity creates for prefab thumbnails, which otherwise intercepted UI clicks.
- The reward code only counts nectar contact within 0.008 units of the beak tip, not anywhere on the bird's body.
- IslandSpawner falls back to the Inspector value and logs a warning if the --islands value cannot be parsed, and logs the count it actually built.
- Both build scripts name their scenes explicitly, assert their required settings, write an explicit SUCCESS or FAILED line and exit with a non-zero code on failure.

- Downloaded models are compared with the server copy by MD5 checksum before use (results in Chapter 7).

The most important check is structural rather than a test. The two trained models accept different input widths, and ML-Agents only rebuilds its sensors when an agent is disabled and enabled again; changing the model on a live agent leaves the old sensor in place and logs no error. DemoController therefore writes the model and its stacking value in only one method, always together, inside a disable and enable cycle:

```
public void SelectModel(bool rented)
{
    bird.SetActive(false); // OnDisable -> CleanupSensors(), m_Initialized = false
    behaviourParameters.BrainParameters.NumStackedVectorObservations = rented ?
3 : 1;
    behaviourParameters.Model = rented ? rentedGpuModel : onDeviceModel;
    bird.SetActive(true); // OnEnable -> LazyInitialize() -> InitializeSensors()
    bird.GetComponent<Agent>().EndEpisode();
}
```

This was verified in the Editor through the real button methods: Start produced 66 observation values (the stacked 30-value vector plus the three ray sensors), the On-Device button rebuilt the sensors to 46 values, and switching back restored 66. RaceController follows the same rule for the challenger bird.

6.8 User Interface Implementation

The minigame interface is implemented by GameManager and UIController. UIController exposes a button-click delegate that GameManager subscribes to in Start, and methods to set the banner text, the timer and the two progress bars. GameManager calls them from its state machine every frame, as described in Section 5.4. During testing the Start button appeared to do nothing; the cause was a second EventSystem added at the scene root on top of the one inside the UI prefab, and removing it restored click routing. GameManager also disables the URP Rendering Debugger's runtime UI, which can create a third EventSystem of its own.

The phone interface is built almost entirely in code. DemoController handles the authored Start and model buttons. DemoToggles builds the Debug, Trails, Split and Reset buttons at runtime and finds its collaborators itself, so adding the component to the scene is the whole integration. Reset returns the camera, toggles, model and both scores to their

opening state in one tap. `RaySensorVisualizer` draws the nine rays with pooled line renderers and prints the ten observations, recomputed from the same transforms the agent uses rather than read from the live sensor, because reading the sensor's buffer a second time could corrupt the observation the agent is about to send. `FlightTrail` adds a fading ribbon to each bird and clears it when the bird is respawned, so an episode reset does not draw a streak across the island. `RaceController` builds a second island at scene load, disabled, so the race can start without a visible freeze, and gives each bird its own camera and half of the screen. `TouchCameraInput` routes each drag or pinch to whichever camera's viewport contains it, so split screen needs no special case.

All five runtime canvases are configured by `UiScaling`, the single place that sets a `CanvasScaler`. It exists because of three problems that were invisible in the Editor and only appeared on the phone's 3120-pixel-wide screen: a canvas created in code defaults to constant pixel size and was unreadably small; two canvases with the same reference resolution but different match settings resolved to different sizes, so buttons meant to share a row did not; and anything positioned from the canvas centre moved off screen when the aspect ratio changed. Every canvas now uses `scale-with-screen-size` at 1920 by 1080 with a match value of 0.5, and the button row is anchored to the bottom edge. The resulting screens are shown in Appendix 2.

CHAPTER 7

TESTING AND EVALUATION

This chapter describes how the system was tested and evaluates the two trained models. The project does not use an automated unit-testing framework such as the Unity Test Framework; testing was carried out by direct verification in the Unity Editor, by scripted checks run for this report against the real project and model files, and on the target phone. Every result below is taken from one of those sources or from the trainers' own logs.

7.1 Test Cases

Table 7.1 lists the test cases, the expected and actual outcome of each, and the result. Where a test first failed, the result column records the outcome after the fix, and the fix is described in the sections that follow or in Section 8.4.

Table 7.1 Test cases and results

ID	Test	Expected	Actual	Result
TC01	Heuristic flight with W/A/S/D, E/C and arrow keys	Bird moves and turns; rays react	All controls responsive, 0 console errors	Pass
TC02	Beak contact with a flower	Nectar drains, colliders off and colour changes at 0	As expected	Pass
TC03	FlowerArea scan with undefined tag	Scan finds all flowers	UnityException until tags defined	Pass after fix
TC04	Boundary collision in training mode	-0.5 reward	Negative early rewards in both runs	Pass
TC05	mlagents-learn connects to the Editor	Training starts on Play	Connected on port 5004	Pass
TC06	Inference Only with no trainer running	Bird flies and feeds	Flies and feeds, clean console	Pass
TC07	Minigame Start, countdown, bars, winner	Full game playable	Worked after duplicate EventSystem removed	Pass after fix
TC08	ONNX input widths of six models	10 for HB_01 era, 30 for HB_16 era	As expected	Pass
TC09	MD5 of deployed models vs training output	Byte-identical	5 of 5 matched	Pass
TC10	Forward pass of each model	5 actions in [-1, 1]	6 of 6 in range	Pass

ID	Test	Expected	Actual	Result
TC11	Scene configuration read-back	360 flowers, 1 boundary, correct parameters	As expected	Pass
TC12	IslandSpawner with --islands 28	28 islands in a 6 x 5 grid	28 islands, 107,438 objects	Pass
TC13	Headless player on the server	Player runs without a monitor	Segfault with --no-graphics	Pass after fix
TC14	Resume from step 25,442,592	Continues same schedule	Resumed, finished at 50,002,848	Pass
TC15	Runtime model switch	Sensors rebuilt to match model	66 -> 46 -> 66 values	Pass
TC16	APK build with asserts	SUCCESS line, ARM64 only	54.4 MB arm64-v8a APK	Pass
TC17	APK on the Galaxy S24 Ultra	All buttons respond, smooth	Confirmed on device	Pass
TC18	Split-screen race challenger island	360 flowers registered	Only 2 of 360 before fix	Pass after fix

7.2 Unit Testing

Each component was first verified on its own. Flower.cs was checked by flying the bird into a single flower under heuristic control and watching the nectar drain, the colliders switch off and the colour change at zero (TC02). FlowerArea.cs was checked by confirming that its scan registered every flower on the island; the scan exposed Unity's behaviour of throwing an exception from CompareTag for a tag that does not exist, rather than returning false (TC03), which was fixed by defining the flower_plant, nectar and boundary tags before the code ran. HummingbirdAgent.cs was checked under heuristic control (TC01).

For this report the trained models were also tested as units in their own right, outside Unity, with a short script using ONNX Runtime. Each model file was loaded, its input shapes were read, it was given random observations of the right shape, and its outputs were checked. Its MD5 checksum was compared with the file the trainer exported. Table 7.2 shows the results. Every deployed model is byte-identical to the trainer's output, so the files in the Unity project are exactly the trained policies; every model accepts the input width its training configuration implies; and every model returns five actions within the legal range.

Table 7.2 Model integrity tests (TC08 to TC10)

Model file	Source checkpoint	obs_3 width	MD5 match	Actions in [-1, 1]
Hummingbird_Random_Baseline.onnx	HB_baseline_random, 13,896	10	Yes	Yes (-0.016 to 0.005)
Hummingbird_Early_500k.onnx	HB_01, 499,896	10	Yes	Yes
Hummingbird_Mid_2500k.onnx	HB_01, 2,499,896	10	Yes	Yes
OnDevice_Final.onnx	HB_01, 5,000,024	10	Yes	Yes
Hummingbird_HB16_25M.onnx	HB_16 partial, about 25.4M	30	not re-checked	Yes
RentedGPU_Final.onnx	HB_16, 50,002,848	30	Yes	Yes

The near-zero outputs of the random-baseline model are themselves a useful check: an untrained network with small initial weights produces actions close to zero, which is why the random-policy bird barely moves. The Unity scenes were tested in the same way by an Editor script that opened each scene and read back its configuration (TC11). It confirmed 360 flowers on 120 flower plants per island, 3,837 GameObjects per island, exactly one boundary-tagged object, a Rigidbody of mass 1 with linear damping 2 and no gravity, a decision period of 5, a Max Step of 5000, and the model and stacking pairs described in Chapter 6 in all three scenes.

7.3 Integration Testing

Integration testing checked the joins between components. The first was the join between Unity and the trainer: mlagents-learn had to connect to the Editor over gRPC (TC05). It initially failed to start at all because the installed `setuptools` no longer provided `pkg_resources`, and later failed intermittently on worker restarts because of a version check inside the ML-Agents environment code that used Python's deprecated `StrictVersion` class. Both were traced to their source and fixed, as recorded in Section 8.4.

The second join was between training and inference: the exported ONNX file had to load in Unity's Inference Engine and drive the bird with no trainer running (TC06). The third was between the spawner and the trainer, which passes `--islands` through `--env-args`; the spawner's own log line and the island count in the Editor render confirmed it (TC12). The fourth was the contract between a model and the Behavior Parameters stacking value, tested through the demo's own buttons (TC15).

The fifth join was between mlagents 1.1.0 and the server's Python packages, and it produced the clearest example of an integration failure. The ONNX export at the end of a smoke test failed because PyTorch 2.13's new exporter imports onnxscript. Installing onnxscript silently upgraded numpy, protobuf and onnx, after which mlagents would not even import, with three different errors appearing one after another as each was fixed. The resolution was to change the package that demanded the dependency rather than add it: PyTorch 2.2.2 uses the legacy exporter and needs no onnxscript at all. The test that proved the fix was end to end: a short run was allowed to finish and an ONNX file was confirmed to exist on disk.

7.4 System Testing

System tests exercised the whole pipeline. Both training runs are themselves system tests: each started from a clean configuration, ran to its full step budget and exported a model that then worked in Unity. HB_16 additionally tested recovery, being stopped at 25,442,592 steps on one instance and completed on another (TC14).

The trained policy's ONNX export was verified under two independent deployment contexts. First, in the Training scene, with Behavior Type set to Inference Only and no live Python trainer connected, the hummingbird was confirmed flying and feeding autonomously using only the exported network. Second, in the separately-built Flower Island minigame, the same trained model, running as the Opponent, was confirmed competing correctly against a human, keyboard-controlled Player under the shared timer and nectar-threshold win conditions, with the Console remaining clean throughout a full playthrough.

An approximate per-episode flower count was also estimated from the final reward and the known reward formula (41.29 reward implies roughly 18–25 flowers foraged per episode, best estimate ~20). This is presented explicitly as a reward-derived estimate rather than a directly measured count, since a precise count would require either a dedicated StatsRecorder-based counter (future work, Chapter 8) or manual observation via the project's own custom NectarCounterUI during a live episode. The Hummingbird.onnx model verified above was later superseded by the extended training run described in Section 7.5.3; both placements described in this section were verified again with the newer model following that run.

The Android application was tested on the Galaxy S24 Ultra itself (TC16, TC17): the APK installed, launched, opened on the Start screen, and responded to every button, including live switching between the models, the debug view, trails, the split-screen race and Reset. The race exposed one further defect (TC18). When the challenger bird was enabled in the same frame as its island, its flower scan ran before the flowers' own Awake methods had assigned their nectar colliders, registered 2 flowers out of 360 and stopped. The symptom was a challenger model that looked hopeless, not an error message. Enabling the bird one frame after its island fixed it.

7.5 Model Evaluation

7.5.1 On-Device Baseline, HB_01

The final training run (identified as HB_01) reached its configured budget of 5,000,000 steps, completing at 5,000,024 steps, with a final logged Mean Reward of 41.29; read directly from the trainer's own training_status.json output rather than estimated. Table 7.3 shows the reward recorded at each retained checkpoint over the final portion of training, demonstrating a clear, still-improving upward trend through to the end of the run.

Table 7.3 Checkpoint rewards recorded during the HB_01 training run

Step	Mean Reward
4,001,896	29.22
4,247,024	38.23
4,499,920	34.02
4,999,920	41.09
5,000,024 (final)	41.29

Fig. 7.1 plots every logged value of HB_01's mean cumulative reward together with all twelve saved checkpoints. The five checkpoints with a reward recorded in training_status.json are drawn as open circles; for the seven earlier checkpoints the reward is read from the logged curve at the same step. The highest single logged summary was 47.94 at step 4,250,000, a noisy peak; the checkpointed values are the fairer measure. Fig. 7.2 is the TensorBoard view of the same run, captured on the laptop at the end of training.

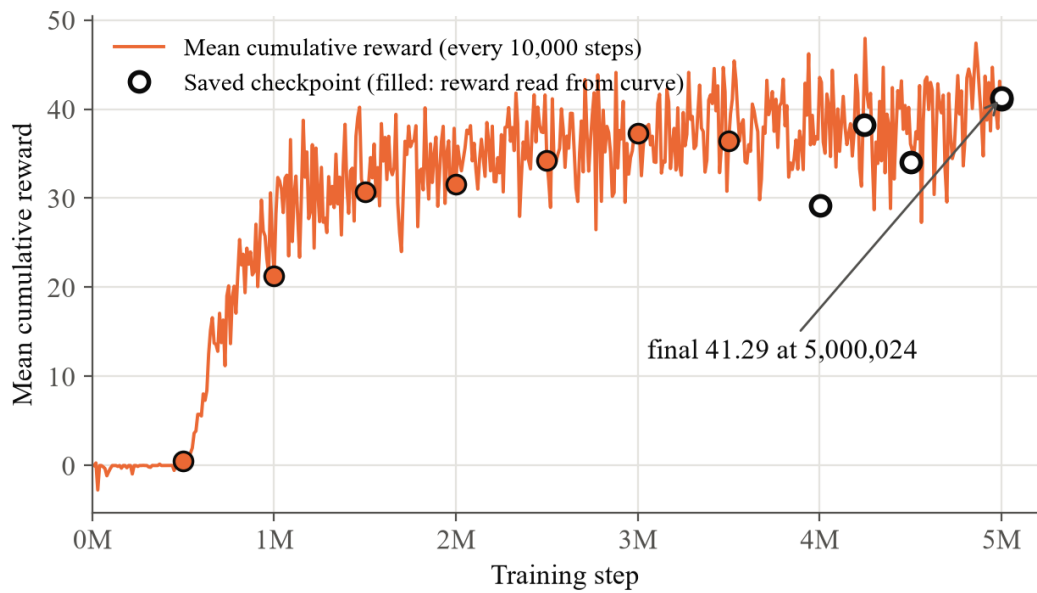


Fig. 7.1 HB_01 mean cumulative reward with saved checkpoints marked (0 to 5,000,024 steps)

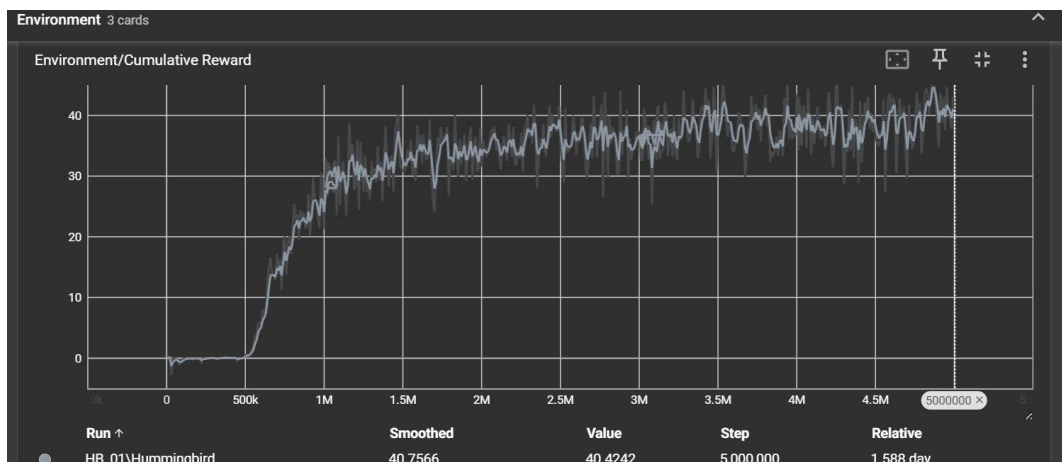


Fig. 7.2 TensorBoard view of HB_01's Environment/Cumulative Reward

The reward curve's shape is fully explained by the agent's own reward function and training dynamics rather than being anomalous: it begins near zero, dips slightly negative in the earliest steps as the untrained agent frequently collides with the island boundary (each collision penalised at -0.5 via `OnCollisionEnter`), then climbs steadily from roughly the 1,000,000-step mark onward as the policy learns to associate proximity and alignment with flowers to positive reward, levelling off in the high 30s to low 40s by the end of training.

7.5.2 Theoretical Ceiling and Interpretation of the Reward

An earlier version of this analysis estimated a ceiling from the number of flowers, assuming about 20 flowers per island and arriving at roughly 60. That assumption was wrong. The island was measured directly in the Editor and holds 360 flowers on 120 flower plants (TC11), so flowers are never the limiting resource within an episode, and the flower-based ceiling does not apply. The binding limit is the episode length. An episode is 5,000 physics steps and the most the agent can earn in one step is 0.03 (0.01 for drinking plus the full 0.02 alignment bonus), so the theoretical maximum is $5,000 \times 0.03 = 150$. That maximum assumes the bird drinks on every single step, which it cannot, because it must also fly between flowers at a top speed of roughly 1 unit per second and must leave each flower once it is drained. Allowing for that travel time, a realistic score for a very good policy is roughly 90 to 110.

Against that band, HB_01's final reward of 41.29 is about 40 percent of what is realistically reachable, and HB_16's 96.58 sits inside the band. A separate live measurement of the RentedGPU_Final model in the Training scene recorded 39.53 reward by step 1,869 of an episode, or 0.0212 reward per step, which projects to about 105.7 over a full episode and is consistent with the training mean. The earlier flower-based estimate only appeared compatible with HB_01 because HB_01's score was low enough not to contradict it.

The remaining gap to the ceiling is attributable to four identifiable, non-bug factors: the Decision Requester's period of 5 physics steps introduces a small reaction-time lag near tightly-clustered flowers; moveForce=2 combined with the RigidBody's linear damping gives the bird real momentum, causing slight overshoot on approach; the reward function contains no explicit efficiency or smoothness incentive beyond the nectar reward and boundary penalty; and training on a single island (rather than several in parallel) yields somewhat less varied experience per unit of wall-clock time. None of these represent implementation defects, they are expected trade-offs of the reward design and the single-island, hardware-constrained training configuration.

7.5.3 Why HB_01 Stopped at 41.29, and the HB_16 Retraining

Two findings from HB_01's own artefacts explain its result. First, it never plateaued. Its last checkpoints read 29.2, 38.2, 34.0, 41.1 and 41.3, still climbing when

the run reached `max_steps`. With a linear learning-rate schedule the learning rate falls to zero exactly at `max_steps`, as Fig. 7.3 (a) shows for both runs, so the last stretch of HB_01 was barely learning, and its flat-looking tail was the schedule ending rather than the task being solved. Second, as explained in Section 6.4, the agent could not observe its own velocity.

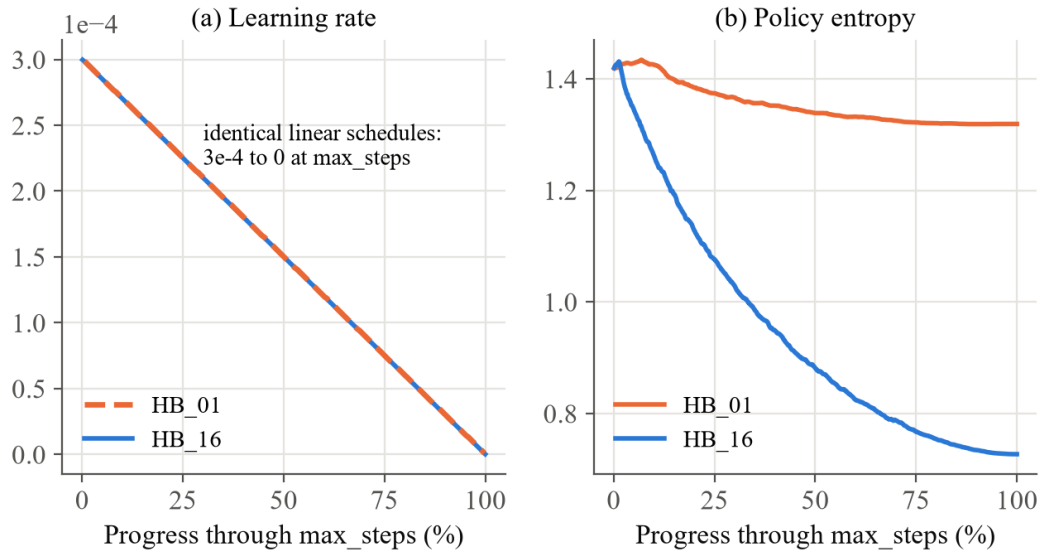


Fig. 7.3 (a) Learning-rate schedule and (b) policy entropy of both runs against progress through `max_steps`

Part (b) of Fig. 7.3 shows the policy entropy, a measure of how random the policy's actions still are. HB_01's entropy fell only from 1.42 to 1.32 over its whole run, so it ended still acting quite randomly. HB_16's fell from 1.42 to 0.73, the sign of a policy that has committed to precise, low-jitter control, which is what hovering in front of a flower for 100 consecutive steps requires.

Following completion of the on device training run described above, a further training run, identified as HB_16, was carried out using rented cloud GPU infrastructure. Two motivations drove this extension. First, a genuine gap in the agent's observation space was identified after the original run concluded: the ten observation values described in Chapter 5 give the network no direct signal for its own velocity, so a policy driven by a physics based force (`Rigidbody.AddForce` with mass 1 and linear damping 2, giving the bird real momentum) can only apply proportional control to a system it cannot fully observe, producing overshoot near flowers. The fix identified was to raise the Behavior Parameters' Stacked Vectors setting from 1 to 3, giving the network three consecutive 0.1

second frames it can numerically difference into an implicit velocity estimate; this expands the network's input from 10 to 30 values and, because it changes the network's architecture, requires training entirely from step 0 rather than resuming from HB_01's saved checkpoint. Second, the original 5,000,000 step budget was itself constrained primarily by the laptop's CPU only hardware (Section 8.2) rather than by any property of the task, so a substantially larger budget of 50,000,000 steps, a tenfold increase, was set for this run to test how far the same reward function and environment could be pushed given adequate compute.

Training for HB_16 was carried out on a rented Jarvis Labs L4 GPU instance (28 vCPU, 124GB RAM) rather than on the project's own hardware. Because Unity's physics simulation is bound to the CPU rather than the GPU, throughput was scaled by running many simulation arenas in parallel across the rented vCPUs instead of relying on GPU compute; a short sweep across several arena counts and process layouts identified 112 parallel arenas (28 islands per Unity process, four processes) as the fastest configuration measured, reaching roughly 3,237 steps per second in the sweep against HB_01's on device throughput of roughly 44 steps per second. The full 50,000,000 step run was completed across two sessions: the first was stopped intentionally at step 25,442,592 when the rented instance needed to be released, and the second resumed cleanly from the saved checkpoint on a freshly rented instance the same day, confirmed directly from the trainer's own resume log message. No cost was incurred during the roughly 7.5 hour gap between sessions since the first instance had already been terminated. Table 7.6 in Section 7.7 compares wall clock time and throughput directly between the two runs, computed from each run's own TensorBoard event file timestamps rather than estimated.

HB_16 completed the full 50,000,000 step budget with a final Mean Reward of 96.58 and a best recorded value of 97.728, exceeding the HB_01 baseline of 41.29 by 56.44, an improvement of roughly 2.35 times. As with HB_01 (Section 7.5.1), the run had not plateaued at the point the step budget ended: recorded values through the final portion of training continued rising until the linear learning rate schedule, which anneals to zero exactly at the configured step budget, brought further learning to a near stop. The sharpest gains were concentrated in approximately the final 10,000,000 steps. Table 7.4 lists the key values recorded during this run, Fig. 7.4 plots its full curve, and Fig. 7.5 plots both runs on a shared axis for direct comparison.

Table 7.4 Key values recorded during the HB_16 training run

Training step	Milestone	Mean reward
150,000	First logged summary	-1.70
25,442,592	First session stopped (user request)	88.375 (highest so far)
45,250,000	Best logged summary of the run	97.728 (best ever)
45,749,952	Checkpoint	95.958
48,499,936	Checkpoint	95.444
49,249,920	Checkpoint	96.303
49,749,984	Checkpoint	97.566
50,000,000	Final logged summary (std of reward 5.978)	96.58 (final)
50,002,848	Final exported checkpoint	96.345

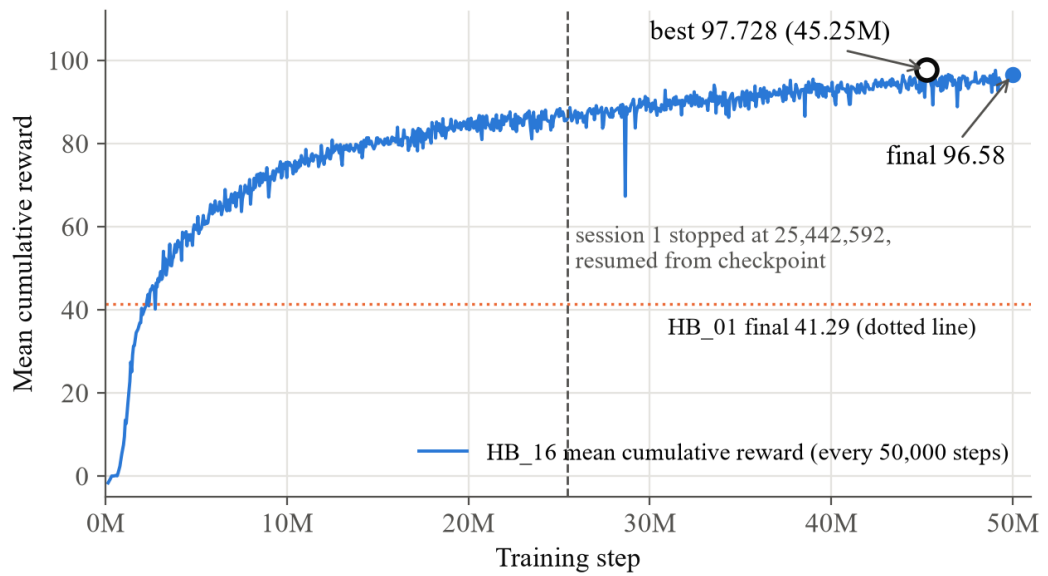


Fig. 7.4 HB_16 mean cumulative reward (0 to 50,000,000 steps)

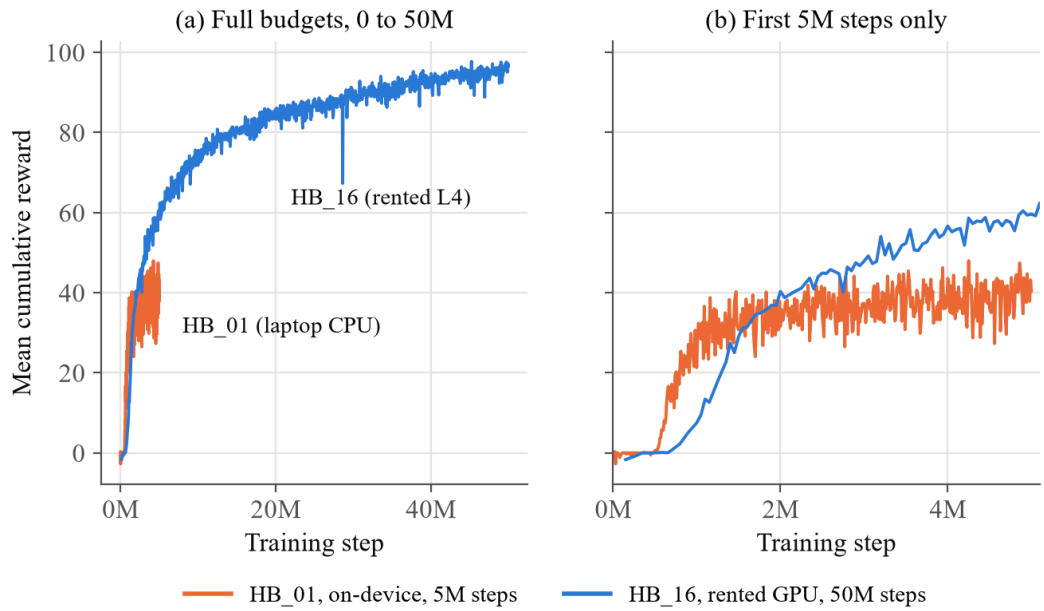


Fig. 7.5 Mean cumulative reward of HB_01 and HB_16 over (a) the full budgets and (b) the first 5M steps

Part (b) of Fig. 7.5 contains a result that is easy to miss. Over the first five million steps HB_16 is slower to start than HB_01: its extra inputs and larger batches mean it needs more experience before its first rise. By about 2.5 million steps it has overtaken HB_01, and at five million steps, the whole of HB_01's budget, it is already near 60 against HB_01's 41. The improvement is therefore not only a matter of the larger budget; the velocity-aware policy learns a better controller at the same step count.

The earlier version of this report did not compare HB_16 against the ceiling, because that ceiling had been derived from a roughly 20 flower layout and the flower count of the spawned arenas had not been confirmed. HB_16's 112 training arenas were instead generated at runtime by a dedicated spawner script rather than built in the Editor, and the exact flower count per arena in that generated layout has not yet been independently confirmed against the single island figure. Both questions are now settled: every spawned arena is a clone of the same prefab with 360 flowers (the 28-island grid holds 10,080), and the correct, step-based ceiling of Section 7.5.2 applies equally to both runs.

The resulting policy, copied into the Unity project as RentedGPU_Final.onnx, was wired into both the Training scene's single Hummingbird and the Flower Island minigame's Opponent bird, replacing the HB_01 model used in the verification described

in Section 7.4. Both placements were confirmed working in Play mode with Behavior Type set to Inference Only, real flight and feeding behaviour, and a clean Console. No further training is planned; this extended run closes the machine learning training scope of the project, beyond the base course requirement already satisfied by HB_01.

7.5.4 Training Diagnostics

Fig. 7.6 shows two further diagnostics from the logs. The value estimate (a) is the trainer's own prediction of the reward to come. HB_16's rises smoothly to about 9.7 and HB_01's to about 4.0, mirroring the difference in actual reward, which shows the critic learnt a consistent picture of the task in both runs. The policy loss (b) stays small and bounded throughout both runs, as expected for PPO's clipped objective; there is no sign of the sudden spikes that would indicate an unstable update.

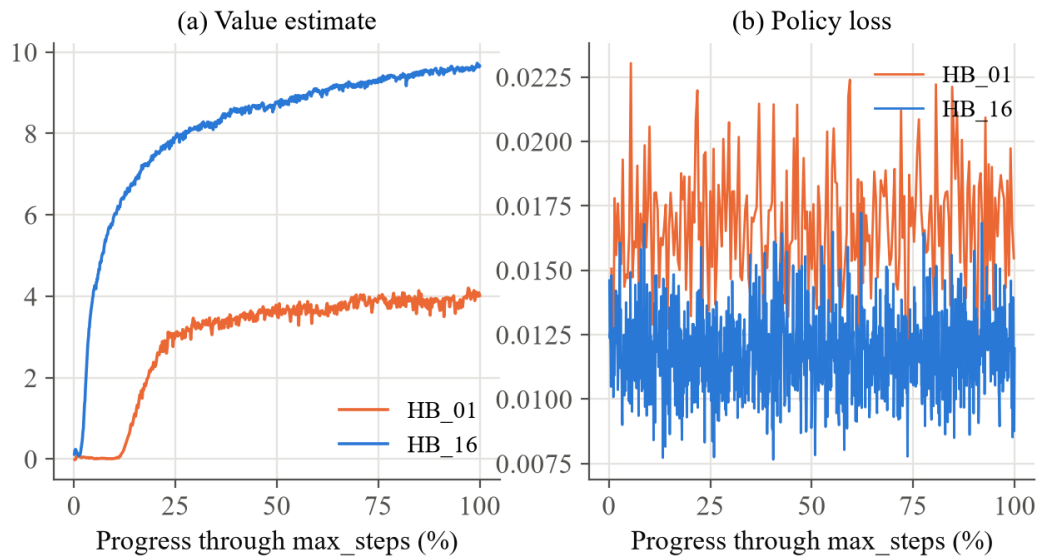


Fig. 7.6 (a) Value estimate and (b) policy loss of both runs

7.6 Performance Metrics

Table 7.5 collects the performance metrics of the system in one place. They cover the quality of the learned policy, the efficiency of training, and the cost of building and deploying the result.

Table 7.5 Performance metrics of the system

Metric	HB_01 (laptop)	HB_16 (rented L4)
Final mean cumulative reward	41.29	96.58

Metric	HB_01 (laptop)	HB_16 (rented L4)
Best logged mean reward	47.94 (noisy single summary)	97.728
Standard deviation of reward at the end	not logged	5.978
Steps trained	5,000,024	50,002,848
Active compute time	31.44 hours	5.11 hours
Average throughput	about 44 steps/s	about 2,717 steps/s
Policy parameters	79,114	84,234
Final policy entropy	1.32	0.73
Share of wall clock in Unity (env_step)	95.5 percent	not separately logged
Model file size	320,997 bytes	345,468 bytes
Android APK size / build time	54.4 MB / 19 minutes (both models)	

The throughput figures explain where training time goes. Fig. 7.7 breaks down the final 16,742-second session of HB_01 using its timers.json file: 15,994 seconds (95.5 percent) were spent inside the Unity environment step, running physics and sensors, and only 587 seconds (3.5 percent) in the PPO update. A GPU can only accelerate that 3.5 percent, which is why the CPU-versus-GPU comparison in Section 7.7 shows so little difference.

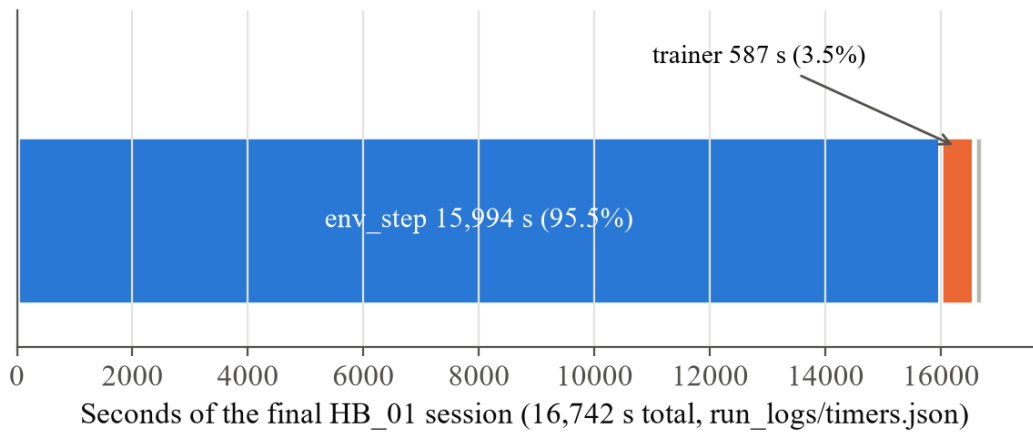


Fig. 7.7 Wall-clock breakdown of HB_01's final training session

7.7 Comparative Analysis

7.7.1 On-Device versus Rented Training

Table 7.6 Time and throughput comparison between on device training (HB_01) and rented cloud GPU training (HB_16)

Metric	HB_01 (on device, CPU only)	HB_16 (rented Jarvis Labs L4)
Steps trained	5,000,024	50,000,000 (10 times more)
Active compute time	31.44 hours (summed across 6 stop and resume sessions)	5.11 hours (summed across 2 sessions)
Real calendar span	38.18 hours (spread over about 1.5 days around laptop availability)	About 12.7 hours in a single day (including a 7.5 hour gap between sessions)
Average throughput	About 44 steps per second	About 2,717 steps per second

Table 7.6 compares the two runs as engineering exercises. HB_16 trained ten times as many steps in about a sixth of the active compute time, because its throughput was about 62 times higher. Its calendar span was also far shorter, even with a 7.5-hour gap between sessions, because it did not have to be fitted around the laptop's other uses.

7.7.2 Comparison Against a Public Reimplementation

Since neither the base course nor the ML-Agents documentation publish exact benchmark numbers for the Hummingbirds environment, the closest comparison point identified was a public GitHub reimplementation of the same tutorial, [gunjanmimo/Hummingbirds-Unity-ML](#) [5], which publishes real TensorBoard charts using the same reward tags (Environment/Cumulative Reward, Policy/Extrinsic Reward) as this project.

Table 7.7 places the two results side by side.

Table 7.7 Final result comparison against a public tutorial reimplementation

	This project (HB_01)	gunjanmimo/Hummingbirds-Unity-ML [5]
Training steps	5,000,024	~1,600,000 (still climbing)
Final Mean Reward	41.29	~33–34 (at 1.6M steps)
Hardware	CPU-only (integrated graphics)	Not specified

This project's final reward is meaningfully higher than the comparison reimplementation's reported value, though this is not presented as a fully controlled comparison; the two runs were trained for different step budgets (this project's run continued roughly three times longer), and hardware and minor configuration differences between the two implementations were not independently verified. The comparison is included as the only real, publicly available data point found for this specific tutorial's environment, rather than as a rigorous benchmark claim.

With HB_16 added, the comparison is no longer close: at 96.58 after 50 million steps the rented-GPU model is roughly three times the reimplementation's reported value, although the same caveat about uncontrolled differences applies.

7.7.3 Throughput Across Process and Arena Layouts

Before the long run was started, the sweep mode of the training script measured how fast different layouts trained on the L4 instance. Unity's main loop is single-threaded within each process, so the same number of arenas can be split across processes in different ways, and the split matters. Table 7.8 and Fig. 7.8 show the six ways of arranging 28 arenas.

Table 7.8 Measured throughput of process and island layouts on the L4 instance

Islands per process x processes	Total arenas	Steps per second
7 x 4	28	2,665
14 x 2	28	2,247
4 x 7	28	2,231
28 x 1	28	2,014
2 x 14	28	1,472
1 x 28	28	873
7 x 4 (second sweep)	28	2,642
7 x 8	56	2,880
14 x 4	56	2,892
14 x 8	112	3,106
28 x 4 (chosen)	112	3,237

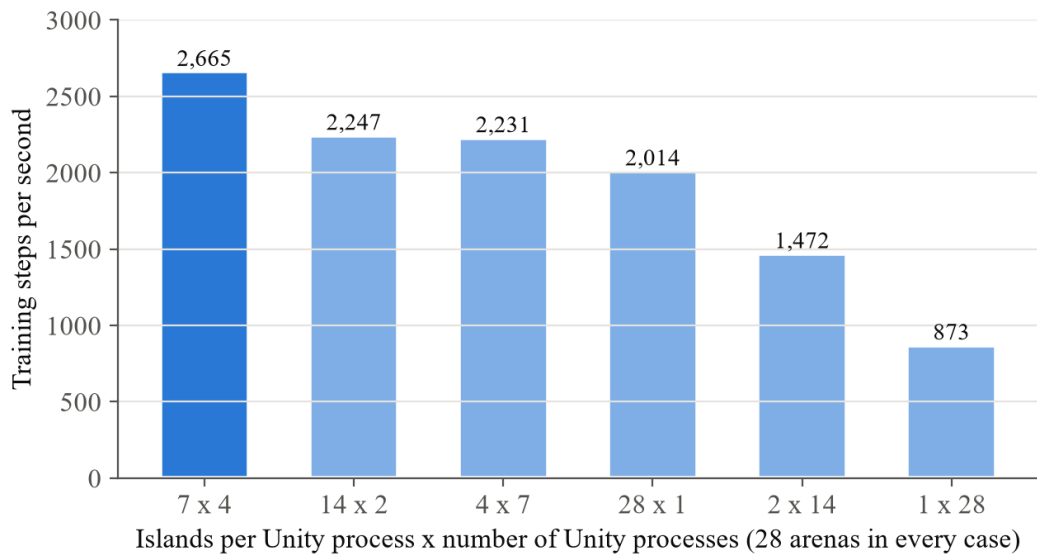


Fig. 7.8 Throughput of six ways of arranging the same 28 arenas

The spread is large. On identical hardware with an identical number of arenas, the best split (7 islands in each of 4 processes, 2,665 steps per second) was three times as fast as the worst (1 island in each of 28 processes, 873). Neither extreme wins: one process with 28 islands is limited by its single main thread, and 28 processes of one island each pay 28 times the per-process overhead. Increasing the total arena count, shown in Fig. 7.9, gave sharply diminishing returns: four times the arenas (28 to 112) bought only 22 percent more throughput, which indicates the 28 vCPUs were saturated. The chosen layout, 28 islands in each of 4 processes, was the fastest measured.

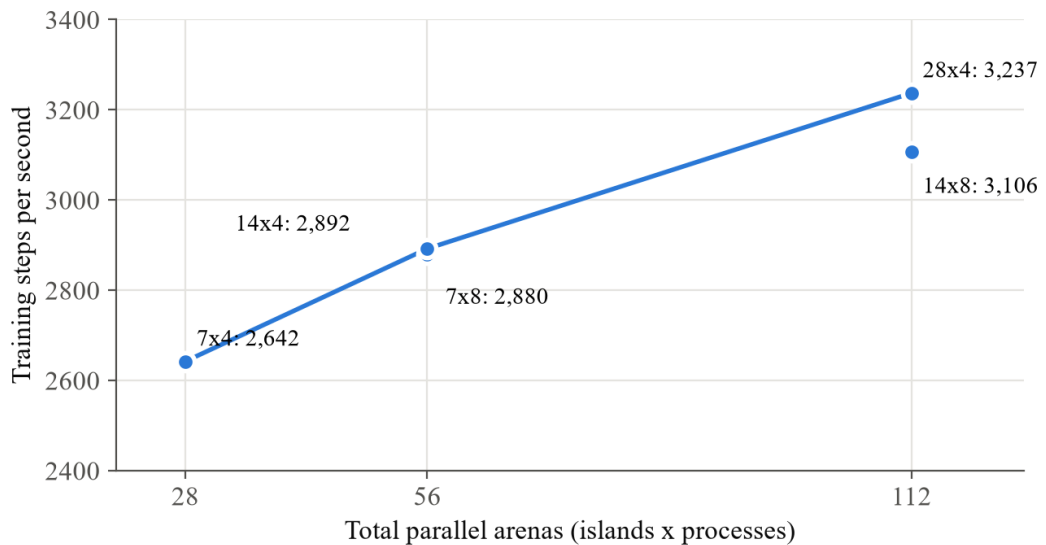


Fig. 7.9 Throughput against total number of parallel arenas

7.7.4 Headless Rendering and CPU versus GPU

Fig. 7.10 collects the three other comparisons measured on the server. Part (a) is the overall gain in average throughput between the two runs. Part (b) is the effect of `-batchmode` on the headless player running under Xvfb: 64 steps per second without it and 832 with it, the 13 times speed-up described in Section 6.6.7. Part (c) is the comparison between CPU and CUDA training on the same instance.

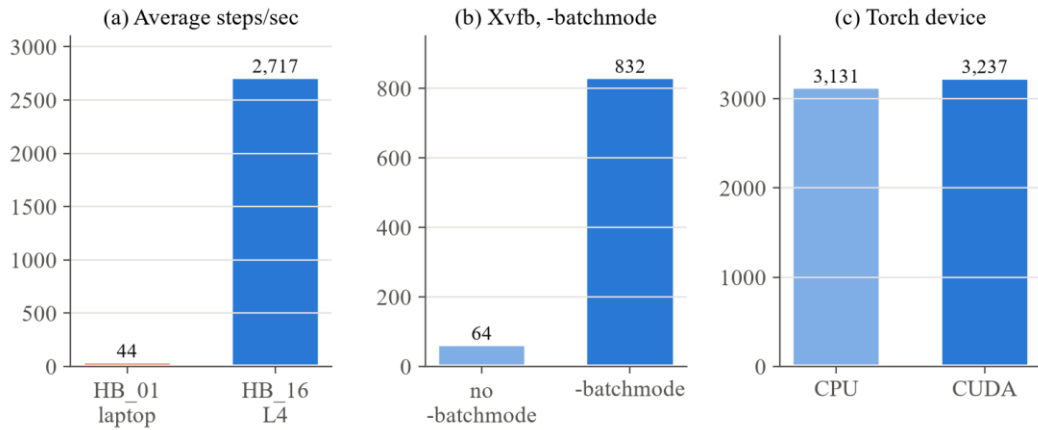


Fig. 7.10 (a) Average throughput of both runs, (b) effect of `-batchmode`, (c) CPU vs CUDA training

The tenfold increase in achievable step budget was made practical primarily by running many simulation arenas in parallel rather than by the GPU itself. A direct comparison of CPU only and CUDA enabled training on the same rented instance measured a difference of only about 3.4 percent in steps per second, closely matching the 3.5 percent of HB_01's wall-clock time that its own timers attribute to the trainer. This confirms that, for an agent of this size using vector and ray cast observations rather than camera based vision, parallel CPU throughput, not raw GPU compute, is the limiting factor on training speed; the practical implication for any future retraining is to rent for vCPU count rather than for GPU tier.

CHAPTER 8

RESULTS AND CONCLUSION

8.1 Results

The project met every objective set in Section 1.3. The outcome of each development sprint, recorded at the time, is summarised below, followed by the results of the extended work and the list of deliverables. Table 6.3 in Section 6.6.1 summarises the same outcomes in one table.

- All Sprint I objectives were met: the project opened cleanly with correct lighting, the ML-Agents package was installed at a safe, working version (4.1.0, not the broken 3.0.0 or deprecated 2.0.1 default), and the Console reported 0 errors and 0 warnings after each step.
- Both scripts compiled and ran cleanly. Because the `flower_plant` tag did not yet exist in the project at the time `FlowerArea.cs` was first tested, an early test run produced 999+ console errors from Unity's `CompareTag()` throwing on an undefined tag; this is documented fully in the retrospective below, since the same underlying trap recurred twice more later in the project.
- Training completed successfully to the configured 5,000,000-step budget (reaching 5,000,024 steps at the final checkpoint), reaching a final Mean Reward of 41.29, confirmed directly from the trainer's own `training_status.json` output. Full results and analysis are presented in Chapter 7. A further training run beyond the base course's own scope subsequently extended this using rented cloud GPU infrastructure; see Section 7.5.3.
- The Flower Island scene was confirmed fully playable end-to-end via live testing: the Start button responds correctly, the countdown plays, both nectar progress bars update in real time, and the Console remains clean (0 errors, 0 warnings) from scene load through a complete game session.

The extended work produced three further results. The `HB_16` run reached a final mean reward of 96.58 and a best of 97.728 after 50,000,000 steps, 2.35 times the `HB_01` baseline, with the reward function, episode length and physics unchanged. The throughput study established that 28 islands in each of 4 processes was the fastest

measured layout at 3,237 steps per second, that the split of arenas across processes changes throughput by a factor of three, and that a GPU contributes only 3.4 percent on this workload. The Android build packaged both models into a 54.4 MB application that runs offline on the Galaxy S24 Ultra, with live model switching, a debug view of the agent's perception, flight trails and a split-screen race.

Taken together, the ten deliverables listed below span the full scope of this project: the migrated codebase and trained model at its technical core, the analysis and requirements documentation and UML diagrams that frame it, and the process artefacts (sprint records, troubleshooting tooling, and the report and presentation themselves) that make the work reproducible and reviewable. Read together with the implementation account in Chapter 6, these deliverables constitute a complete, independently verifiable record of both what was built and how it was built.

- A fully migrated Unity ML-Agents project running on Unity 6000.4.7f1 and ML-Agents Toolkit 4.1.0, with every scripting API discrepancy against the 2019.3-era base tutorial identified and resolved.
- Three complete, independently-verified C# scripts (Flower.cs, FlowerArea.cs, HummingbirdAgent.cs) implementing the full agent-environment interaction loop, reproduced in Appendix 3.
- A trained PPO policy reaching a final Mean Reward of 41.29 over 5,000,024 training steps entirely on CPU only hardware (Table 7.3), subsequently extended by a supplementary rented cloud GPU training run reaching a final Mean Reward of 96.58 over 50,000,000 steps (Section 7.5.3), with the complete checkpoint history for both runs documented in the corresponding tables.
- A verified, ONNX-exported inference model, confirmed working with no live Python connection in two independent deployment contexts.
- A fully playable, custom-built competitive minigame deploying the trained model against a human player, including original debug tooling (CameraFollowBird.cs, NectarCounterUI.cs) not present in the base course.
- A documented engineering record of real, root-caused fixes for version conflicts, deprecated APIs, and platform-specific bugs, intended as a practical reference for adapting similar legacy tutorials to current toolchains.

- A structured, sprint-based project management record (Section 6.6.1), with each of the five sprints documenting its own objectives, functional and architecture documentation, outcome, and retrospective, usable as a template for organising future ML-Agents or similar applied AI projects.
- A formal System-Level and Software-Level Requirements specification (Sections 3.1, 3.2 and 4.1), covering hardware, software, functional, and non-functional requirements, prepared to support the project's academic review process.
- A reusable Win32 AttachConsole/GenerateConsoleCtrlEvent PowerShell script for cleanly stopping mlagents-learn mid-run with a checkpoint saved at the exact current step, applicable beyond this specific project to any future mlagents-learn training session on Windows.
- A supporting presentation deck and this written project report, both built directly from verified project facts, figures, and code rather than generic templated content, prepared for guide review and academic submission.
- The rented-GPU training infrastructure: IslandSpawner.cs, the command-line Linux build, setup_server.sh with its pinned environment, and train_16.sh with its smoke, sweep, run and status modes.
- An Android application of 54.4 MB built by a command-line script, carrying both trained models and the presentation toolkit, verified on the target phone.

8.2 Discussion and Analysis

The central result of the project is the difference between 41.29 and 96.58, and the most useful thing the project learned is where that difference came from. It did not come from a better algorithm, a bigger network or a changed reward: the algorithm, network size and reward function are identical in both runs. It came from two diagnoses made by reading HB_01's own logs. The run had not converged, it had run out of budget with its learning rate annealed to zero; and the agent's observations did not contain the one quantity, velocity, that a controller of a body with momentum needs. The fix for the first was compute, bought cheaply by running 112 arenas in parallel. The fix for the second was a single Inspector field that let the network see three consecutive frames.

Fig. 7.5 (b) suggests that the second fix mattered as much as the first: at HB_01's own budget of five million steps, HB_16 was already scoring about 60 against HB_01's 41. The two changes cannot be fully separated, because HB_16 also doubled its batch size and enabled input normalisation, a limitation discussed in Section 8.3. The direction of the evidence is nevertheless clear, and it is consistent with the physical argument: a policy that can estimate velocity can brake before a flower instead of overshooting it, and this is exactly what the flight trails show on the phone, where the HB_01 bird traces loops around each flower and the HB_16 bird clean arcs.

The single-island training configuration used to produce the results in this chapter was not the original plan; it was arrived at empirically after a genuine hardware failure. An initial attempt at the base course's default multi-island parallel training configuration (8 islands) caused a full system freeze: free RAM dropped below 1GB and the Unity Editor became fully unresponsive for an extended period. This was scaled down first to 4 islands, which remained tight, and finally to the single-island configuration used throughout Chapters 6 and 7, which has been the stable, reproducible baseline for every result reported here. This is recorded not merely as a limitation but as a substantive finding in its own right: on an 8GB-RAM, integrated-graphics laptop, single-island training was not just the safe choice but the only configuration that trained to completion without crashing, and the final Mean Reward of 41.29 demonstrates that a meaningful policy can still be learned under this constraint.

This also directly explains why the theoretical-ceiling analysis in Section 7.5.2 attributes part of the remaining reward gap to reduced training variety per wall-clock hour: a multi-island setup would expose the policy to more parallel experience per unit of time, at the cost of the RAM headroom this hardware simply does not have. Any future re-run with more islands (Section 8.6) would need to be carried out on hardware with a larger RAM budget than was available for this project. Section 7.5.3 documents a subsequent training run that used rented cloud GPU infrastructure specifically to work around this local hardware ceiling.

The throughput results change how that constraint should be understood. On this workload a GPU is almost irrelevant, because 95.5 percent of the time is spent in Unity's physics on the CPU, and the right hardware to rent is the one with the most CPU cores

rather than the most powerful GPU. Among the processes, the layout matters as much as the core count. These findings are specific to a small network with vector and ray observations; an agent with camera observations would shift work onto the GPU and could change the conclusion.

Taken together, the results in this chapter establish three things directly relevant to this project's stated research objectives: the trained PPO policy converged to a stable, positive reward (41.29) that is well within an interpretable, reward-derived theoretical ceiling rather than an unexplained plateau; the resulting policy is meaningfully competitive against the only comparable public benchmark located for this specific environment; and the trained model was independently verified to function correctly under two separate inference deployments with no live training connection, satisfying the practical deployment objective set out in Chapter 1. The hardware constraints under which these results were obtained are documented explicitly rather than abstracted away, since they materially shaped both the training configuration used and the ceiling against which the final reward should be judged. The supplementary training run described in Section 7.5.3, using rented cloud GPU infrastructure, subsequently extended these results well beyond the local hardware ceiling documented above, reaching a final Mean Reward of 96.58 without altering the reward function, physics, or episode length used for this chapter's primary analysis.

8.3 Limitations

- Each configuration was trained once. Reinforcement learning results vary between random seeds, and without repeated runs the size of that variation for this task is unknown.
- HB_16 changed several things at once: observation stacking, input normalisation, batch and buffer size, and the step budget. The report attributes the improvement mainly to stacking and budget on physical and log-based grounds, but no ablation run was done to measure each change separately.
- Performance is measured as the trainer's mean episode reward during training, not by a separate evaluation of the final policy with exploration switched off. The flowers foraged per episode are estimated from reward rather than counted.

- All training and evaluation used one island layout. How well either policy transfers to a different layout was not tested.
- There is no automated test suite. Testing relied on in-Editor observation, scripted checks of files and scenes, and on-device confirmation.
- Frame rate on the phone could not be measured with Android's standard tools, because Unity renders through its own surface; smoothness was confirmed by observation only.
- The comparison with the public reimplementations is uncontrolled, since the two used different budgets and possibly different settings.
- Long Unity Editor sessions were found to degrade inference behaviour, so any in-Editor observation of a model has to be confirmed in a fresh Editor session; built players are not affected.

8.4 Lessons Learnt

The retrospective notes written at the end of each sprint are reproduced below, followed by the lessons from the extended work.

8.4.1 Sprint I, Project Setup and Foundation

- Real bug avoided proactively: ML-Agents 3.0.0 is confirmed broken on Unity 6000.x; version 4.1.0 was installed explicitly by technical name instead of accepting the Package Manager default.
- Unity 6 UI drift: the Default Contact Offset setting had moved tabs since the tutorial was recorded; the correct current location was confirmed against official documentation rather than assumed.

8.4.2 Sprint II, Core Environment Scripting

- Genuine bug, corrected root cause: Unity's `Component.CompareTag()` does not silently return false for an undefined tag; it throws a `UnityException`. This was hit directly when `FlowerArea.FindChildFlowers()` recursed through the scene hierarchy checking for a `flower_plant` tag that had not yet been defined in `ProjectSettings/TagManager.asset`, producing a large number of console errors. The fix was to define the tag early, before the code that depends on it runs. The

same pattern recurred for the nectar and boundary tags later in Sprint III, and was applied proactively both times once the trap was understood.

- A "Moving file failed: Access is denied" error occurred while saving FlowerBud.prefab after attaching Flower.cs; a full PC restart cleared the underlying file lock.

8.4.3 Sprint III, Agent Development and Scene Wiring

- Same tag-definition trap as Sprint II, hit twice more: the nectar tag (used by CompareTag("nectar") in TriggerEnterOrStay) and the boundary tag (used in OnCollisionEnter) both had to be proactively defined in TagManager.asset before the code using them could safely run, having learned from the Sprint II incident.
- A real InvalidOperationException on Input.GetKey() inside Heuristic() was traced to Unity 6's default Active Input Handling being set to the new Input System only; resolved via Edit → Project Settings → Player → Other Settings → Active Input Handling → Both, followed by an Editor restart.
- A second "Moving file failed: Access is denied" error, while saving FloatingIsland.prefab, was this time traced conclusively (via Resource Monitor's Associated Handles search) to a leftover file handle held by the Claude desktop application from earlier direct file reads against a live, open Unity project, resolved by closing and reopening the app, and avoided going forward by not reading project files directly while the Editor holds them open.

8.4.4 Sprint IV, Training and Inference

- Root-caused, not just worked around: mlagents-learn initially crashed with ModuleNotFoundError: No module named 'pkg_resources', caused by setuptools 82+ removing that module while mlagents 1.1.0 still imports it the old way; fixed by pinning setuptools<81. A related StrictVersion AttributeError recurred specifically on worker restarts even after this fix; the true root cause was traced directly into mlagents_envs/environment.py's _check_communication_compatibility(), which was patched to parse version strings as plain tuples instead of relying on Python's deprecated distutils.version.StrictVersion.

- A `ModuleNotFoundError`: No module named 'onnxscript' occurred during ONNX checkpoint export under PyTorch 2.13's default exporter; fixed by patching the export call to pass `dynamo=False`, forcing the legacy ONNX exporter path.
- The course's `trainer_config.yaml` used unsigned scientific notation (`5.0e6`) for `max_steps`; PyYAML's float resolver requires a signed exponent (`5.0e+6`) or silently parses the value as a string rather than a number. This was avoided by writing `max_steps` as a plain integer instead.
- Stopping Play in the Unity Editor does not gracefully stop `mlagents-learn`; a dedicated PowerShell script using the Win32 `AttachConsole/GenerateConsoleCtrlEvent` API was built to send a genuine `Ctrl+C` signal to the trainer process, allowing training to be stopped cleanly with a checkpoint saved at the exact current step.

8.4.5 Sprint V, Minigame and Course Wrap-up

This sprint involved the most significant debugging effort of the project. The Minigame's Start button initially appeared completely unresponsive across multiple test sessions. The root cause was a duplicate `EventSystem` in the scene: `UI.prefab` already ships with its own nested `EventSystem`, and a second one had inadvertently been added at the scene root while building the scene through direct file editing rather than the normal Editor "create scene → drag in prefab" workflow. With two active `EventSystems` present, Unity's own UI click routing became unpredictable: clicks would silently route through the wrong `EventSystem`, so `GameManager`'s button-click subscription never fired. A related issue was also found and fixed during the same investigation: `GameManager`'s `uiController` and `opponent` field references, originally set via direct file edits, resolved to stale objects at runtime despite being byte-for-byte correct in the saved scene file, and was resolved by re-assigning those references via the Editor's own drag-and-drop, which regenerates a reliable object reference tied to the live prefab instance in a way that direct file edits cannot fully replicate.

The general lesson drawn from this sprint (applied to the write-up throughout this report) is that whenever a scene or prefab is wired up via direct file editing rather than the Unity Editor's own UI, a deliberate sanity check for pre-existing duplicate components is warranted before adding anything new, and any reference that begins behaving

unexpectedly is best re-assigned through the Editor directly rather than continuing to hand-edit the underlying YAML.

8.4.6 Development Environment

Because both the Unity Editor and ML-Agents training ran on integrated graphics throughout this project (the machine's discrete NVIDIA MX130 GPU is permanently disabled at the BIOS level after previously causing system crashes), a small set of Editor-side settings were deliberately tuned to keep Scene view interaction usable during development. Edit → Preferences → General → Interaction Mode was set to No Throttling, which removes an idle/wake stutter cycle that is otherwise strongly associated with the default "Monitor Refresh Rate" interaction mode on integrated graphics; confirmed against Unity's own source-level tooltips rather than assumed. Occlusion Culling was enabled and Easing was disabled in the Scene view's own Camera settings menu for additional headroom, and a habit of pressing Escape to deselect GameObjects (particularly the Hummingbird, which carries multiple colliders) before orbiting the Scene view was adopted, since Unity recalculates gizmos for the currently-selected object every frame regardless of whether it is moving.

With 8GB of total system RAM, running the Unity Editor, a browser, and VS Code simultaneously was observed to push memory usage above 90% during longer sessions, consistent with reports from other developers working under similar constraints; the practical mitigation adopted was simply closing background applications before starting a training run. These settings did not eliminate every trace of Editor lag on integrated graphics (some residual lag is the genuine ceiling of the available hardware rather than a fixable setting) but together they kept iteration speed workable throughout Sprints I through V without requiring any hardware upgrade.

A separate, recurring "Moving file failed: Access is denied" error was also encountered twice while saving prefab files during Sprints II and III. The second occurrence was root-caused via Windows Resource Monitor's Associated Handles search to a locked file handle rather than antivirus interference; nonetheless, as a general precaution against real-time scanning contention on a frequently-written project folder, the project directory was added as a permanent "All components" exclusion in the machine's antivirus software, which reduced this class of friction going forward.

Taken together, the Editor-side tuning, the antivirus exclusion, and the discipline of restarting the Claude desktop application whenever a file-lock error recurred were not cosmetic conveniences; they were direct preconditions for the Sprint IV training run evaluated in Chapter 7. Without a usable Scene view and a project directory free of real-time scanning contention, the many hours of iterative scripting, scene wiring, and hyperparameter adjustment documented across Sprints I through III would have been considerably slower, and the eventual single-island training configuration would have been harder to diagnose and commit to with confidence.

A general troubleshooting hierarchy emerged organically across the sprints whenever a file-level or environment-level obstacle appeared: first check the Unity Console for an explicit error message and act on it directly; if a save or move operation failed with an access-denied error, check Resource Monitor's Associated Handles search for the actual locking process before assuming antivirus interference; and if an external tool (the Claude desktop application, in this project's case) was found to be holding a handle on a live project file, close and reopen it rather than restarting the whole machine, reserving a full restart for cases where the specific handle could not be identified. This hierarchy, refined through direct experience rather than assumed upfront, is recorded here as a practical takeaway independent of the specific bugs that prompted it.

With the development environment stabilised and the full twelve-part HummingbirdAgent.cs script, Flower.cs, and FlowerArea.cs verified compiling and running cleanly across Sprints I through III, the project was in a position to attempt a full, uninterrupted training run. Chapter 7 presents the quantitative results of that run in detail, including the final Mean Reward, its interpretation against a theoretical ceiling, and a comparison against the only publicly available benchmark identified for this specific tutorial environment.

8.4.7 Extended Work on Rented Hardware and Android

- Measure before renting. HB_01's timers.json showed that 95.5 percent of training time was Unity physics, which predicted, correctly, that a GPU would add almost nothing and that the rental should be chosen by vCPU count.
- Decide whether a run converged from its checkpoint history, not its final number. A run that ends on budget with a linear schedule always looks flat at the end.

- Check the observation space against the physics being controlled. A body with momentum needs a velocity signal.
- When a pinned package stack demands a new dependency, change the package that makes the demand rather than adding the dependency; installing onnxscript broke three other pins, while an older PyTorch needed nothing.
- A headless Unity 6 URP player needs a 24-bit virtual display and -batchmode, never --no-graphics.
- A trained model and its observation stacking are a contract. Set them together, and only while the agent is disabled, or the agent silently receives the wrong input with no error logged.
- Unity interleaves Awake and OnEnable when activating a new hierarchy, so shared collections belong in field initialisers, and code that depends on other objects' Awake should wait a frame.
- User interfaces built in code need one shared scaling configuration and edge anchoring; problems that are invisible in the Editor appear on a phone's wider screen.
- Do not trust exit codes or summary numbers from build tools: batch-mode Unity needed an explicit SUCCESS line, and the Android build summary over-reported the APK size by more than twenty times.
- Restart the Unity Editor before believing that a model has got worse; a long session degraded inference while the model file was byte-identical to the trained checkpoint.

8.4.8 Personal Learning Outcomes

Beyond the specific technical deliverables, this project reinforced several broader skills directly relevant to the MCA Generative AI specialization and to applied AI/ML work generally. Working through the full reinforcement learning pipeline end-to-end (designing observations and a reward function, configuring a trainer, and interpreting a training curve rather than only reading about these concepts) built a concrete, defensible understanding of PPO that would be difficult to acquire from theory alone. Equally significant was the experience of maintaining and migrating an existing codebase against a moving target: adapting a several-years-old tutorial to a current engine and toolkit

version surfaced real API deprecations, dependency conflicts, and platform-specific bugs that had to be diagnosed methodically rather than guessed at, mirroring the kind of maintenance work common in production ML systems.

The project also reinforced the value of root-causing problems rather than accepting the first workaround that resolves a symptom; several issues documented in this section (the `StrictVersion` bug, the `duplicate-EventSystem` bug) initially appeared fixed after a partial fix, only to resurface later under slightly different conditions, and were only fully resolved once traced to their actual source. This debugging discipline, together with the hands-on RL and Unity ML-Agents experience itself, is considered as valuable an outcome of this project as the final trained model.

8.5 Conclusion

This project successfully implemented, trained, and deployed a complete reinforcement learning pipeline for an autonomous hummingbird agent using the Unity ML-Agents Toolkit, executed on a current engine and toolkit release (Unity 6000.4.7f1, ML-Agents 4.1.0) rather than the several-years-older versions the base tutorial was originally authored against. Every C# script in the environment (`Flower.cs`, `FlowerArea.cs`, and the 430-line `HummingbirdAgent.cs`) was written and independently verified rather than copied wholesale, requiring genuine API migration work (the `float[]`-to-`ActionBuffers` transition, and the `Rigidbody.velocity`-to-`linearVelocity` rename) at multiple points.

The trained PPO policy reached a final Mean Reward of 41.29 after 5,000,024 training steps, entirely on CPU-only, memory-constrained consumer hardware, and was verified to perform correctly under pure inference in two independent deployment contexts: the original training environment and a separately-built, competitive two-player minigame. Beyond the base tutorial's scope, two original supporting tools were also built and integrated: a lightweight third-person chase camera (`CameraFollowBird.cs`) and a runtime-built on-screen nectar counter (`NectarCounterUI.cs`), both used to aid direct visual verification of agent behaviour during development. This on device result was subsequently extended by a supplementary cloud GPU training run (Section 7.5.3), reaching a final Mean Reward of 96.58 over 50,000,000 steps and confirming the same pipeline's scalability well beyond local hardware limits; the on device figure above

remains the course scoped deliverable completed entirely on the hardware originally available for this project.

Equally significant to the numerical result is the debugging and engineering record documented in Chapters 6 and 8: several genuine, non-trivial issues were diagnosed to their true root cause rather than merely worked around, including a `setuptools/pkg_resources` incompatibility, a `StrictVersion` bug inside ML-Agents' own communication-compatibility check, an ONNX export failure under a newer PyTorch default exporter, and a duplicate-EventSystem bug that silently broke UI click routing in the minigame scene. This body of work demonstrates not only an understanding of reinforcement learning concepts in the abstract, but the practical, applied engineering skill of adapting a legacy tutorial codebase to a current toolchain, a realistic and directly relevant skill for applied AI/ML work.

The extended work carried the project further than the course. By reading its own training logs rather than accepting the first result, the project found why the baseline stopped where it did, fixed the cause without changing how reward is earned, and more than doubled the result on rented hardware for a few hundred rupees. It then put both models on a phone in a form that lets anyone see the difference for themselves. Together these show the whole life of an applied reinforcement learning system, from environment design to a trained model running on a device in someone's hand.

With the base "ML-Agents: Hummingbirds" course completed end-to-end (all ten units plus the optional bonus Live Session reviewed for conceptual takeaways) this project has satisfied its original scope of building genuine, verified, hands-on depth in reinforcement learning before extending into original territory. The instructor's own closing remark in the Course Wrap Up unit, that an original ML-Agents project typically takes one week to over a month of trial and error even for an experienced practitioner, is a useful calibration point for the future-enhancement items listed in Section 8.6: they are treated here as a realistic next phase of work rather than a short addendum, to be undertaken with the same debugging discipline and root-cause-first approach documented throughout this report.

8.6 Future Enhancements

Several extensions were identified during the project and deliberately parked until the base tutorial was fully complete, consistent with a decision made early in the project to follow the base course exactly before adding any original contribution. These are recorded here as concrete future work, together with items that arose from the extended work.

- GenAI-directed garden layouts: using a generative AI API (e.g. the Gemini API) to procedurally generate garden/flower layouts, and testing whether the trained agent adapts effectively to AI-generated, rather than hand-placed, environments, of particular relevance to the MCA Generative AI specialization.
- Curriculum learning study: training on an easy-to-hard progression of flower layouts versus training on the hard configuration only, and comparing convergence speed and final performance, directly applying the curriculum learning framework of Bengio et al. [1] surveyed in Chapter 2.
- Multi-agent competitive training: extending beyond the current single-agent-trained-then-deployed-competitively setup toward genuine multi-agent self-play training, where multiple hummingbirds compete for scarce flowers during training itself.
- Generalization testing: training on one flower layout and evaluating performance on unseen layouts, to directly measure the policy's generalization gap.
- Exact per-episode metrics: adding a StatsRecorder-based flower-count tracker to HummingbirdAgent.cs, to replace the current reward-derived flower-count estimate (Section 7.4) with an exact, directly logged count.
- Control scheme redesign: the current Heuristic control scheme (WASD + E/C + arrow keys, all relative to the bird's own facing direction) was noted during development as not feeling like a natural fit for a bird specifically; a future iteration could redesign the control scheme itself, potentially alongside adding mouse-look support.
- Extended before/after training comparison footage: recording gameplay footage at each retained checkpoint (500K, 1M, 2M, 3M, 4M, and the final checkpoint),

plus a fresh zero-step run as a genuine random-policy baseline, to visually demonstrate the training progression.

- Repeated runs and ablations: training each configuration with several random seeds, and training variants of HB_16 that change one factor at a time, so that the contribution of stacking, normalisation, batch size and budget can be measured separately.
- A separate evaluation mode: running the final policies deterministically over a fixed set of episodes to report evaluation reward and exact flowers per episode, rather than training-time means.
- Faster Android builds: stripping the unused Inference Engine GPU compute shaders, which dominated the first build's time although the application only uses the CPU backend.

If this work is continued, the items above are best taken in a particular order. Repeated runs and an ablation of the HB_16 changes come first, because every later comparison depends on knowing how much of the 41.29 to 96.58 improvement each change produced and how much a result varies between seeds. A deterministic evaluation mode and an exact flower counter come next, since they make every later experiment easier to measure. Curriculum learning and generalisation to new layouts, including generated layouts, are the natural research steps after that, and multi-agent self-play is the largest change, because it alters the task itself. The infrastructure already built for this project, a runtime spawner, a resumable training script and a measured cost of about Rs 43 per hour, means that each of these experiments is now a matter of configuration and rental time rather than new engineering.

CHAPTER 9

APPENDICES

Appendix 1 Plagiarism Certificate

The plagiarism (similarity) check report and the signed Own Work Declaration for this report are held by the Department of Computer Applications and are not reproduced in this public copy.

Appendix 2 Screenshots

The screenshots in this appendix show the Android demo application running on the Samsung Galaxy S24 Ultra and further views of the Unity scenes. The phone screens were captured from the installed application over USB with adb screencap while the phone drove it, so they show exactly what the device renders. Fig. 9.1 is the start screen. Fig. 9.2 has the debug view on: the rays, the current nectar target and the ten live observation values are drawn over the scene while the HB_16 bird feeds, with the beak alignment reading 0.89 and the distance to the nectar 0.02 m. Fig. 9.3 is the split-screen race a few seconds in, with the HB_16 model on the left already leading the HB_01 model on the right by 6.00 to 3.00 nectar, and both trails drawn. Fig. 9.4 and Fig. 9.5 are further views of the island rendered from the Unity project.

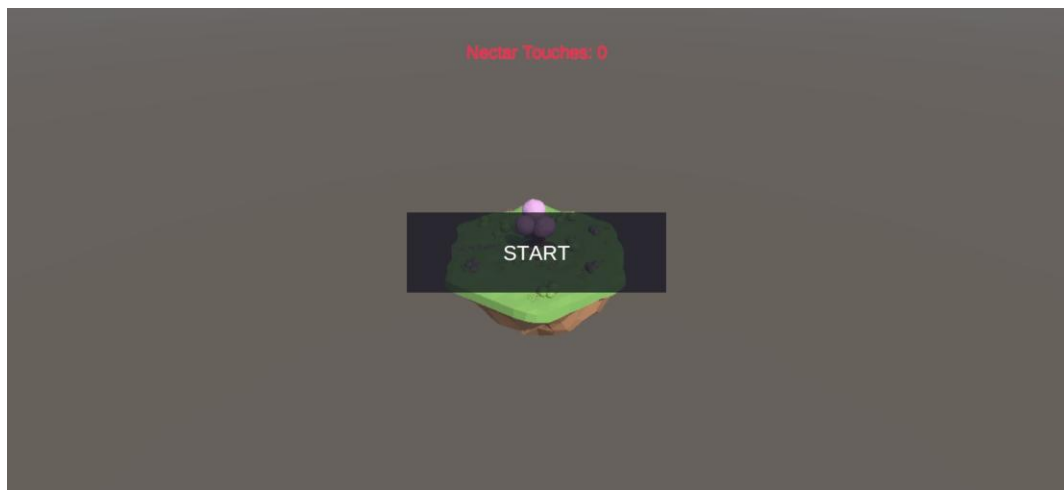


Fig. 9.1 Android demo on the Galaxy S24 Ultra, start screen

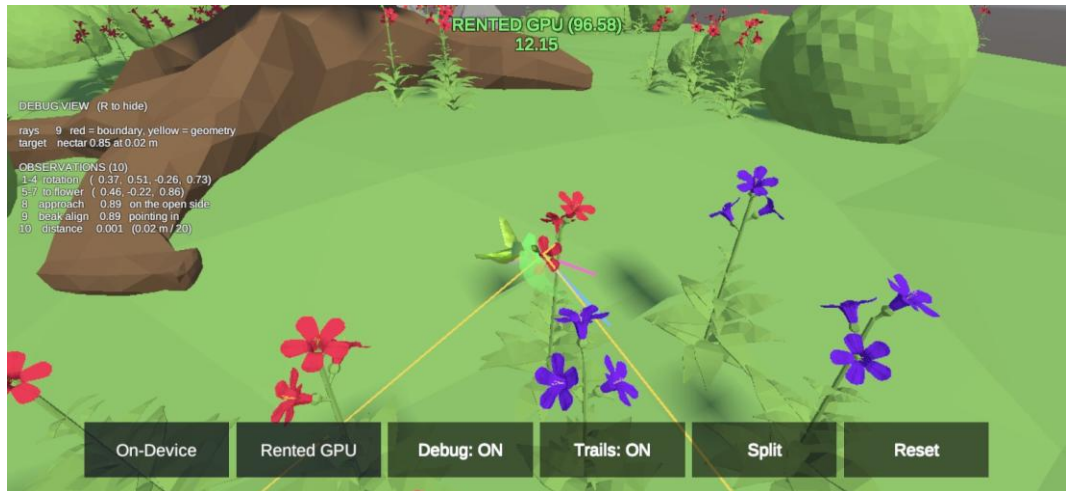


Fig. 9.2 Android demo with the debug view switched on (rays and the ten observations)

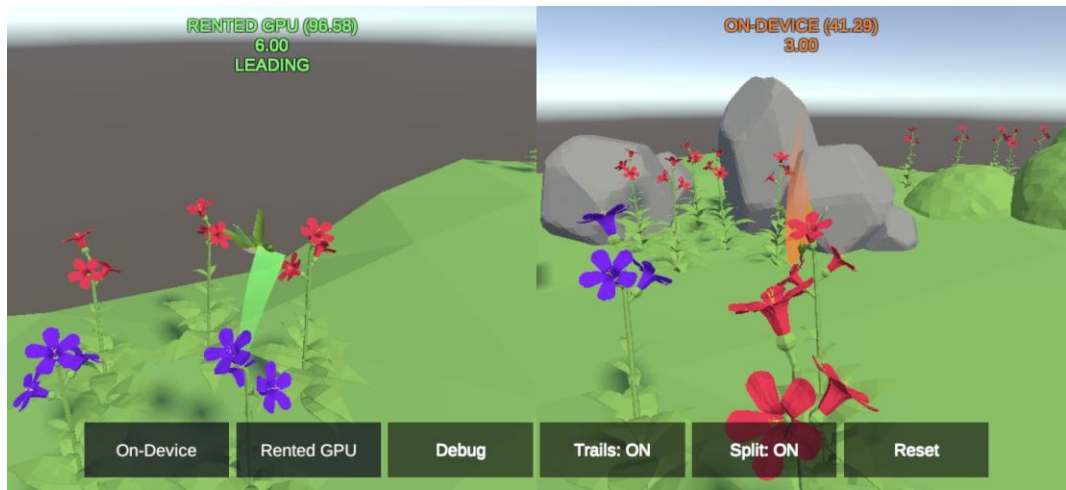


Fig. 9.3 Android demo split-screen race between the HB_16 and HB_01 models with trails

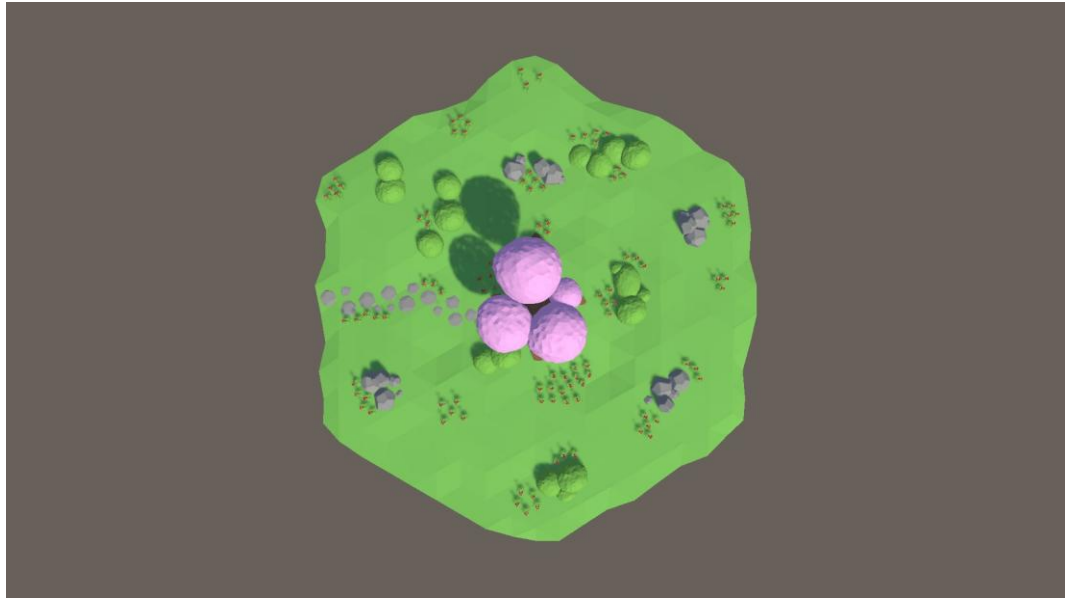


Fig. 9.4 Top view of the floating island, showing the flower plants spread across it



Fig. 9.5 Close view of a flower plant; each bloom carries a nectar trigger collider

Appendix 3 Sample Coding

This appendix reproduces the core scripts of the project exactly as they are in the Unity project, together with the HB_16 trainer configuration. The remaining scripts are described in Section 6.1.

A3.1 HummingbirdAgent.cs

```
using System.Collections;
using System.Collections.Generic;
```

```

using UnityEngine;
using Unity.MLAgents;
using Unity.MLAgents.Actuators;
using Unity.MLAgents.Sensors;

// A Hummingbird Machine Learning Agent
public class HummingbirdAgent : Agent
{
    [Tooltip("Force to apply when moving")]
    public float moveForce = 2f;

    [Tooltip("Speed to pitch up or down")]
    public float pitchSpeed = 100f;

    [Tooltip("Speed to rotate around the up axis")]
    public float yawSpeed = 100f;

    [Tooltip("Transform at the tip of the beak")]
    public Transform beakTip;

    [Tooltip("The agent's camera")]
    public Camera agentCamera;

    [Tooltip("Whether this is training mode or gameplay mode")]
    public bool trainingMode;

    // The rigidbody of the agent
    new private Rigidbody rigidbody;

    // The flower area that the agent is in
    private FlowerArea flowerArea;

    // The nearest flower to the agent
    private Flower nearestFlower;

    // Read-only view of the flower the agent is currently targeting, so the debug
    // visualizer can draw a line to the flower the agent actually chose rather than
    // re-deriving it and risking pointing at a different one.
    public Flower NearestFlower { get { return nearestFlower; } }

    // Allows for smoother pitch changes
    private float smoothPitchChange = 0f;

    // Allows for smoother yaw changes
    private float smoothYawChange = 0f;

    // Maximum angle that the bird can pitch up or down
    private const float MaxPitchAngle = 80f;

```

```

// Maximum distance from the beak tip to accept nectar collision
private const float BeakTipRadius = 0.008f;

// Whether the agent is frozen (intentionally not flying)
private bool frozen = false;

// The amount of nectar the agent has obtained this episode
public float NectarObtained { get; private set; }

// Initialize the agent
public override void Initialize()
{
    rigidbody = GetComponent<Rigidbody>();
    flowerArea = GetComponentInParent<FlowerArea>();

    // If not training mode, no max step, play forever
    if (!trainingMode) MaxStep = 0;
}

// Reset the agent when an episode begins
public override void OnEpisodeBegin()
{
    // Guard against ghost instances that Unity sometimes spins up outside any
    // real loaded scene (e.g. while generating a Project window preview
    // thumbnail for Hummingbird.prefab). Checking for an empty scene name,
    // not scene.IsValid() -- IsValid() is also true for these ghost/preview
    // scenes, since it only confirms the scene handle is structurally
    // registered with Unity, not that it's the real gameplay scene.
    if (string.IsNullOrEmpty(gameObject.scene.name))
    {
        return;
    }

    if (trainingMode)
    {
        // Only reset flowers in training when there is one agent per area
        flowerArea.ResetFlowers();
    }

    // Reset nectar obtained
    NectarObtained = 0f;

    // Zero out velocities so that movement stops before a new episode begins
    rigidbody.linearVelocity = Vector3.zero;
    rigidbody.angularVelocity = Vector3.zero;

    // Default to spawning in front of flower

```

```

    bool inFrontOfFlower = true;
    if (trainingMode)
    {
        // Spawn in front of flower 50% of the time during training
        inFrontOfFlower = UnityEngine.Random.value > 0.5f;
    }

    // Move the agent to a new random position
    MoveToSafeRandomPosition(inFrontOfFlower);

    // Recalculate the nearest flower now that the agent has moved
    UpdateNearestFlower();
}

// Called when an action is received from either the player input or the neural
network
//
// actionBuffers.ContinuousActions[i] represents:
// Index 0: move vector x (+1 = right, -1 = left)
// Index 1: move vector y (+1 = up, -1 = down)
// Index 2: move vector z (+1 = forward, -1 = backward)
// Index 3: pitch angle (+1 = pitch up, -1 = pitch down)
// Index 4: yaw angle (+1 = turn right, -1 = turn left)
public override void OnActionReceived(ActionBuffers actionBuffers)
{
    // Don't take actions if frozen
    if (frozen) return;

    // Calculate movement vector
    Vector3 move = new Vector3(actionBuffers.ContinuousActions[0],
actionBuffers.ContinuousActions[1], actionBuffers.ContinuousActions[2]);

    // Add force in the direction of the move vector
    rigidbody.AddForce(move * moveForce);

    // Get the current rotation
    Vector3 rotationVector = transform.rotation.eulerAngles;

    // Calculate pitch and yaw rotation
    float pitchChange = actionBuffers.ContinuousActions[3];
    float yawChange = actionBuffers.ContinuousActions[4];

    // Calculate smooth rotation changes
    smoothPitchChange = Mathf.MoveTowards(smoothPitchChange, pitchChange,
2f * Time.fixedDeltaTime);
    smoothYawChange = Mathf.MoveTowards(smoothYawChange, yawChange, 2f
* Time.fixedDeltaTime);

```

```

        // Calculate new pitch and yaw based on smoothed values
        // Clamp pitch to avoid flipping upside down
        float pitch = rotationVector.x + smoothPitchChange * Time.fixedDeltaTime *
pitchSpeed;
        if (pitch > 180f) pitch -= 360f;
        pitch = Mathf.Clamp(pitch, -MaxPitchAngle, MaxPitchAngle);

        float yaw = rotationVector.y + smoothYawChange * Time.fixedDeltaTime *
yawSpeed;

        // Apply the new rotation
        transform.rotation = Quaternion.Euler(pitch, yaw, 0f);
    }

    // Collect vector observations from the environment
    public override void CollectObservations(VectorSensor sensor)
    {
        // If nearestFlower is null, observe an empty array and return early
        if (nearestFlower == null)
        {
            sensor.AddObservation(new float[10]);
            return;
        }

        // Observe the agent's local rotation (4 observations)
        sensor.AddObservation(transform.localRotation.normalized);

        // Get a vector from the beak tip to the nearest flower
        Vector3 toFlower = nearestFlower.FlowerCenterPosition - beakTip.position;

        // Observe a normalized vector pointing to the nearest flower (3 observations)
        sensor.AddObservation(toFlower.normalized);

        // Observe a dot product that indicates whether the beak tip is in front of the
        flower (1 observation)
        // (+1 means directly in front, -1 means directly behind)
        sensor.AddObservation(Vector3.Dot(toFlower.normalized, -
nearestFlower.FlowerUpVector.normalized));

        // Observe a dot product that indicates whether the beak is pointing toward the
        flower (1 observation)
        // (+1 means directly toward, -1 means directly away)
        sensor.AddObservation(Vector3.Dot(beakTip.forward.normalized, -
nearestFlower.FlowerUpVector.normalized));

        // Observe the relative distance from the beak tip to the flower (1 observation)
        sensor.AddObservation(toFlower.magnitude / FlowerArea.AreaDiameter);

```

```

        // 10 total observations
    }

    // When Behavior Type is set to "Heuristic Only" on the agent's Behavior
    Parameters,
    // this function will be called. Its return values will be fed into
    // OnActionReceived() instead of using the neural network
    public override void Heuristic(in ActionBuffers actionsOut)
    {
        ActionSegment<float> continuousActionsOut = actionsOut.ContinuousActions;

        // Create placeholders for all movement/turning
        Vector3 forward = Vector3.zero;
        Vector3 left = Vector3.zero;
        Vector3 up = Vector3.zero;
        float pitch = 0f;
        float yaw = 0f;

        // Convert keyboard inputs to movement and turning
        // All values should be between -1 and +1

        // Forward/backward
        if (Input.GetKey(KeyCode.W)) forward = transform.forward;
        else if (Input.GetKey(KeyCode.S)) forward = -transform.forward;

        // Left/right
        if (Input.GetKey(KeyCode.A)) left = -transform.right;
        else if (Input.GetKey(KeyCode.D)) left = transform.right;

        // Up/down
        if (Input.GetKey(KeyCode.E)) up = transform.up;
        else if (Input.GetKey(KeyCode.C)) up = -transform.up;

        // Pitch up/down
        if (Input.GetKey(KeyCode.UpArrow)) pitch = 1f;
        else if (Input.GetKey(KeyCode.DownArrow)) pitch = -1f;

        // Turn left/right
        if (Input.GetKey(KeyCode.LeftArrow)) yaw = -1f;
        else if (Input.GetKey(KeyCode.RightArrow)) yaw = 1f;

        // Combine the movement vectors and normalize
        Vector3 combined = (forward + left + up).normalized;

        // Add the 3 movement values, pitch, and yaw to the actionsOut array
        continuousActionsOut[0] = combined.x;
        continuousActionsOut[1] = combined.y;
        continuousActionsOut[2] = combined.z;
    }

```

```

        continuousActionsOut[3] = pitch;
        continuousActionsOut[4] = yaw;
    }

    // Prevent the agent from moving and taking actions
    public void FreezeAgent()
    {
        if (string.IsNullOrEmpty(gameObject.scene.name) || rigidbody == null)
        {
            return;
        }
        Debug.Assert(trainingMode == false, "Freeze/Unfreeze not supported in training");
        frozen = true;
        rigidbody.Sleep();
    }

    // Resume agent movement and actions
    public void UnfreezeAgent()
    {
        if (string.IsNullOrEmpty(gameObject.scene.name) || rigidbody == null)
        {
            return;
        }
        Debug.Assert(trainingMode == false, "Freeze/Unfreeze not supported in training");
        frozen = false;
        rigidbody.WakeUp();
    }

    // Move the agent to a safe random position, i.e. does not collide with anything
    // If in front of flower, also point the beak at the flower
    // - inFrontOfFlower: whether to choose a spot in front of a flower
    private void MoveToSafeRandomPosition(bool inFrontOfFlower)
    {
        bool safePositionFound = false;
        int attemptsRemaining = 100; // Prevents an infinite loop
        Vector3 potentialPosition = Vector3.zero;
        Quaternion potentialRotation = new Quaternion();

        // Loop until a safe position is found or we run out of attempts
        while (!safePositionFound && attemptsRemaining > 0)
        {
            attemptsRemaining--;
            if (inFrontOfFlower)
            {
                // Pick a random flower

```

```

        Flower randomFlower =
flowerArea.Flowers[UnityEngine.Random.Range(0, flowerArea.Flowers.Count)];

        // Position 10 to 20 cm in front of the flower
        float distanceFromFlower = UnityEngine.Random.Range(0.1f, 0.2f);
        potentialPosition = randomFlower.transform.position +
randomFlower.FlowerUpVector * distanceFromFlower;

        // Point beak at flower (bird's head is center of transform)
        Vector3 toFlower = randomFlower.FlowerCenterPosition -
potentialPosition;
        potentialRotation = Quaternion.LookRotation(toFlower, Vector3.up);
    }
    else
    {
        // Pick a random height from the ground
        float height = UnityEngine.Random.Range(1.2f, 2.5f);

        // Pick a random radius from the center of the area
        float radius = UnityEngine.Random.Range(2f, 7f);

        // Pick a random direction rotated around the y axis
        Quaternion direction = Quaternion.Euler(0f, UnityEngine.Random.Range(-
180f, 180f), 0f);

        // Combine height, radius, and direction to pick a potential position
        potentialPosition = flowerArea.transform.position + Vector3.up * height +
direction * Vector3.forward * radius;

        // Choose and set random starting pitch and yaw
        float pitch = UnityEngine.Random.Range(-60f, 60f);
        float yaw = UnityEngine.Random.Range(-180f, 180f);
        potentialRotation = Quaternion.Euler(pitch, yaw, 0f);
    }

    // Check to see if the agent will collide with anything
    Collider[] colliders = Physics.OverlapSphere(potentialPosition, 0.05f);

    // Safe position has been found if no colliders are overlapped
    safePositionFound = colliders.Length == 0;
}

Debug.Assert(safePositionFound, "Could not find a safe position to spawn");

// Set the position and rotation
transform.position = potentialPosition;
transform.rotation = potentialRotation;
}

```

```

// Update the nearest flower to the agent
private void UpdateNearestFlower()
{
    foreach (Flower flower in flowerArea.Flowers)
    {
        if (nearestFlower == null && flower.HasNectar)
        {
            // No current nearest flower and this flower has nectar, so set to this flower
            nearestFlower = flower;
        }
        else if (flower.HasNectar)
        {
            // Calculate distance to this flower and distance to the current nearest flower
            float distanceToFlower = Vector3.Distance(flower.transform.position,
beakTip.position);
            float distanceToCurrentNearestFlower =
Vector3.Distance(nearestFlower.transform.position, beakTip.position);

            // If current nearest flower is empty, or this flower is closer, update nearest
flower
            if (!nearestFlower.HasNectar || distanceToFlower <
distanceToCurrentNearestFlower)
            {
                nearestFlower = flower;
            }
        }
    }
}

// Called when the agent's collider enters a trigger collider
// - other: the trigger collider
private void OnTriggerEnter(Collider other)
{
    TriggerEnterOrStay(other);
}

// Called when the agent's collider stays in a trigger collider
// - other: the trigger collider
private void OnTriggerStay(Collider other)
{
    TriggerEnterOrStay(other);
}

// Handles when the agent's collider enters or stays in a trigger collider
// - collider: the trigger collider
private void TriggerEnterOrStay(Collider collider)
{

```

```

// Check if agent is colliding with nectar
if (collider.CompareTag("nectar"))
{
    Vector3 closestPointToBeakTip = collider.ClosestPoint(beakTip.position);

    // Check if the closest collision point is close to the beak tip
    // Note: a collision with anything but the beak tip should not count
    if (Vector3.Distance(beakTip.position, closestPointToBeakTip) <
BeakTipRadius)
    {
        // Look up the flower for this nectar collider
        Flower flower = flowerArea.GetFlowerFromNectar(collider);

        // Attempt to take 0.01 nectar
        // Note: this is per fixed time step, meaning it happens every 0.02 seconds,
or 50x per second
        float nectarReceived = flower.Feed(0.01f);

        // Keep track of nectar obtained
        NectarObtained += nectarReceived;

        // Update the on-screen debug counter, if one exists in the scene
        if (nectarReceived > 0f && NectarCounterUI.Instance != null)
        {
            NectarCounterUI.Instance.RegisterTouch();
        }

        if (trainingMode)
        {
            // Calculate reward for getting nectar
            float bonus = 0.02f *
Mathf.Clamp01(Vector3.Dot(transform.forward.normalized, -
nearestFlower.FlowerUpVector.normalized));
            AddReward(0.01f + bonus);
        }

        // If flower is empty, update the nearest flower
        if (!flower.HasNectar)
        {
            UpdateNearestFlower();
        }
    }
}

// Called when the agent collides with something solid
// - collision: collision info
private void OnCollisionEnter(Collision collision)

```

```

    {
        if (trainingMode && collision.collider.CompareTag("boundary"))
        {
            // Collided with the area boundary, give a negative reward
            AddReward(-0.5f);
        }
    }

    // Called every frame
    private void Update()
    {
        // Draw a line from the beak tip to the nearest flower
        if (nearestFlower != null)
        {
            Debug.DrawLine(beakTip.position, nearestFlower.FlowerCenterPosition,
                Color.green);
        }
    }

    // Called every 0.02 seconds
    private void FixedUpdate()
    {
        // Avoids scenario where nearest flower nectar is stolen by opponent and not
        updated
        if (nearestFlower != null && !nearestFlower.HasNectar)
        {
            UpdateNearestFlower();
        }
    }
}

```

A3.2 Flower.cs

```

using UnityEngine;

/// <summary>
/// Manages a single flower with nectar
/// </summary>
public class Flower : MonoBehaviour
{
    [Tooltip("The color when the flower is full")]
    public Color fullFlowerColor = new Color(1f, 0f, .3f);

    [Tooltip("The color when the flower is empty")]
    public Color emptyFlowerColor = new Color(.5f, 0f, 1f);

    /// <summary>
    /// The trigger collider representing the nectar

```

```

/// </summary>
[HideInInspector]
public Collider nectarCollider;

// The solid collider representing the flower petals
private Collider flowerCollider;

// The flower's material
private Material flowerMaterial;

/// <summary>
/// A vector pointing straight out of the flower
/// </summary>
public Vector3 FlowerUpVector
{
    get { return nectarCollider.transform.up; }
}

/// <summary>
/// The center position of the nectar collider
/// </summary>
public Vector3 FlowerCenterPosition
{
    get { return nectarCollider.transform.position; }
}

/// <summary>
/// The amount of nectar remaining in the flower
/// </summary>
public float NectarAmount { get; private set; }

/// <summary>
/// Whether the flower has any nectar remaining
/// </summary>
public bool HasNectar
{
    get { return NectarAmount > 0f; }
}

/// <summary>
/// Attempts to remove nectar from the flower
/// </summary>
/// <param name="amount">The amount of nectar to remove</param>
/// <returns>The actual amount successfully removed</returns>
public float Feed(float amount)
{
    // Track how much nectar was successfully taken (cannot take more than is
    // available)

```

```

float nectarTaken = Mathf.Clamp(amount, 0f, NectarAmount);

// Subtract the nectar
NectarAmount -= amount;

if (NectarAmount <= 0)
{
    // No nectar remaining
    NectarAmount = 0;

    // Disable the flower and nectar colliders
    flowerCollider.gameObject.SetActive(false);
    nectarCollider.gameObject.SetActive(false);

    // Change the flower color to indicate that it is empty
    flowerMaterial.SetColor("_BaseColor", emptyFlowerColor);
}

// Return the amount of nectar that was taken
return nectarTaken;
}

/// <summary>
/// Resets the flower
/// </summary>
public void ResetFlower()
{
    // Refill the nectar
    NectarAmount = 1f;

    // Enable the flower and nectar colliders
    flowerCollider.gameObject.SetActive(true);
    nectarCollider.gameObject.SetActive(true);

    // Change the flower color to indicate that it is full
    flowerMaterial.SetColor("_BaseColor", fullFlowerColor);
}

/// <summary>
/// Called when the flower wakes up
/// </summary>
private void Awake()
{
    // Find the flower's mesh renderer and get the main material
    MeshRenderer meshRenderer = GetComponent<MeshRenderer>();
    flowerMaterial = meshRenderer.material;

    // Find flower and nectar colliders

```

```

        flowerCollider = transform.Find("FlowerCollider").GetComponent<Collider>();
        nectarCollider =
transform.Find("FlowerNectarCollider").GetComponent<Collider>();
    }
}

```

A3.3 FlowerArea.cs

```

using System.Collections.Generic;
using UnityEngine;

/// <summary>
/// Manages a collection of flower plants and attached flowers
/// </summary>
public class FlowerArea : MonoBehaviour
{
    // The diameter of the area where the agent and flowers can be used
    // for observing relative distance from agent to flower
    public const float AreaDiameter = 20f;

    // These three are created here at field-initializer time rather than in Awake(),
    because
    // Awake() is not early enough on every path. When an island is Instantiated at
    runtime (the
    // race demo, and IslandSpawner on the training server) Unity interleaves Awake
    and OnEnable
    // as it walks the new hierarchy, so the agent's
    // OnEnable -> LazyInitialize -> OnEpisodeBegin -> ResetFlowers chain can reach
    // FindChildFlowers() before this component's own Awake() has run, and every
    collection below
    // is still null -- a NullReferenceException, thrown before a single flower is
    registered.
    // Field initializers run when the component is constructed, which is ahead of
    Awake and
    // OnEnable on every path, so the race cannot happen at all.

    // The list of all flower plants in this flower area (flower plants have multiple
    flowers)
    private List<GameObject> flowerPlants = new List<GameObject>();

    // A lookup dictionary for looking up a flower from a nectar collider
    private Dictionary<Collider, Flower> nectarFlowerDictionary = new
    Dictionary<Collider, Flower>();

    // Backing field for the Flowers property, and a guard so FindChildFlowers()
    // only ever runs once, no matter when it's first triggered.
    private List<Flower> flowers = new List<Flower>();
    private bool flowersFound = false;

```

```

// The list of all flowers in the flower area.
// Lazily found on first access (see EnsureFlowersFound) rather than in Awake()
// or Start(), because relying on either creates a race condition: populating in
// Start() isn't guaranteed to run before other scripts' Start() (e.g.
// GameManager could ask for a flower before this list exists), while populating
// in Awake() isn't guaranteed to run after every Flower's own Awake() has set
// its nectarCollider (causing duplicate-null dictionary keys). First access is
// always safe because every real caller queries this from a Start()-phase
// method or later, by which point every object's Awake() has already run.
public List<Flower> Flowers
{
    get
    {
        EnsureFlowersFound();
        return flowers;
    }
}

/// <summary>
/// Reset the flowers and flower plants
/// </summary>
public void ResetFlowers()
{
    EnsureFlowersFound();

    // Rotate each flower plant around the Y axis and subtly around X and Z
    foreach (GameObject flowerPlant in flowerPlants)
    {
        float xRotation = UnityEngine.Random.Range(-5f, 5f);
        float yRotation = UnityEngine.Random.Range(-180f, 180f);
        float zRotation = UnityEngine.Random.Range(-5f, 5f);
        flowerPlant.transform.localRotation = Quaternion.Euler(xRotation, yRotation,
zRotation);
    }

    // Reset each flower
    foreach (Flower flower in flowers)
    {
        flower.ResetFlower();
    }
}

/// <summary>
/// Gets the <see cref="Flower"/> that a nectar collider belongs to
/// </summary>
/// <param name="collider">The nectar collider</param>
/// <returns>The matching flower</returns>

```

```

public Flower GetFlowerFromNectar(Collider collider)
{
    EnsureFlowersFound();
    return nectarFlowerDictionary[collider];
}

// Awake() used to create the three collections above. It has been removed rather
// than left
// empty, and must not come back: EnsureFlowersFound() can legitimately run
// before Awake() on a
// runtime-instantiated island, so re-assigning fresh collections in Awake() would
// silently
// discard every flower already registered -- an empty arena with no error, which is a
// worse
// failure than the NullReferenceException this replaced. The field initializers are
// the only
// place these should be created.

/// <summary>
/// Finds all flowers exactly once, on whichever comes first: this object's own
/// Start(), or an external caller asking for Flowers/ResetFlowers/
/// GetFlowerFromNectar before that. See the comment on the Flowers property for
/// why this can't safely live in Awake() or unconditionally in Start().
/// </summary>
private void EnsureFlowersFound()
{
    if (flowersFound) return;
    flowersFound = true;
    FindChildFlowers(transform);
}

/// <summary>
/// Called when the game starts
/// </summary>
private void Start()
{
    EnsureFlowersFound();
}

/// <summary>
/// Recursively finds all flowers and flower plants that are children of a parent
// transform
/// </summary>
/// <param name="parent">The parent of the children to check</param>
private void FindChildFlowers(Transform parent)
{
    for (int i = 0; i < parent.childCount; i++)
    {

```

```

    Transform child = parent.GetChild(i);

    if (child.CompareTag("flower_plant"))
    {
        // Found a flower plant, add it to the flowerPlants list
        flowerPlants.Add(child.gameObject);

        // Look for flowers within the flower plant
        FindChildFlowers(child);
    }
    else
    {
        // Not a flower plant, look for a Flower component
        Flower flower = child.GetComponent<Flower>();
        if (flower != null)
        {
            // Found a flower, add it to the flowers list
            flowers.Add(flower);

            // Add the nectar collider to the lookup dictionary
            nectarFlowerDictionary.Add(flower.nectarCollider, flower);

            // Note: there are no flowers that are children of other flowers
        }
        else
        {
            // Flower component not found, so check children
            FindChildFlowers(child);
        }
    }
}
}
}

```

A3.4 IslandSpawner.cs

```

using UnityEngine;

/// <summary>
/// Builds the training grid at runtime by cloning the FloatingIsland prefab into
/// a roughly square grid as soon as the scene loads.
///
/// Why this is done in code instead of simply placing N islands in Training.unity:
/// one island is ~3,800 GameObjects, so a 28-island scene would be ~107,000 of
/// them.
/// That is heavy to author on a low-RAM machine, it bloats the build, and - the
/// part that actually matters - it bakes the island count into the build. Spawning
/// instead makes the count a launch flag, so the question "how many islands does

```

```

/// this rented box actually fit?" is answered by re-running the same build with a
/// different number, rather than rebuilding and re-uploading it for every guess.
///
/// mlagents-learn passes everything after --env-args straight through to the
/// player, so the count is set like this:
///   mlagents-learn ... --env-args -logFile out.log --islands 28
/// </summary>
public class IslandSpawner : MonoBehaviour
{
    // The island to clone. Assign to
    Assets/Hummingbird/Prefabs/FloatingIsland.prefab.
    public GameObject islandPrefab;

    // Used when no --islands flag was passed, e.g. when pressing Play in the Editor.
    // Kept at 1 so opening the scene by hand stays light on this machine.
    public int defaultIslandCount = 1;

    // Centre-to-centre gap between islands. FlowerArea.AreaDiameter is 20f and the
    // island's visible geometry measures ~19.5 units across, so 25 leaves a real
    // margin and keeps one island's colliders clear of its neighbour's.
    public float spacing = 25f;

    // The island already present in Training.unity. It is reused as the first grid
    // slot instead of being destroyed, so a 1-island run allocates nothing new.
    // Wired explicitly in the Inspector rather than looked up by name, because once
    // clones exist there are several similarly-named objects and GameObject.Find
    // gives no ordering guarantee about which one it returns.
    public GameObject existingIsland;

    private void Awake()
    {
        if (islandPrefab == null)
        {
            Debug.LogError("IslandSpawner: islandPrefab is not assigned, no islands will
be spawned.");
            return;
        }

        int count = Mathf.Max(1,
ReadIslandCountFromCommandLine(defaultIslandCount));

        int columns = Mathf.CeilToInt(Mathf.Sqrt(count));
        int rows = Mathf.CeilToInt(count / (float)columns);

        // Centre the grid on the origin so the scene camera still frames it.
        float xOffset = (columns - 1) * spacing * 0.5f;
        float zOffset = (rows - 1) * spacing * 0.5f;

```

```

for (int i = 0; i < count; i++)
{
    Vector3 position = new Vector3(
        (i % columns) * spacing - xOffset,
        0f,
        (i / columns) * spacing - zOffset);

    if (i == 0 && existingIsland != null)
    {
        existingIsland.transform.position = position;
        continue;
    }

    GameObject clone = Instantiate(islandPrefab, position, Quaternion.identity);
    clone.name = "FloatingIsland_" + i;
}

// Logged so the Unity -logFile on the rented box confirms what actually ran.
// Without this there is no way to tell a working --islands flag from one that
// was silently ignored, and the whole point of the run is measuring the count.
Debug.Log("IslandSpawner: spawned " + count + " islands in a " + columns +
"x" + rows +
" grid at " + spacing + "-unit spacing.");
}

/// <summary>
/// Reads "--islands N" from the process command line, falling back to the
/// Inspector value when the flag is absent or unparseable.
/// </summary>
private int ReadIslandCountFromCommandLine(int fallback)
{
    string[] args = System.Environment.GetCommandLineArgs();

    // Stop one short of the end: a trailing "--islands" has no value after it.
    for (int i = 0; i < args.Length - 1; i++)
    {
        if (args[i] != "--islands" && args[i] != "-islands") continue;

        int parsed;
        if (int.TryParse(args[i + 1], out parsed) && parsed > 0) return parsed;

        Debug.LogWarning("IslandSpawner: could not read an island count from '" +
            args[i + 1] + "', falling back to " + fallback + ".");
    }

    return fallback;
}
}

```

A3.5 DemoController.cs

```
using Unity.InferenceEngine;
using Unity.MLAgents;
using Unity.MLAgents.Policies;
using UnityEngine;
using UnityEngine.UI;

/// <summary>
/// Drives the phone demo: a Start button, then live switching between the two trained
/// models while the bird is flying.
///
/// The whole reason this class exists is that the two models are NOT interchangeable.
/// They were trained with different observation stacking, so each one only accepts a
/// specific input width:
///
///   RentedGPU_Final (HB_16) obs_3 [batch, 30] -> Stacked Vectors 3
///   OnDevice_Final (HB_01) obs_3 [batch, 10] -> Stacked Vectors 1
///
/// ML-Agents builds its sensors once, in Agent.InitializeSensors() during
/// LazyInitialize(), and assigning BehaviorParameters.Model does not rebuild them.
/// Setting the fields on a live agent therefore leaves the OLD sensor in place feeding
/// the new model the wrong number of observations - and ML-Agents logs no error at
/// all
/// when that happens, so it looks like the model itself is broken.
///
/// Agent.OnDisable() calls CleanupSensors() and sets m_Initialized = false, so a
/// disable -> change -> enable cycle does rebuild them. That is what SelectModel
/// does,
/// and it is why the two fields are only ever written inside that one method: there is
/// no code path here that can set a model without also setting its matching stacking.
/// </summary>
public class DemoController : MonoBehaviour
{
    [Header("Agent")]
    [Tooltip("The Hummingbird GameObject. Leave it DISABLED in the scene - it is
    enabled on Start.")]
    [SerializeField] private GameObject bird;

    [Header("Models (each is locked to its own Stacked Vectors value)")]
    [Tooltip("HB_01 baseline, mean reward 41.29 - requires Stacked Vectors 1")]
    [SerializeField] private ModelAsset onDeviceModel;

    [Tooltip("HB_16 rented-GPU model, mean reward 96.58 - requires Stacked
    Vectors 3")]
    [SerializeField] private ModelAsset rentedGpuModel;
```

```

[Header("UI")]
[SerializeField] private GameObject startPanel;
[SerializeField] private GameObject inGamePanel;
[SerializeField] private Text modelLabel;

/// <summary>
/// The bird this controller owns. Exposed so DemoToggles' Reset can restart every
*other*
/// bird's episode without restarting this one twice, since SelectModel already ends
its own.
/// </summary>
public GameObject Bird { get { return bird; } }

private BehaviorParameters behaviourParameters;

private void Awake()
{
    behaviourParameters = bird.GetComponent<BehaviorParameters>();

    // Belt and braces: the bird should already be disabled in the scene, but if it
    // is not, disabling it here still happens before it can build any sensors,
    // because a disabled GameObject's Awake/OnEnable never ran in the first place.
    bird.SetActive(false);

    if (startPanel != null) startPanel.SetActive(true);
    if (inGamePanel != null) inGamePanel.SetActive(false);
}

/// <summary>Hooked to the Start button.</summary>
public void OnStartPressed()
{
    if (startPanel != null) startPanel.SetActive(false);
    if (inGamePanel != null) inGamePanel.SetActive(true);

    // Open on the better model, so the first thing anyone sees is the good one.
    SelectModel(true);
}

/// <summary>Hooked to the "Rented GPU" button.</summary>
public void UseRentedGpu()
{
    SelectModel(true);
}

/// <summary>Hooked to the "On-Device" button.</summary>
public void UseOnDevice()
{
    SelectModel(false);
}

```

```

}

/// <summary>
/// The only place either field is written. Both always change together, inside the
/// disable/enable cycle that forces ML-Agents to rebuild the observation sensors.
/// </summary>
public void SelectModel(bool rented)
{
    bird.SetActive(false); // OnDisable -> CleanupSensors(), m_Initialized = false

    behaviourParameters.BrainParameters.NumStackedVectorObservations =
rented ? 3 : 1;
    behaviourParameters.Model = rented ? rentedGpuModel : onDeviceModel;

    bird.SetActive(true); // OnEnable -> LazyInitialize() -> InitializeSensors()

    // Respawn rather than resuming wherever the bird happened to be left, so each
    // model is judged from a comparable starting point.
    bird.GetComponent<Agent>().EndEpisode();

    if (modelLabel != null)
    {
        modelLabel.text = rented
            ? "Rented GPU - trained reward 96.58"
            : "On-Device - trained reward 41.29";
    }
}
}

```

A3.6 trainer_config_16.yaml (settings section)

```

behaviors:
  Hummingbird:
    trainer_type: ppo

hyperparameters:
  # 16 agents fill the buffer instead of 1, so each update can afford to be
  # bigger and less noisy. Ratio buffer:batch stays at 10:1, as before.
  batch_size: 4096 # HB_01: 2048
  buffer_size: 40960 # HB_01: 20480

  learning_rate: 3.0e-4
  learning_rate_schedule: linear # decays to 0 at max_steps -- see note below

  # beta = entropy bonus = "how much random exploration".
  # Annealing it means the policy is free to explore early and gets to commit to
  # precise, low-jitter hovering late, which is exactly what this task needs.
  # (linear is already the ML-Agents default for these two; stated explicitly

```

```

# here so the report can quote the actual schedule.)
beta: 5.0e-3
beta_schedule: linear
epsilon: 0.2
epsilon_schedule: linear

lambda: 0.95
num_epoch: 3

network_settings:
# HB_01 had this off. Turn it on: of the 10 observations, nine are already
# in the -1..1 range (a quaternion, a unit vector, two dot products) but
# `toFlower.magnitude / 20` is not -- in practice it sits around 0.01-0.1,
# so its gradient signal is ~50x weaker than everything else. Normalising
# puts them all on equal footing.
normalize: true # HB_01: false
hidden_units: 256
num_layers: 2
vis_encode_type: simple

reward_signals:
  extrinsic:
    strength: 1.0
    gamma: 0.99 # ~10 s of sim lookahead: enough to plan 2-3 flowers ahead

# IMPORTANT: pick this number BEFORE starting the real run and do not change
it
# mid-run. The learning rate ramps linearly down to zero across exactly this
# many steps, so stopping early leaves the policy under-annealed (still jittery),
# and setting it far too high means the learning rate never gets low enough to
# fine-tune. Run the smoke test first (`.train_16.sh smoke`), read the measured
# steps/sec, and set this to whatever your rental budget actually covers.
# 30,000,000 steps at ~3040 steps/s ~= 2.7 h ~= Rs. 117 on the L4
# 50,000,000 steps at ~3040 steps/s ~= 4.6 h ~= Rs. 197
# (L4 all-in = Rs. 42.61/hr: Rs. 41.31 compute + Rs. 1.30 for 100GB storage.
# ~3040 steps/s is the MEASURED rate from the 2026-08-31 sweep at the
# fastest layout, ISLANDS=28 NUM_ENVS=4 -- see project memory /
ARCHITECTURE.md.)
max_steps: 50000000 # HB_01: 5000000

time_horizon: 128
summary_freq: 50000 # ~50 finished episodes per printed line = a smooth curve

# A rented box can die or be interrupted. Checkpoint often enough that losing
# one is cheap. At the measured ~3040 steps/s this is a checkpoint roughly
# every 82 seconds, and 40 kept covers the last ~55 minutes of history.
checkpoint_interval: 250000
keep_checkpoints: 40

```

```
# false = strict on-policy PPO (same as HB_01). Setting it true overlaps the
# gradient update with environment stepping -- maybe 20% faster wall-clock, but
# it slightly breaks PPO's on-policy assumption. We are optimising for the
# highest final reward, not the fastest run, so leave it off.
```

```
threaded: false
```

```
env_settings:
```

```
# Number of separate Unity *processes*, each running a full grid of islands.
# Start at 1. Unity's script/main thread is single-threaded, so on a 28-vCPU box
# 2 processes may well be faster than 1 -- measure it during the smoke test rather
# than guessing. (Overridden by --num-envs on the command line.)
```

```
num_envs: 1
```

```
# Every island is ~3,800 GameObjects and 360 flowers, so a 28-island grid is
# ~107,000 objects and ~10,000 flowers to build at startup -- far longer than the
# default 60 s. Without this the trainer gives up before Unity finishes loading.
# Raise it further if a bigger island count ever times out during the sweep.
```

```
timeout_wait: 600
```

```
engine_settings:
```

```
time_scale: 20
```

```
# MUST be false. Measured on the L4 box 2026-08-31: this build segfaults
# (SIGSEGV in PlayerMain, immediately after the scene loads) whenever it runs
with
```

```
# -nographics, because Unity 6's Null graphics device cannot serve this URP
# project. Under a real display it runs fine. train_16.sh therefore starts a tiny
# Xvfb virtual display and leaves graphics enabled; setting this back to true
# reintroduces the crash.
```

```
no_graphics: false # headless server, nothing to draw to
```

Appendix 4 User Documentation

i) Installation Instructions

Opening and running the project in the Unity Editor:

- Install Unity Hub and Unity Editor 6000.4.7f1. Add the Android Build Support module if the phone application is to be rebuilt, and Linux Build Support (Mono) if the server build is to be rebuilt.
- In Unity Hub choose Add and select the project folder, the folder that contains Assets and ProjectSettings.
- Open the project. The packages com.unity.ml-agents 4.1.0 and the Inference Engine are restored automatically from Packages/manifest.json.

- Open Assets/Hummingbird/Scenes/Training.unity and press Play. The bird flies under the RentedGPU_Final model with no Python process needed.

Setting up the training environment on a Windows laptop:

- Install Anaconda and create an environment with Python 3.10.12: `conda create -n ml-agents-1.0 python=3.10.12`.
- Activate it and install the trainer: `pip install mlagents==1.1.0`, then a CPU build of PyTorch.
- Keep setuptools below version 81, because mlagents still imports pkg_resources.
- Start training from the Training folder with `mlagents-learn config/trainer_config.yaml --run-id=<name>`, then press Play in the Editor when prompted.

Setting up a rented Linux server:

- Rent an instance with as many vCPUs as possible and set storage to 100 GB explicitly.
- Upload the whole Training folder to /home (never /root, which the provider wipes).
- Run `bash /home/Training/setup_server.sh` once to build the pinned environment.
- Run `./train_16.sh smoke`, then `./train_16.sh sweep`, then `./train_16.sh` for the real run, preferably inside tmux.

Installing the Android application:

- Enable Developer options and USB debugging on the phone and connect it by USB.
- Install with the adb bundled in Unity's Android module: `adb install -r HummingbirdDemo.apk`.
- Launch Hummingbird AI from the app drawer. The application is landscape only and needs no network connection.

ii) README and Instructions for Using the System

In the Editor, the Training scene runs the trained model in inference. The keys below control the presentation toolkit; in the Flower Island scene the Start button begins

a 60-second match between a keyboard-controlled bird and the trained opponent, and the first bird to collect 8 units of nectar, or the one with more when time runs out, wins. The controls are listed in Table 9.1 and the phone buttons in Table 9.2.

Table 9.1 Controls in the Unity Editor

Key	Action
W / S	Move forward / backward (heuristic control)
A / D	Move left / right
E / C	Move up / down
Arrow up / down	Pitch up / down
Arrow left / right	Turn left / right
R	Debug view: draw the 9 rays and print the 10 observations
T	Start or stop the split-screen race between the two models
Y	Show or hide flight trails
Mouse drag / wheel	Orbit and zoom the chase camera

Table 9.2 Buttons in the Android application

Button	Action
START	Enables the bird with the Rented GPU model
On-Device	Switches to the HB_01 model (reward 41.29), rebuilding the sensors
Rented GPU	Switches to the HB_16 model (reward 96.58), rebuilding the sensors
Debug	Shows the rays and the live observation values
Trails	Shows or hides the fading flight ribbon
Split	Races both models side by side on two islands with a scoreboard
Reset	Returns camera, toggles, model and scores to the opening state
Drag / pinch / double-tap	Orbit, zoom and reset the camera under the finger

Training commands on the rented server, all run from /home/Training:

```
./train_16.sh smoke    # about 10 minutes: does it run at all
./train_16.sh sweep    # about 25 minutes: which process/island split is fastest
./train_16.sh          # the real run, resumes automatically if interrupted
./train_16.sh status    # from a second terminal: steps/s, ETA, cost, reward vs 41.29
```

Rebuilding the phone application from a Windows command prompt, with the Unity Editor closed:

```
Unity.exe -quit -batchmode -projectPath "C:\Users\rohit\Machine Learning" -
buildTarget Android
```

-executeMethod BuildAndroid.Build -buildOutput HummingbirdDemo.apk -logFile build_android.log
(then search the log for [BuildAndroid] SUCCESS; the exit code is not reliable)

Appendix 5 Glossary

Table 9.3 defines the technical terms used in this report, in alphabetical order.

Table 9.3 Glossary of terms used in the report

Term	Meaning
Ablation	An experiment that removes or changes one factor at a time to measure its individual effect.
Action	The output of the policy at each decision; here five continuous values controlling force, pitch and yaw.
Actor-critic	A method with two parts: an actor (the policy) that chooses actions and a critic that estimates value.
Advantage	How much better an action turned out than the value estimate expected; the quantity PPO pushes up.
Agent	The learning entity; here the hummingbird controlled by HummingbirdAgent.cs.
Arena	One floating island with its own bird and flowers; HB_16 trained on 112 at once.
Batch mode	Running the Unity Editor or player from the command line without its window.
Behavior Parameters	The ML-Agents component that fixes observation size, action size, stacking and the model.
Buffer size	The number of experience steps collected before each round of PPO updates.
Checkpoint	A saved copy of the trainer's network and optimiser state at a given step.
Clipped objective	PPO's rule that limits how far one update may change the probability of an action.
Cumulative reward	The total reward an agent earns in one episode; the main performance measure here.
Decision Requester	The component that asks the policy for a new action every N physics steps (N = 5 here).
Entropy	A measure of how random the policy's actions are; high early in training, lower as it commits.
Episode	One attempt from reset to end; here 5,000 physics steps, or 100 simulated seconds.
Epoch	One pass of updates over the collected buffer; PPO here uses three per buffer.
GAE	Generalized Advantage Estimation, a method of computing advantages that balances bias and variance.

Term	Meaning
Gamma	The discount factor that sets how much future reward counts (0.99 here).
gRPC	The remote procedure call protocol used between Unity and the Python trainer.
Heuristic	A hand-coded action source; here keyboard control of the same agent.
IL2CPP	Unity's backend that converts C# to C++ for native compilation, required for ARM64 Android.
Inference	Running a trained network to choose actions, without further training.
Inference Engine	Unity's runtime for ONNX networks, previously called Sentis and before that Barracuda.
Learning rate	The step size of each network update; decays linearly to zero in both runs.
Linear damping	A Rigidbody setting that slows motion over time; 2 for the bird, giving it momentum that fades.
llvmpipe	A software OpenGL renderer on Linux; without -batchmode the server build rendered every frame through it.
max_steps	The step budget of a training run; also the point where the linear schedules reach zero.
mlagents-learn	The command-line trainer of the ML-Agents Python package.
Normalisation	Rescaling each input to zero mean and unit variance using running statistics.
Observation	What the agent perceives at each decision; here 46 or 66 numbers.
On-policy	A method that learns only from experience collected by the current policy, as PPO does.
ONNX	Open Neural Network Exchange, a portable file format for trained networks.
Policy	The mapping from observations to actions that the network represents.
PPO	Proximal Policy Optimization, the on-policy actor-critic algorithm used for training.
Prefab	A reusable Unity object template; the floating island is one prefab cloned for every arena.
Quaternion	A four-number representation of a 3D rotation, used for the bird's orientation observation.
Ray Perception Sensor	An ML-Agents sensor that casts rays and reports what they hit and how far away.
Reward	The scalar signal the agent maximises; here nectar drinking and a boundary penalty.
Rigidbody	Unity's physics component that gives the bird mass, velocity and damping.
Stacked vectors	Feeding the network the last N observation vectors together; N = 3 in HB_16.

Term	Meaning
Step	One decision of one agent as counted by the trainer; five physics steps.
Swish	The activation function x times $\text{sigmoid}(x)$ used in the hidden layers.
Throughput	Training steps completed per second of wall-clock time.
Time horizon	The number of steps collected per agent before a trajectory is sent to the buffer (128).
Trigger collider	A collider that detects overlap without blocking movement; each flower's nectar is one.
URP	Universal Render Pipeline, the Unity rendering setup used by the project.
Value estimate	The critic's prediction of the reward still to come from the current state.
vCPU	A virtual CPU core on a rented server; the resource that limited training speed here.
Xvfb	X virtual framebuffer, a display server with no screen, used to run Unity on the server.

CHAPTER 10

BIBLIOGRAPHY / REFERENCES

- [1]. Bengio, Y., J. Louradour, R. Collobert, and J. Weston. "Curriculum Learning." Proceedings of the 26th International Conference on Machine Learning (ICML), 2009, pp. 41-48.
- [2]. Berner, C., G. Brockman, B. Chan, V. Cheung, P. Dębiak, C. Dennison, et al. "Dota 2 with Large Scale Deep Reinforcement Learning." arXiv preprint arXiv:1912.06680, 2019.
- [3]. Brockman, G., V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba. "OpenAI Gym." arXiv preprint arXiv:1606.01540, 2016.
- [4]. Colledanchise, M., and P. Ögren. Behavior Trees in Robotics and AI: An Introduction. CRC Press, 2018.
- [5]. gunjanmimo. "Hummingbirds-Unity-ML." GitHub.
<https://github.com/gunjanmimo/Hummingbirds-Unity-ML>. Accessed 28 August 2026.
- [6]. Juliani, A., V.-P. Berges, E. Teng, A. Cohen, J. Harper, C. Elion, C. Goy, Y. Gao, H. Henry, M. Mattar, and D. Lange. "Unity: A General Platform for Intelligent Agents." arXiv preprint arXiv:1809.02627, 2020.
- [7]. Kelly, A. (Immersive Limit). "ML-Agents: Hummingbirds." Unity Learn, Unity Technologies. <https://learn.unity.com/>. Accessed 26 August 2026.
- [8]. Lowe, R., Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch. "Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments." Advances in Neural Information Processing Systems (NeurIPS), 2017.
- [9]. Mnih, V., K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, et al. "Human-Level Control Through Deep Reinforcement Learning." Nature, vol. 518, no. 7540, 2015, pp. 529-533.
- [10]. Mnih, V., A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu. "Asynchronous Methods for Deep Reinforcement Learning."

- Proceedings of the 33rd International Conference on Machine Learning (ICML), 2016, pp. 1928-1937.
- [11]. Ng, A. Y., D. Harada, and S. Russell. "Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping." Proceedings of the 16th International Conference on Machine Learning (ICML), 1999, pp. 278-287.
 - [12]. Schulman, J., S. Levine, P. Moritz, M. I. Jordan, and P. Abbeel. "Trust Region Policy Optimization." Proceedings of the 32nd International Conference on Machine Learning (ICML), 2015, pp. 1889-1897.
 - [13]. Schulman, J., P. Moritz, S. Levine, M. Jordan, and P. Abbeel. "High-Dimensional Continuous Control Using Generalized Advantage Estimation." International Conference on Learning Representations (ICLR), 2016.
 - [14]. Schulman, J., F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. "Proximal Policy Optimization Algorithms." arXiv preprint arXiv:1707.06347, 2017.
 - [15]. Silver, D., A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, et al. "Mastering the Game of Go with Deep Neural Networks and Tree Search." Nature, vol. 529, no. 7587, 2016, pp. 484-489.
 - [16]. Sutton, R. S., and A. G. Barto. Reinforcement Learning: An Introduction. 2nd ed., MIT Press, 2018.
 - [17]. Unity Technologies. "Unity ML-Agents Toolkit Documentation (package version 4.1.0)." GitHub. <https://github.com/Unity-Technologies/ml-agents>. Accessed 31 August 2026.
 - [18]. Vinyals, O., I. Babuschkin, W. M. Czarnecki, M. Mathieu, A. Dudzik, J. Chung, et al. "Grandmaster Level in StarCraft II Using Multi-Agent Reinforcement Learning." Nature, vol. 575, no. 7782, 2019, pp. 350-354.
 - [19]. Yannakakis, G. N., and J. Togelius. Artificial Intelligence and Games. Springer, 2018.